Universidad Zaragoza
1542

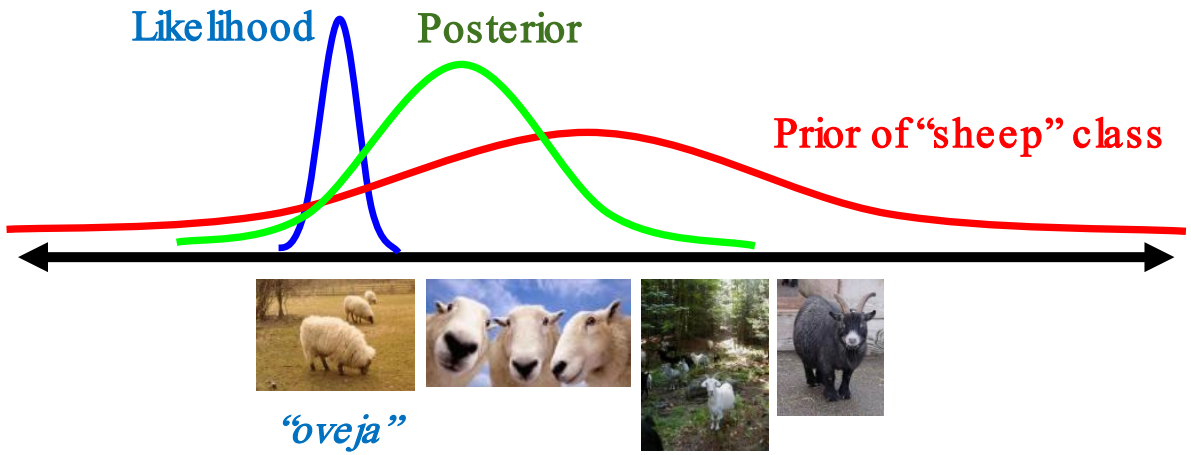
Bayesian deep learning

Applications of Deep Learning (69158)

Rubén Martínez Cantín
Dpto. Informática e Ingeniería de Sistemas.

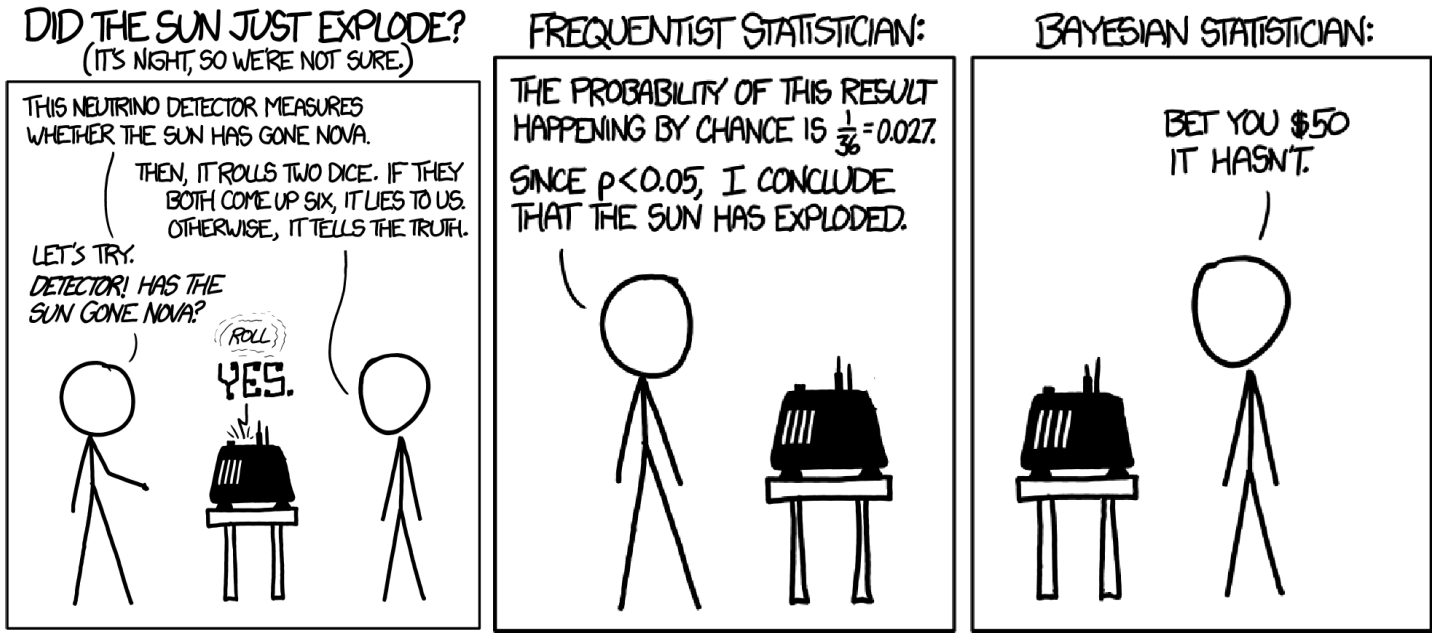
Bayesian learning

$$p(h \mid d) = \frac{p(d \mid h)\,p(h)}{\sum_{h' \in H} p(d \mid h')\,p(h')}$$



Images: Wikipedia, Nando de Freitas.

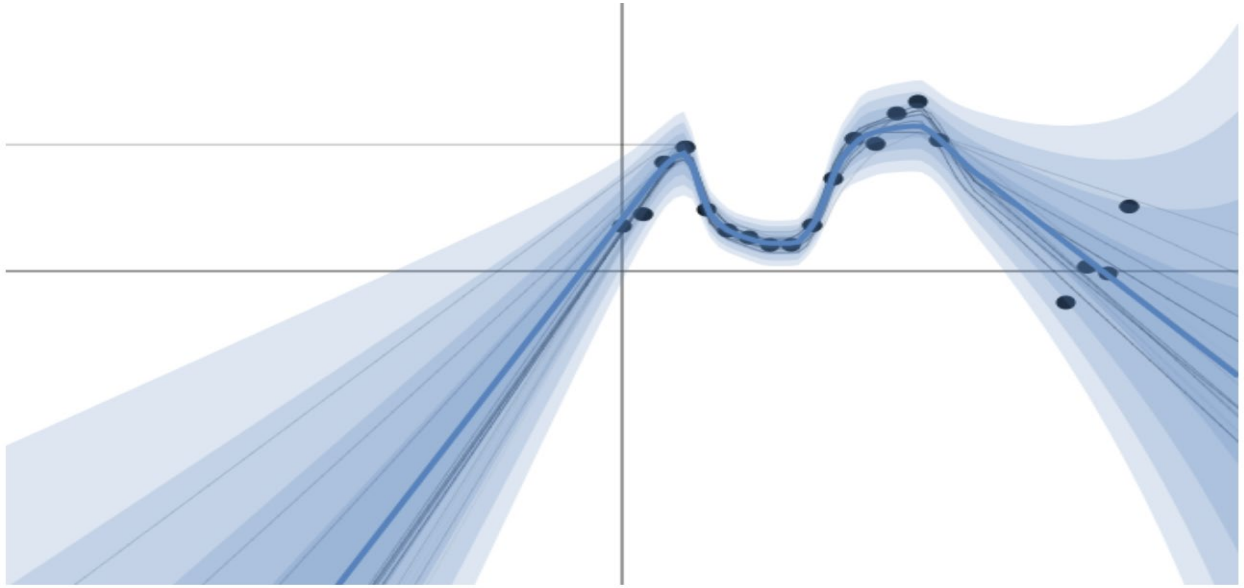
Bayesian learning



https://xkcd.com/1132/

Motivation

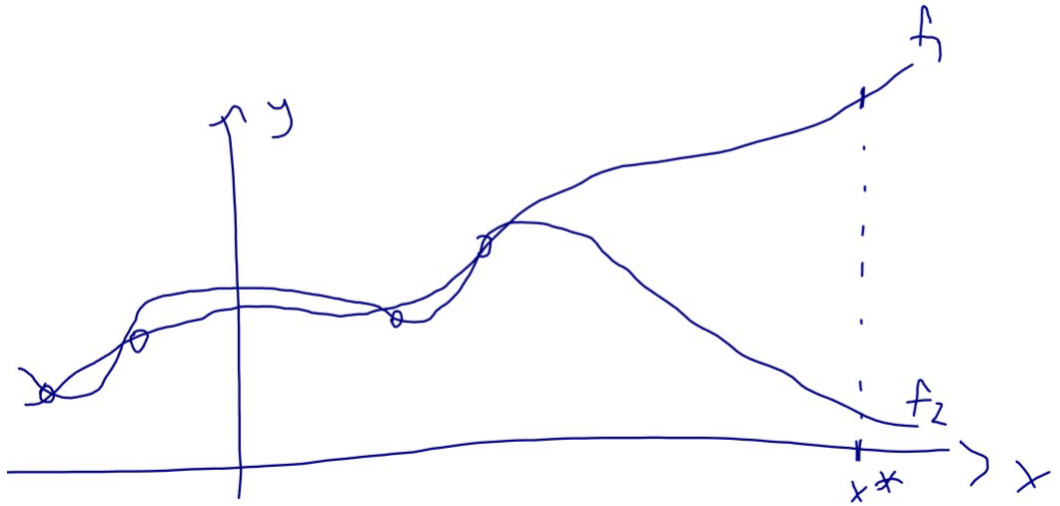
- Quantifying what our models do not know
- Relevant in safety-critical applications, e.g., medical, robotic
- There's not such thing as a free lunch though…
  - Modelling the prior is difficult
  - Computing the posterior is difficult



http://www.cs.ox.ac.uk/people/yarin.gal/website/blog_2248.html

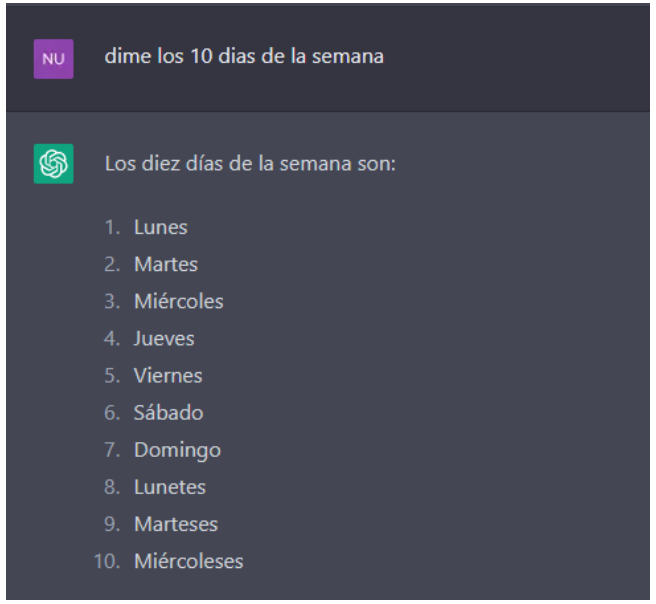
Example: Medical Diagnostics

- dAlbetes: an exciting new startup (not really)
  - claims to automatically diagnose diabetic retinopathy
  - accuracy 99% on their 4 train/test patients
  - engineer trained two deep learning systems to predict probability y
  - given input fondus image x .
- The engineer runs their system on your fondus image x*
- Which model f1, f2 would you want the engineer to use for your diagnosis?
  - None of these! ('I don't know')

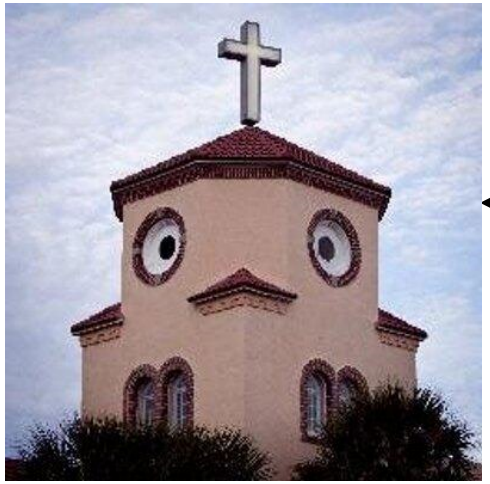


Motivation

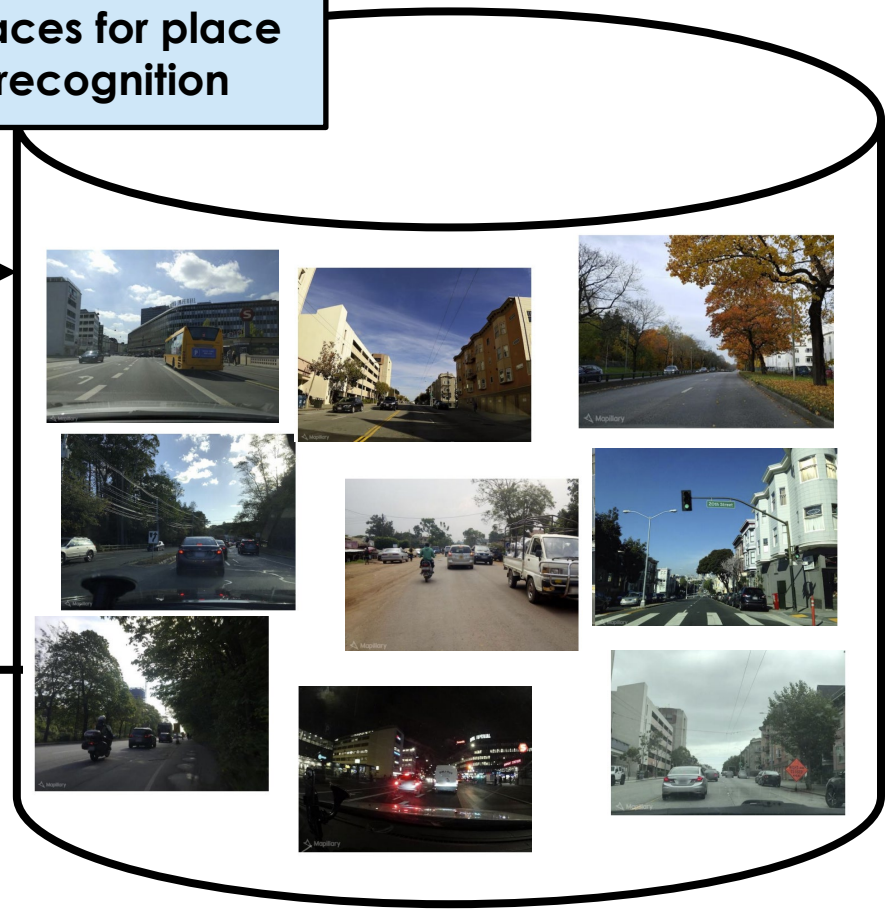
- NN always returns an element, even if it does not make sense!



Out-of-distribution query



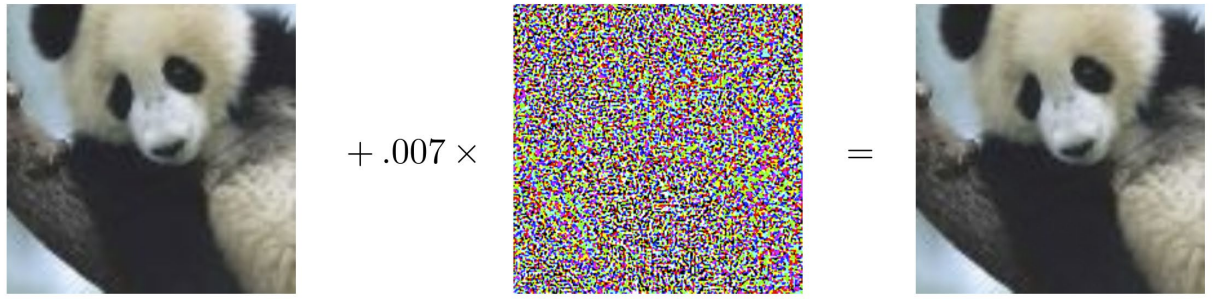
Database of places for place recognition



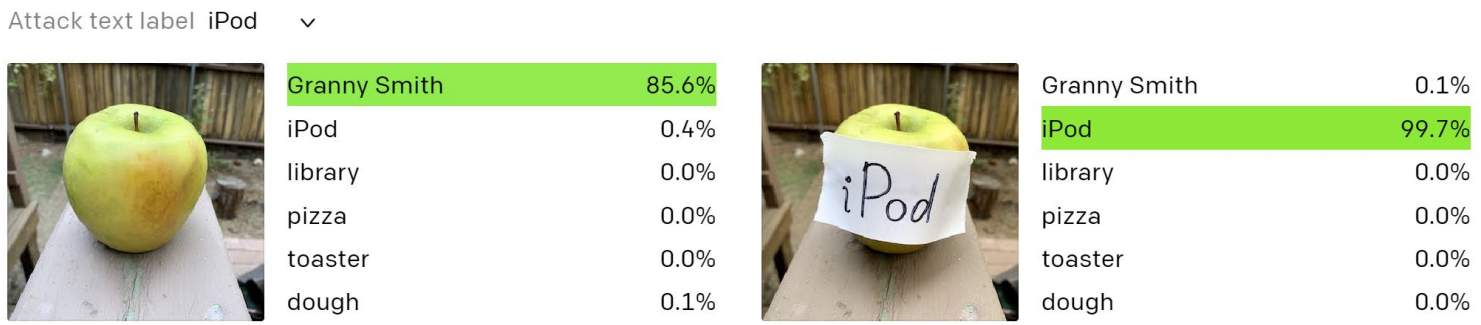
Motivation

Adversarial attacks:

- Slight perturbations can alter the output significantly



- Naïve perturbations too!

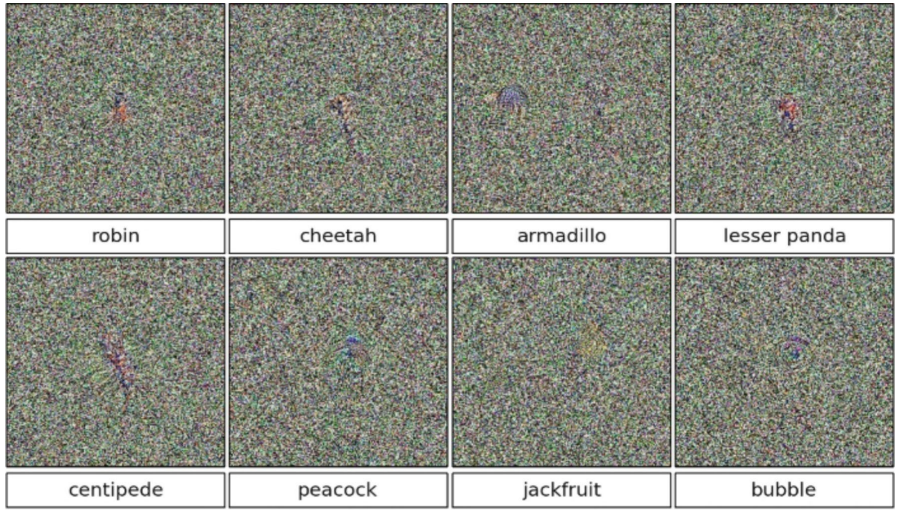


I. J. Goodfellow, J. Shlens, and C. Szegedy, Explaining and harnessing adversarial examples, *arXiv:1412.6572*, 2014

https://openai.com/blog/multimodal-neurons/

Motivation

Nguyen et al., CVPR 2015: all images below have confidence > 99% on a network that produces highly accurate on-distribution predictions.



Hein et al., CVPR 2019: ReLU nets are asymptotically overconfident.

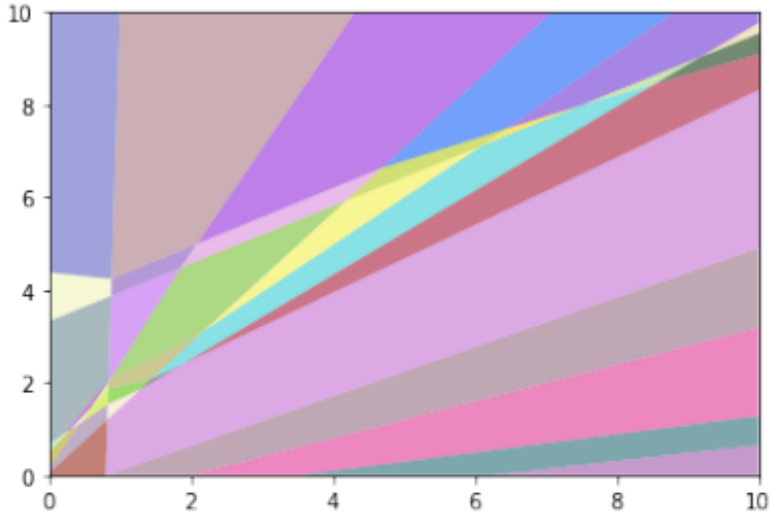


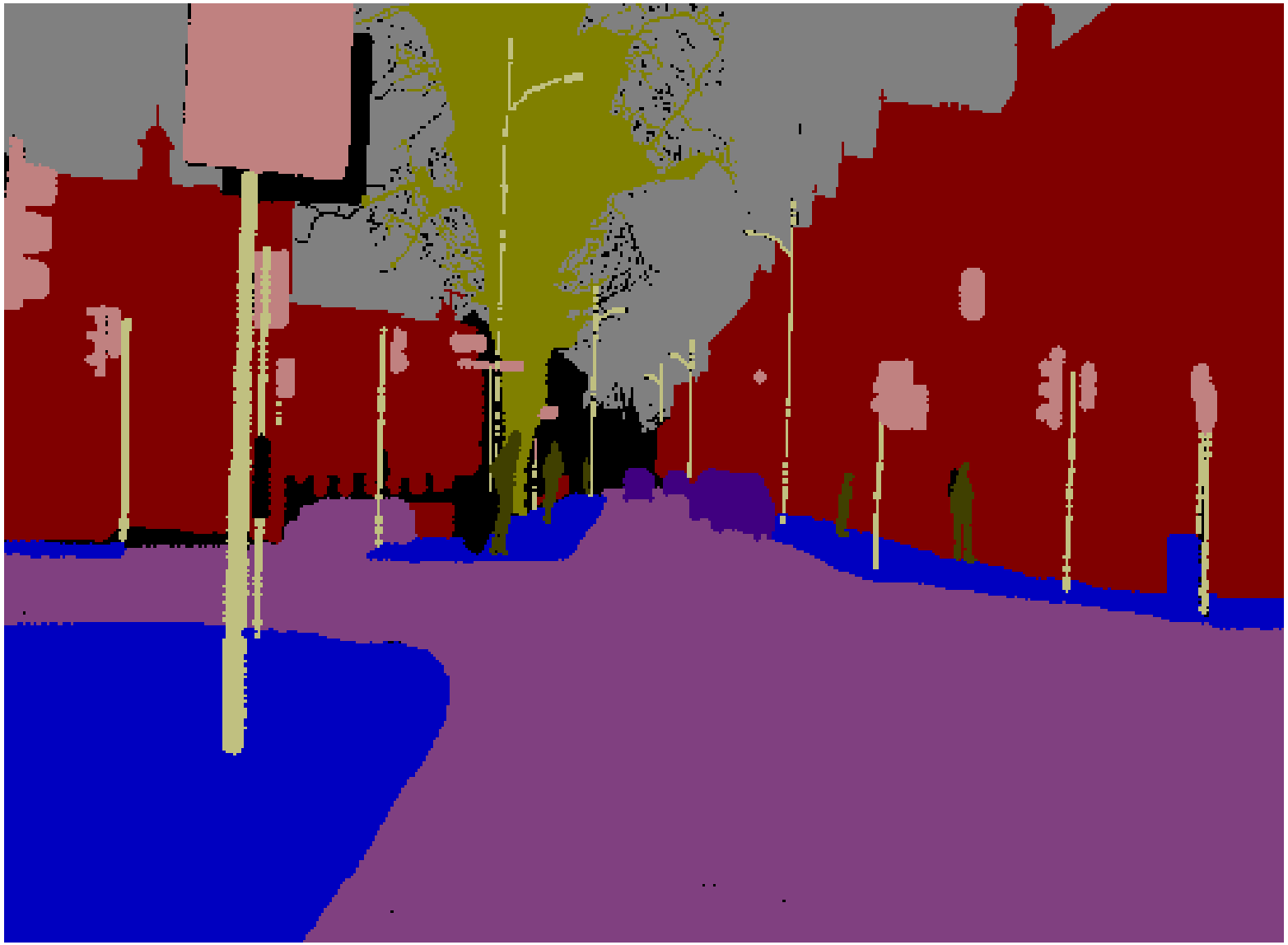
Figure 1: A decomposition of $\mathbb{R}^2$ into a finite set of polytopes for a two-hidden layer ReLU network. The outer polytopes extend to infinity. This is where ReLU networks realize arbitrarily high confidence predictions. The picture is produced with the code of [17].

Teaser: Uncertainty in Autonomous Driving

Universidad Zaragoza

Image: Yarin Gal

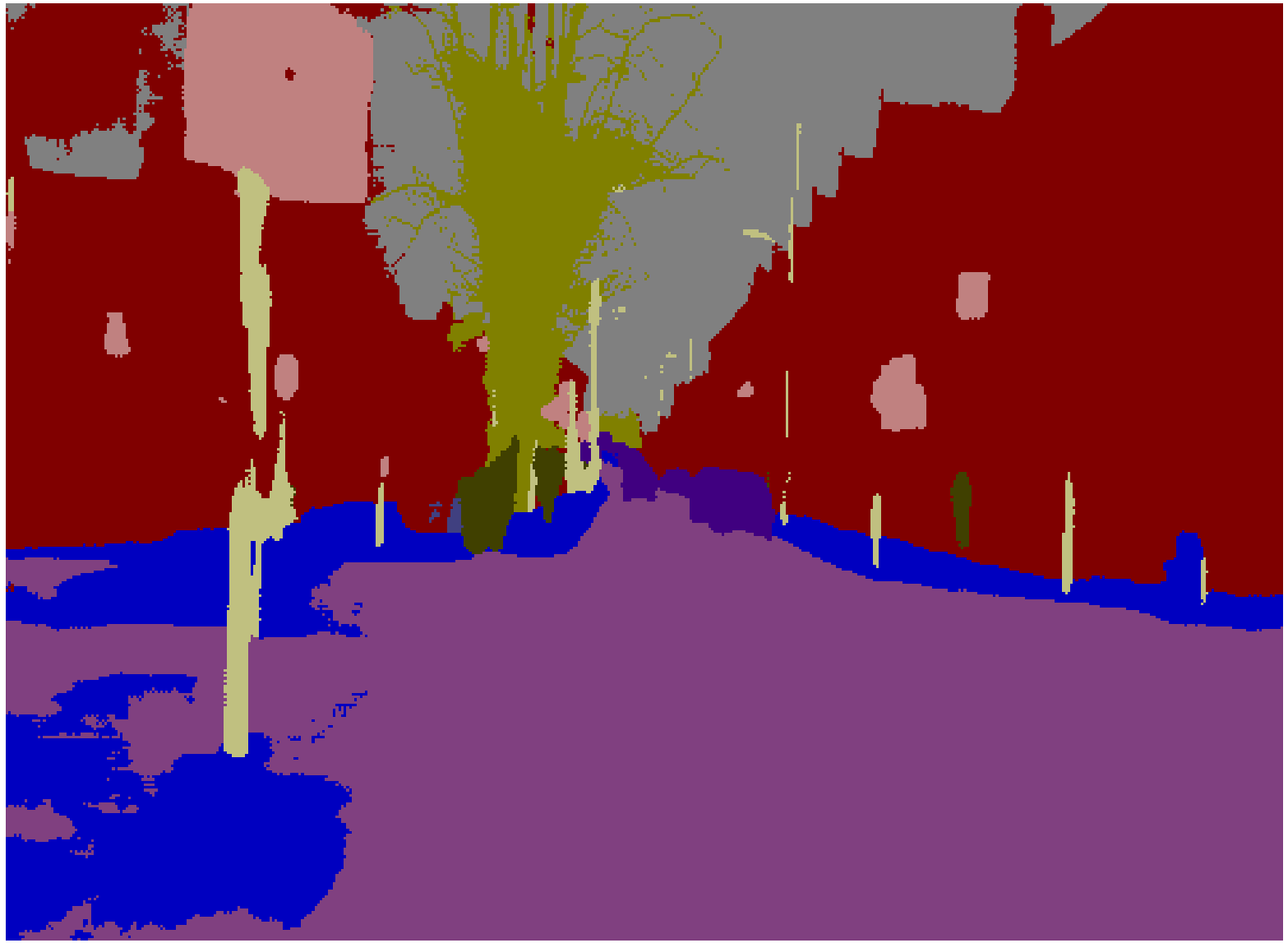
Teaser: Uncertainty in Autonomous Driving



Universidad Zaragoza

Image: Yarin Gal

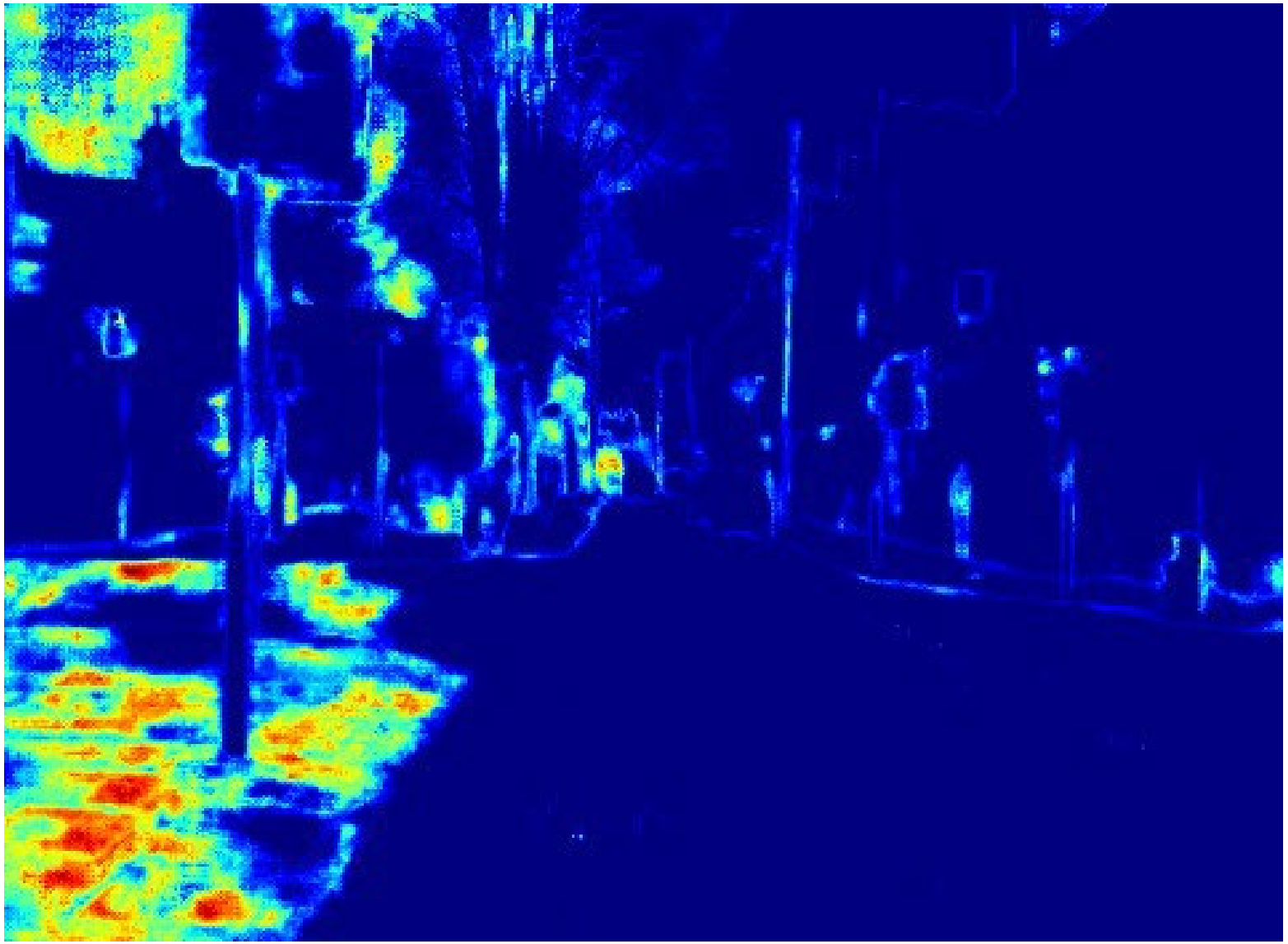
Teaser: Uncertainty in Autonomous Driving



Universidad Zaragoza

Image: Yarin Gal

Teaser: Uncertainty in Autonomous Driving



Universidad Zaragoza

Image: Yarin Gal

Bayesian learning for model parameters

- Step 0: Specify a parametric model with parameters w
- Step 1: Given data, $D = \{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$, write down the expression for the likelihood:

$$p(D|w)$$

- Step 2: Specify a prior (conjugate, informative…):

$$p(w)$$

- Step 3: Compute the posterior:

$$p(w|D) = \frac{p(D|w)p(w)}{p(D)}$$

Bayesian linear regression

- The input/output relation is linear in the model parameters

$$y = x^T w + \epsilon \quad \epsilon \sim N(0, \sigma^2) \quad \text{one data point}$$

$$\boldsymbol{y} = Xw + \epsilon I_n \quad \epsilon \sim N(0, \sigma^2) \quad \text{dataset (size n)}$$

- What is the likelihood?

$$p(y|x, w) = N(y \,|x^T w, \sigma^2) \qquad \text{one data point}$$

$$p(\boldsymbol{y}|X, w) = N(y \mid Xw, \sigma^2 I_n) \quad \text{joint distribution dataset}$$

- What is the prior?
  - For a linear-Gaussian likelihood model, the conjugate prior is a Gaussian.
  - The posterior is also Gaussian → Closed-form, easy, allows recurrence.

$$p(w) = N(w|w_0, \Sigma_0)$$

Bayesian linear regression (cont.)

- Likelihood: $p(\boldsymbol{y}|X, w) = N(\boldsymbol{y} \mid Xw, \sigma^2 I_n)$
- Prior (general case): $p(w) = N(w|w_0, \Sigma_0)$
- Posterior:

$$p(w|\boldsymbol{y}, X) \propto N(\boldsymbol{y} \mid Xw, \sigma^2 I_n) N(w|w_0, \Sigma_0) = N(w|w_n, \Sigma_n)$$

$$w_n = \Sigma_n \Sigma_0^{-1} w_0 + \frac{1}{\sigma^2} \Sigma_n X^T \boldsymbol{y}$$

$$\Sigma_n^{-1} = \Sigma_0^{-1} + \frac{1}{\sigma^2} X^T X$$

- If we use an isotropic Gaussian prior: $p(w) = N(w|0, \tau^2 I_d)$ then

$$w_n = \frac{1}{\sigma^2} \Sigma_n X^T \boldsymbol{y} = (\lambda I_d + X^T X)^{-1} X^T \boldsymbol{y}$$

with: $\lambda = \frac{\sigma^2}{\tau^2}$ ...which is again ridge regression!

Predictions in Bayesian regression

- Now that we have a distribution over parameters, we can integrate the parameters and get a predictive posterior.

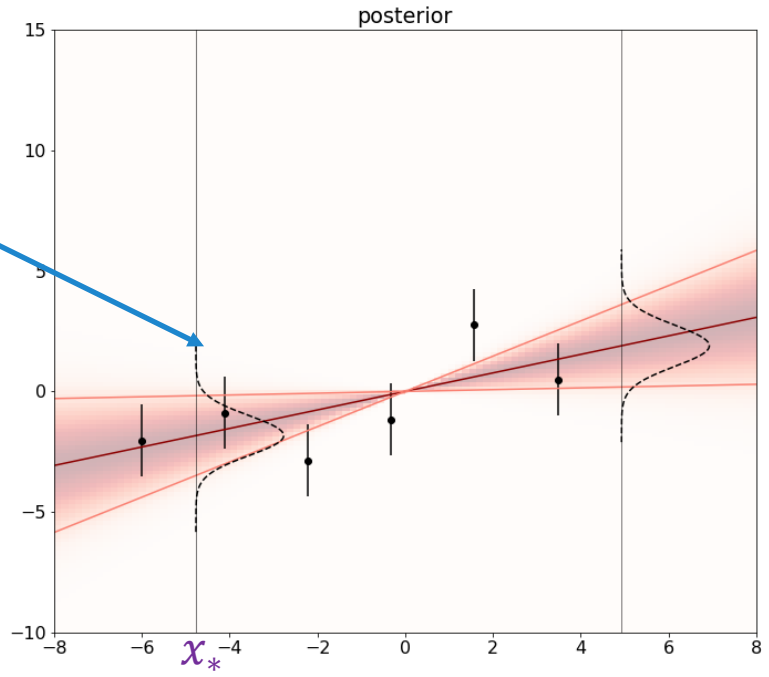
$$p(f_*|x_*, X, \boldsymbol{y}) = \int \underbrace{p(f_*|x_*, w)}_{\substack{likelihood\ of \\ new\ point}} \underbrace{p(w|X, \boldsymbol{y})}_{\substack{model \\ posterior}} dw$$

$$f_* \sim N(\mu(x_*), \sigma_n^2(x_*))$$

Remember

$$y = x^T w + \epsilon$$

$$\mu(x_*) = x_*^T w_n$$
$$\sigma_n^2(x_*) = \sigma^2 + x_*^T \Sigma_n x_*$$

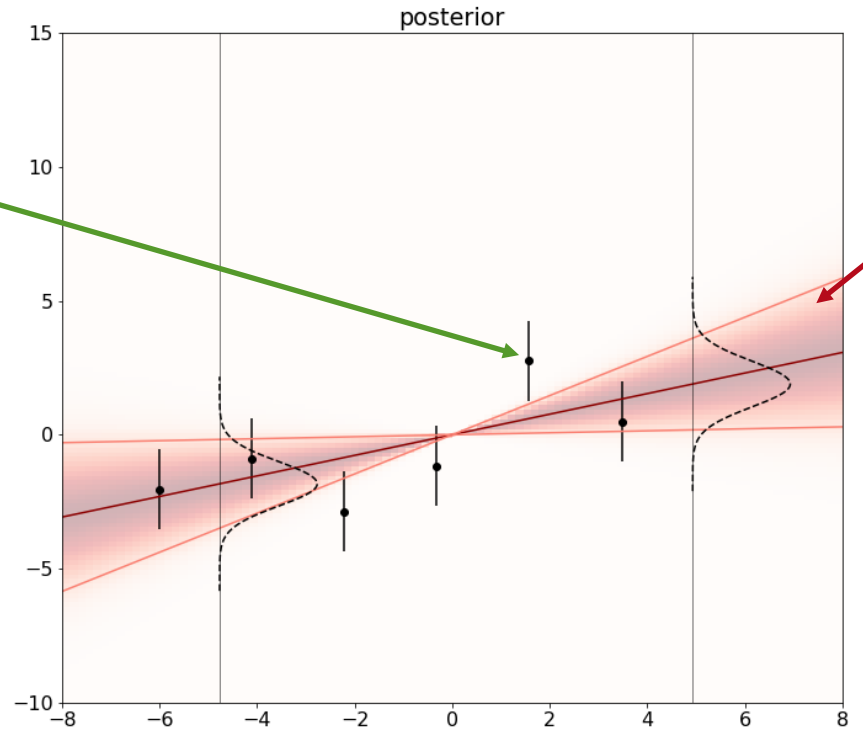


Predictions in Bayesian regression

$$\sigma_n^2(x_*) = \sigma^2 + x_*^T \Sigma_n x_*$$

Aleatoric

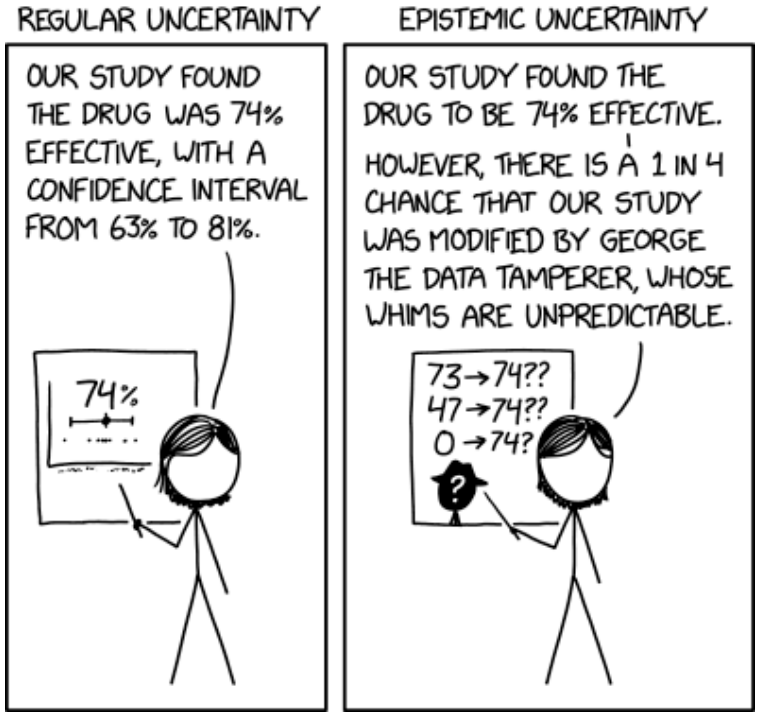
Epistemic



Kevin P. Murphy, Probabilistic Machine Learning: An introduction, section 11.6, MIT Press, 2021, https://probml.github.io/pml-book/

What uncertainties do we need?

- Aleatoric: noise inherent in the observations (data uncertainty)
  - Comes from stochastic sources, irreducible
  - Homoscedastic: constant for different inputs
  - Heteroscedastic: input-dependent
- Epistemic: uncertainty in the model (model uncertainty)
  - Comes from lack of knowledge, reducible by additional training data

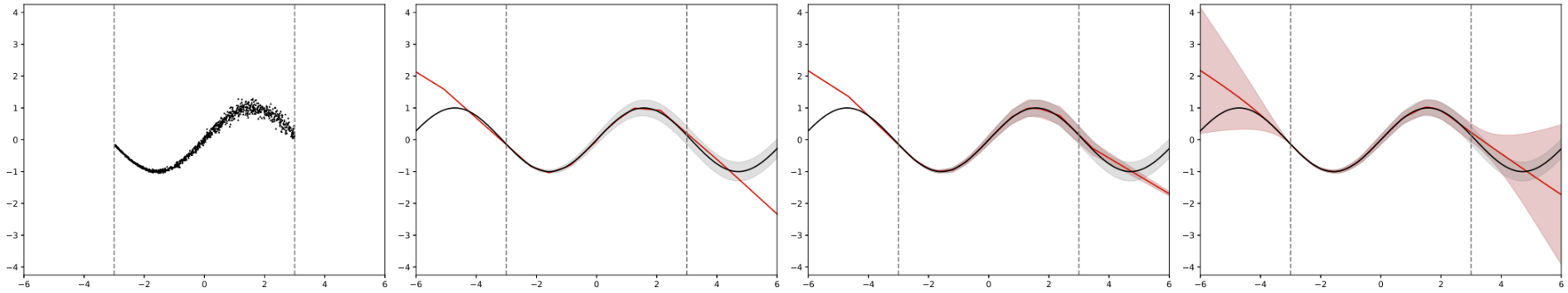


https://xkcd.com/2440/

Alex Kendall and Yarin Gal. What Uncertainties Do We Need in Bayesian Deep Learning for Computer Vision?, NeurIPS, 2017 https://papers.nips.cc/paper_files/paper/2017/hash/2650d6089a6d640c5e85b2b88265dc2b-Abstract.html

Example: toy regression

- Sinusoidal data, input dependent Gaussian noise



Training data

Mean regression

Mean regression + Aleatoric uncertainty

Mean regression + Aleatoric uncertainty + Epistemic uncertainty

F. K. Gustafsson, M. Danelljan, T. B. Schon, Evaluating scalable Bayesian deep learning methods for robust computer vision, CVPRW. 2020.
https://openaccess.thecvf.com/content_CVPRW_2020/papers/w20/Gustafsson_Evaluating_Scalable_Bayesian_Deep_Learning_Methods_for_Robust_Computer_Vision_CVPRW_2020_paper.pdf

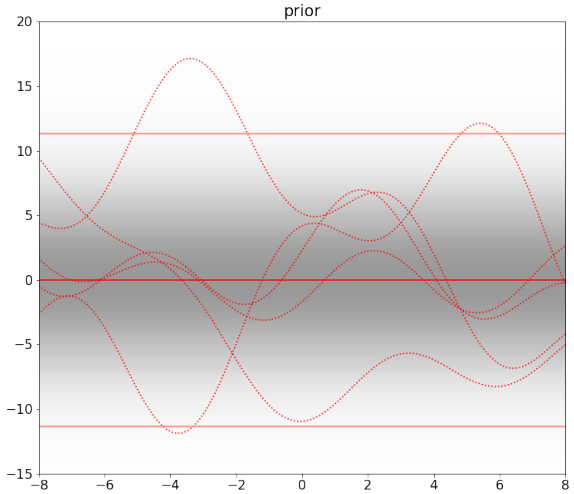
Parametric regression in a feature space

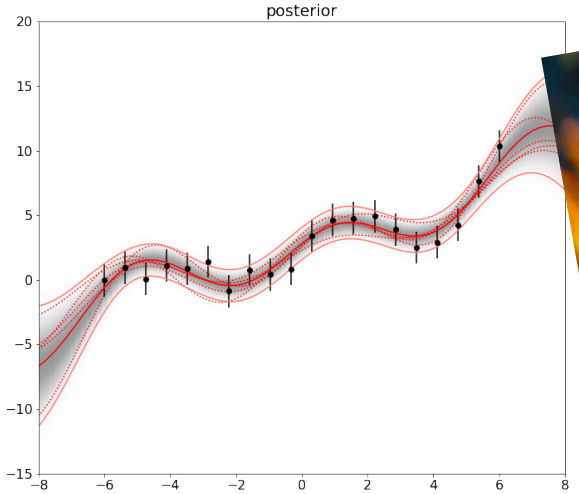
- If we use a feature space to project the inputs $\phi(x)$

$$y = \phi(x)^T w + \epsilon \quad \epsilon \sim N(0, \sigma^2) \qquad \text{one data point } \{x, y\}$$

$$\mathbf{y} = \Phi_X^T w + \epsilon I_n \qquad \text{dataset } \{X, \mathbf{y}\} \text{ (size n)}$$

- The output is nonlinear, but the problem is still linear in the weights!





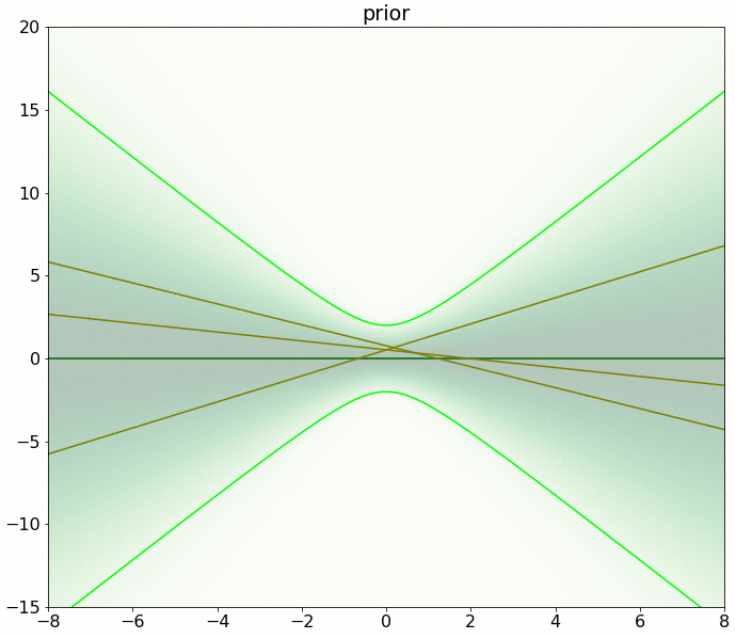
Parametric regression in a feature space

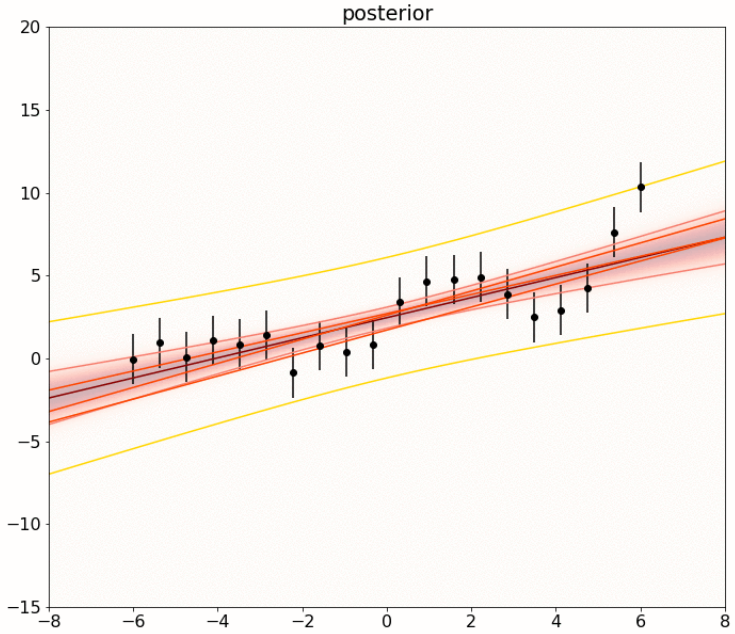
- Linear with a feature space $\phi(x)$

$$y = \phi(x)^T w + \epsilon \quad \epsilon \sim N(0, \sigma^2) \qquad \text{one data point}$$

$$\boldsymbol{y} = \Phi_X^T w + \epsilon I_n \qquad \text{dataset (size n)}$$

- For example: $\phi(x) = (1, x)$ $\qquad y = w_1 + w_2 x + \epsilon$



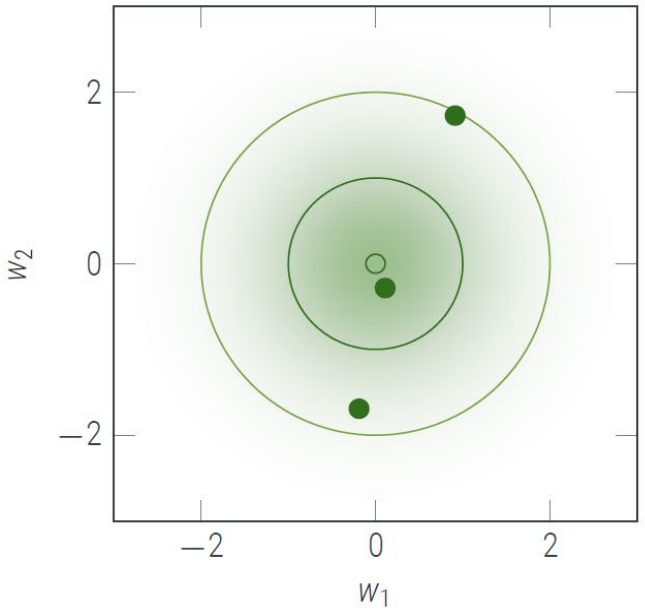


Parametric regression

- Priors: $f(x) = \Phi_X^T w$

- Weight prior:
  $p(w) = N(w|0, \Sigma)$

- Predictive prior:
  $p(f_*) = N(f_*|0, \phi_X^T \Sigma \phi_X)$



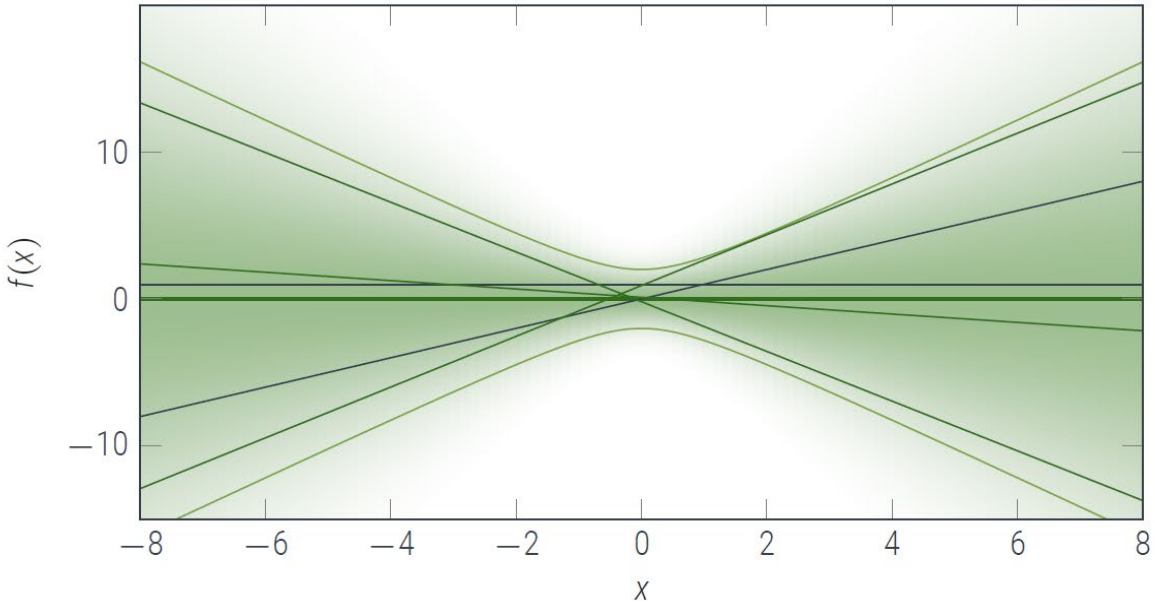


Image: Philipp Hennig

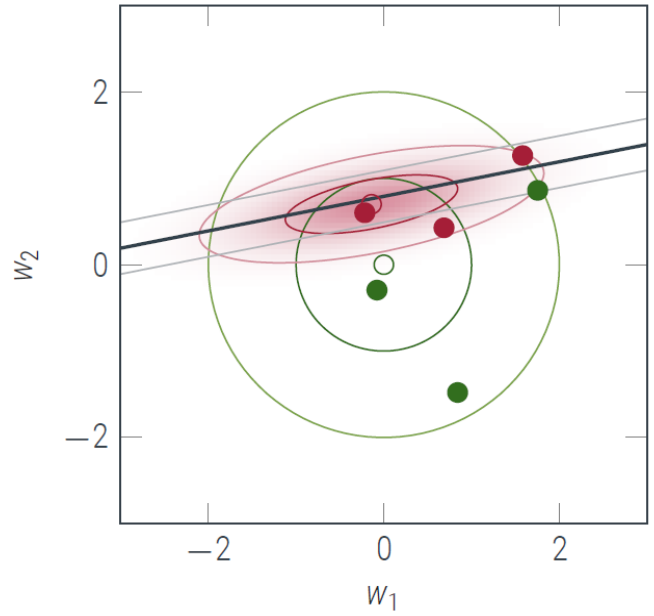
Parametric regression

- Prior vs Likelihood vs Posterior

- Weight space

$p(w)$

$p(w|X,y)$



- Function (predictive) space:

$p(f_*)$

$p(f_*|X,y)$

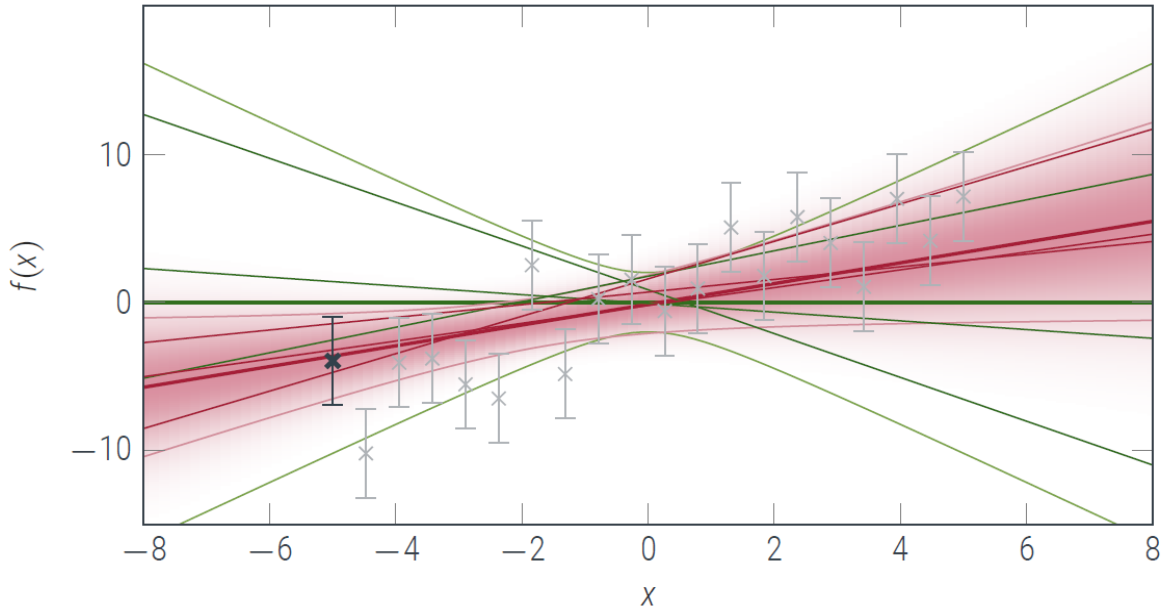


Image: Philipp Hennig

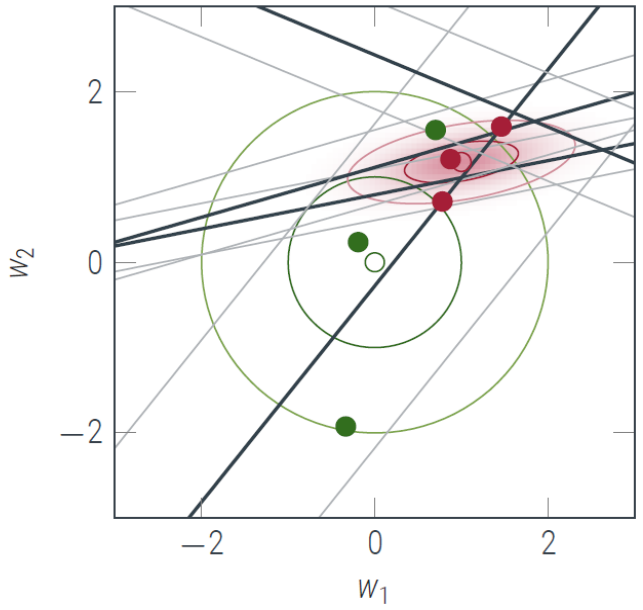
Parametric regression

- Prior vs Likelihood vs Posterior

- Weight space

$p(w)$

$p(w|X,y)$



- Function (predictive) space:

$p(f_*)$

$p(f_*|X,y)$

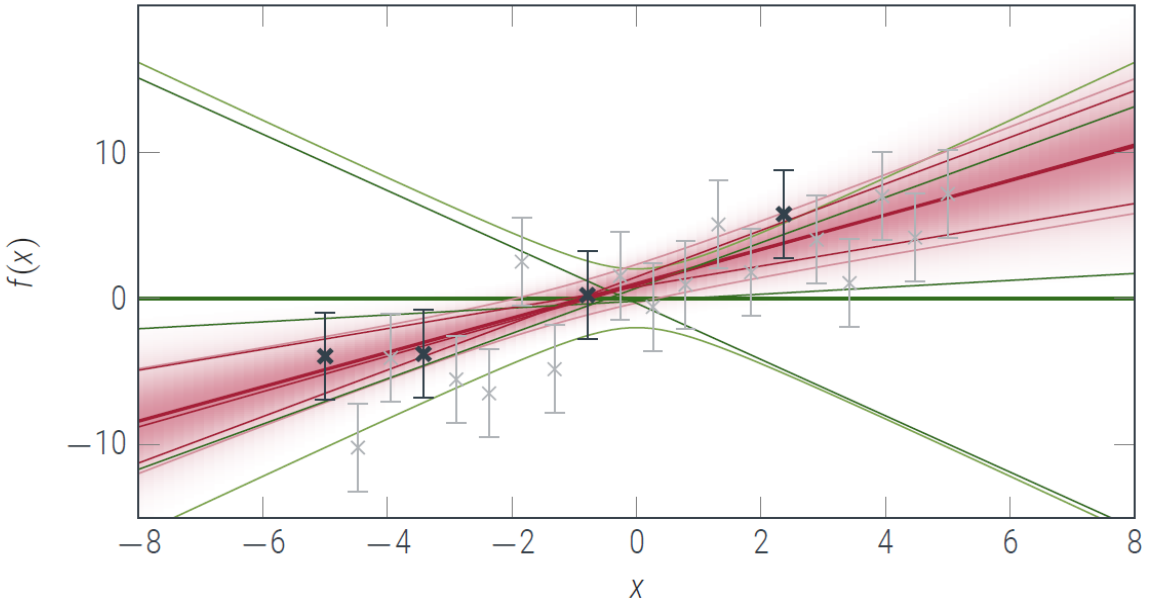
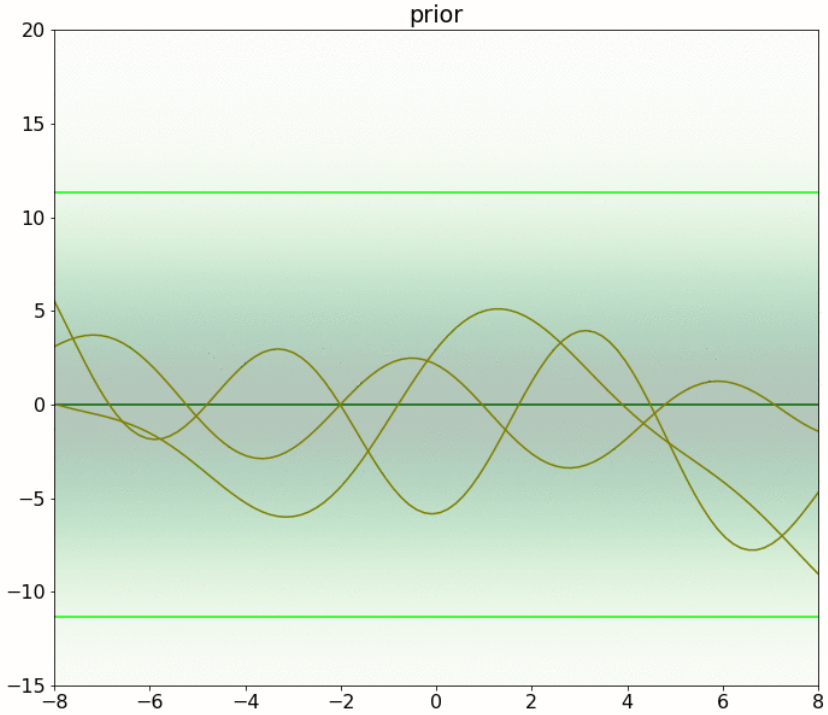
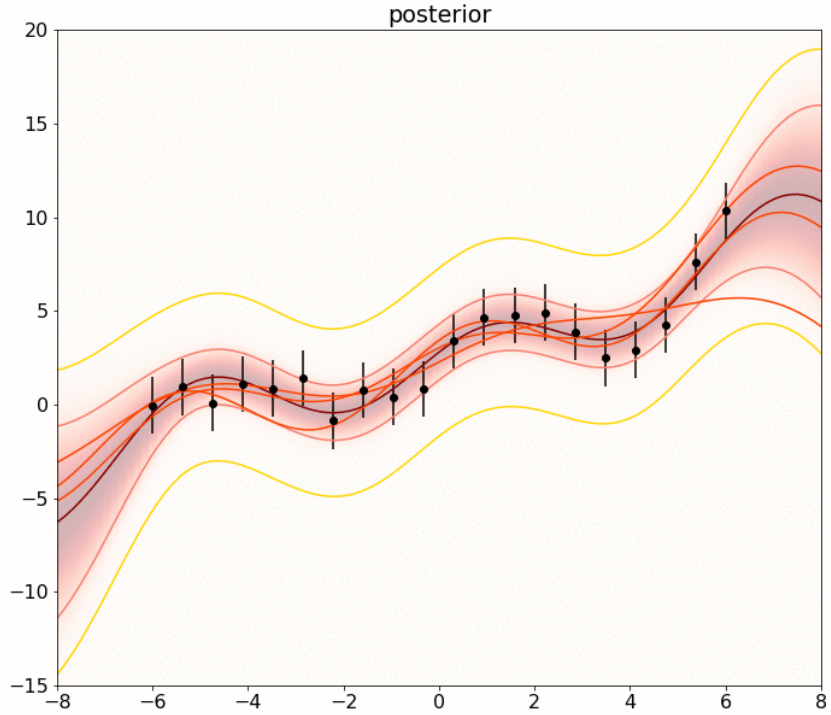


Image: Philipp Hennig

Fourier features

- $\phi(x) = [\sin(x), \sin(2x), \ldots, \cos(x), \cos(2x), \ldots]$





Alternative: higher number of features

- Can we use infinite features? Using the kernel trick!

- The predictive posterior can be written as:

$$\hat{f}_* = \phi_*^T \Sigma \phi_X (\phi_X^T \Sigma \phi_X + \sigma^2 I)^{-1} \boldsymbol{y}$$

$$\sigma_{f_*}^2 = \phi_*^T \Sigma \phi_* - \phi_*^T \Sigma \phi_X (\phi_X^T \Sigma \phi_X + \sigma^2 I)^{-1} \phi_X^T \Sigma \phi_*$$

- All the operations with features are dot products. The result is always a scalar.

- Kernel function: $k(x_1, x_2) = \phi(x_1)^T \Sigma \phi(x_2)$
  - This type of function is called a Mercer kernel.

Gaussian processes

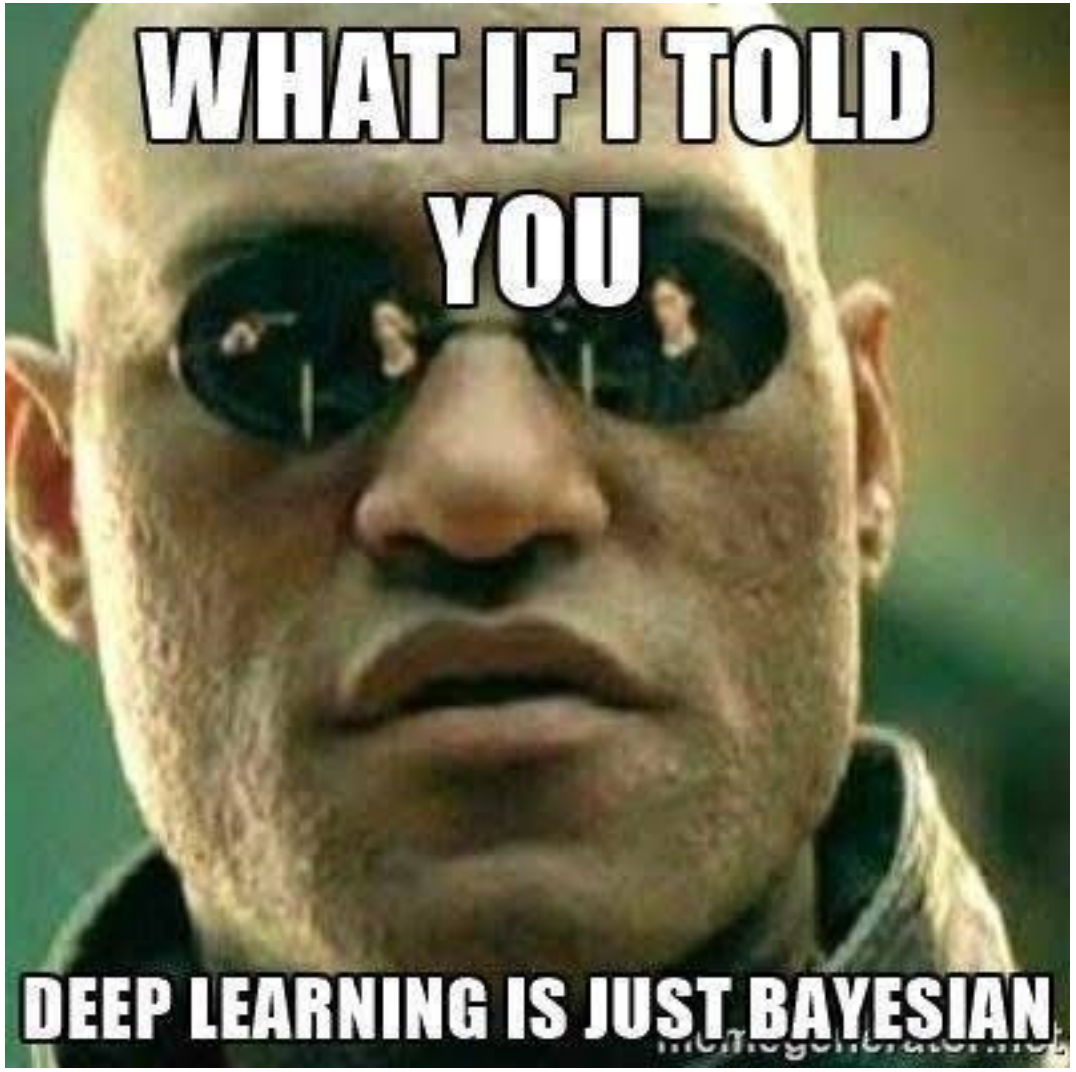
- Rewrite the predictive posterior:

$$\widehat{f_*} = k(x_*, X)\,(k(X,X) + \sigma^2 I)^{-1}\boldsymbol{y}$$
$$\sigma_{f_*}^2 = k(x_*, x_*) - k(x_*, X)\,(k(X,X) + \sigma^2 I)^{-1}k(X, x_*)$$

- If we use a Mercer kernel, we have a covariance matrix, a covariance vector and a variance term.
  - $k(x_1, x_2)$ measures the correlation/similarity of x_1 and x_2.

- The predictive posterior is a conditional Gaussian of:

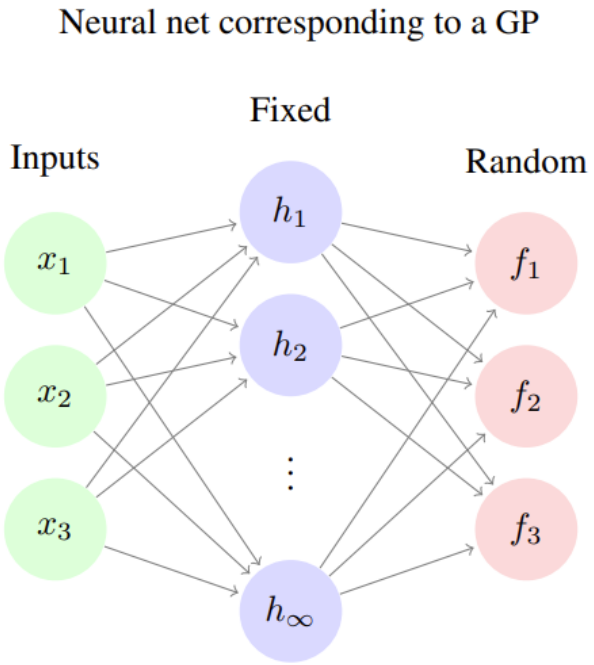
$$\begin{pmatrix}\boldsymbol{y}\\ f_*\end{pmatrix} \sim N\left(0, \begin{bmatrix} k(X,X) + \sigma^2 I & k(X, x_*) \\ k(x_*, X) & k(x_*, x_*)\end{bmatrix}\right)$$

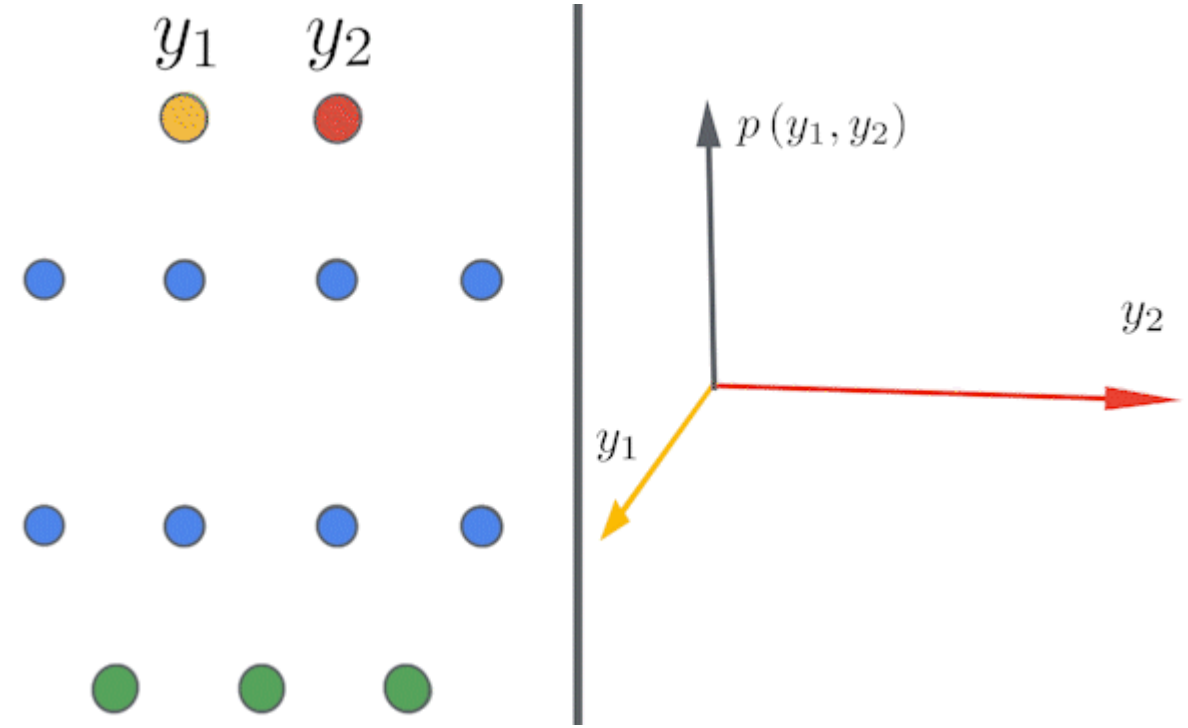

WHAT IF I TOLD YOU
DEEP LEARNING IS JUST BAYESIAN

Infinitely-wide networks

- The Neural Network Gaussian Process (NNGP) corresponds to the infinite width limit of Bayesian neural networks, and to the distribution over functions realized by non-Bayesian neural networks after random initialization.

- The Neural Tangent Kernel describes the evolution of neural network predictions during gradient descent training. In the infinite width limit the NTK usually becomes constant, often allowing closed form expressions for the function computed by a wide neural network throughout gradient descent training. The training dynamics essentially become linearized.





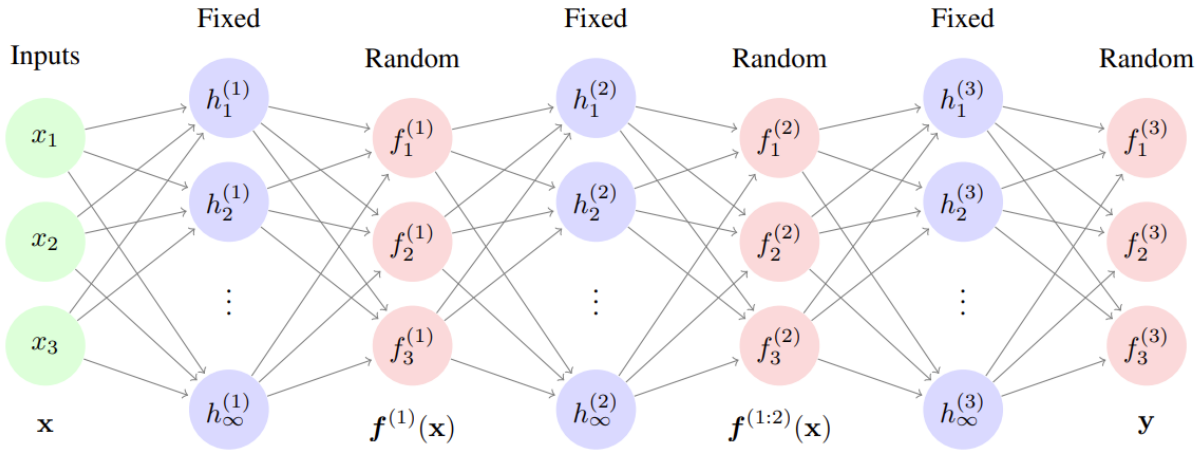
Left: a Bayesian neural network with two hidden layers, transforming a 3-dimensional input into a two-dimensional output. **Right**: output probability density function induced by the random weights of the network. **Video**: as the width of the network increases, the output distribution simplifies, ultimately converging to a multivariate normal in the infinite width limit. https://en.wikipedia.org/wiki/Large_width_limits_of_neural_networks

Deep GPs

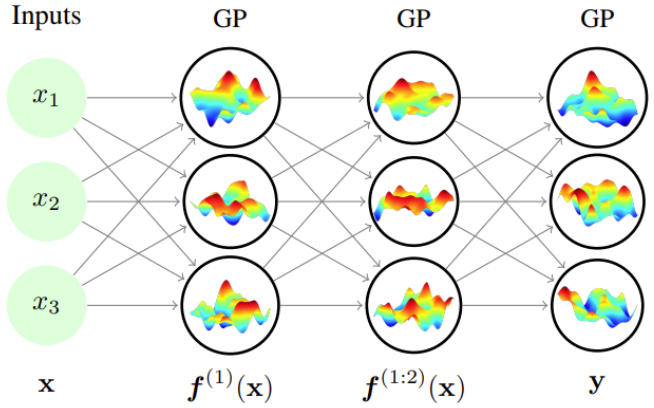
$$\boldsymbol{f}^{(1:L)}(\mathbf{x}) = \boldsymbol{f}^{(L)}(\boldsymbol{f}^{(L-1)}(\ldots \boldsymbol{f}^{(2)}(\boldsymbol{f}^{(1)}(\mathbf{x}))\ldots))$$

$$\text{with each} \quad f_d^{(\ell)} \overset{\text{ind}}{\sim} \text{GP}\left(0, k_d^{(\ell)}(\mathbf{x}, \mathbf{x}')\right)$$

A neural net with fixed activation functions corresponding to a 3-layer deep GP



A net with nonparametric activation functions corresponding to a 3-layer deep GP



https://arxiv.org/pdf/1402.5836.pdf

Deep GPs for Olympic Marathon

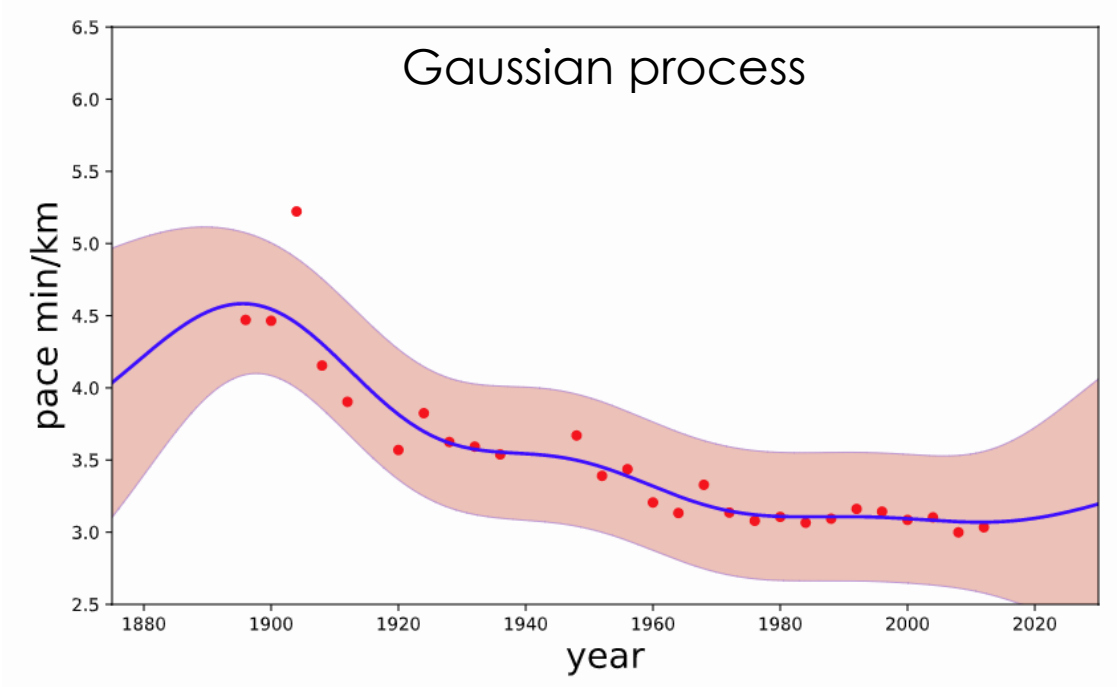
ATHLETICS

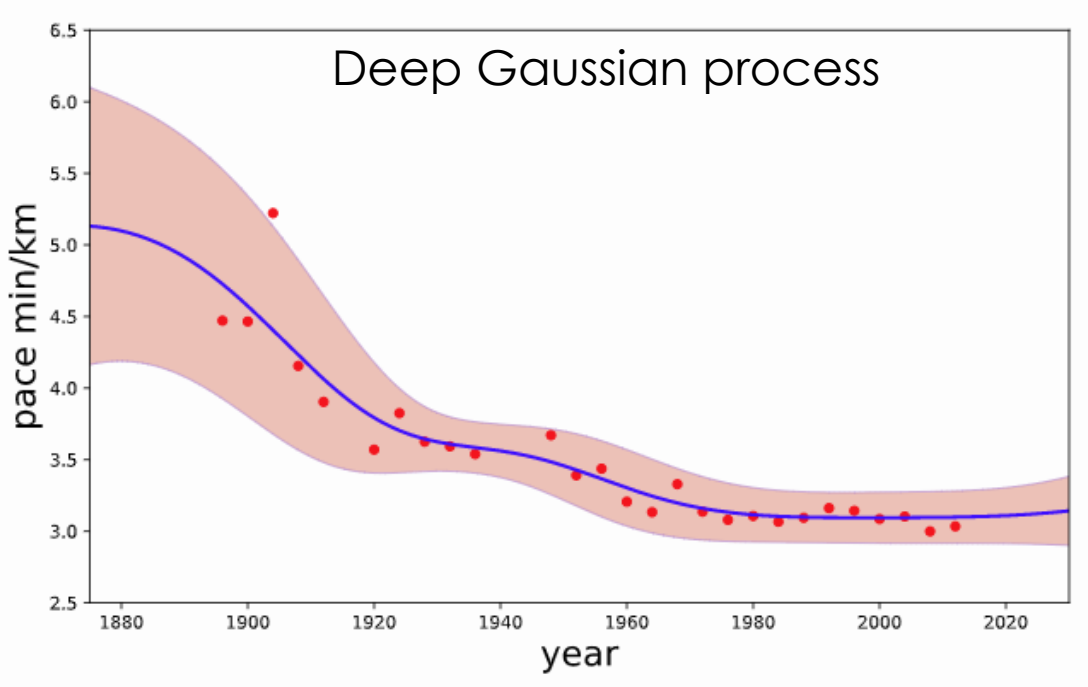
MARATHON AND DECATHLON CHAMPIONSHIPS

The Amateur Athletic Association championships for this year were concluded at Loughborough College Stadium, Leicestershire, on Saturday, with the second, and last, day of the Decathlon and the decision of the Marathon championship.

MARATHON CHAMPIONSHIP (26 miles 385 yds.) (record: 2hrs. 30min. 57 secs., by H. W. Payne, Windsor to Stamford Bridge, on July 5, 1929; standard time: 3hrs. 5min.).—1, T. Holden (Tipton Harriers), 2hrs. 33min. 20 1-5sec.; 1; T. Richards (South London Harriers), 2hrs. 36min. 7sec., 2; D. McNab Robertson (Maryhill Harriers, Glasgow), 2hrs. 37min. 54 3-5sec., 3; J. F. Farrell (Maryhill Harriers), 2hrs. 39min. 4s. 2-5sec., 4; Dr. A. M. Turing (Walton A.C.), 2hrs. 46min. 3secs., 5; L. H. Griffiths (Reading A.C.), 2hrs. 47min. 50 2-5sec., 6.

DECATHLON CHAMPIONSHIP.—H. J. Moesgaard-Kjeldsen (Polytechnic Harriers, London), 5,965 points, 1; Captain H. Whittle (Army and Reading A.C.), 5,650, 2;

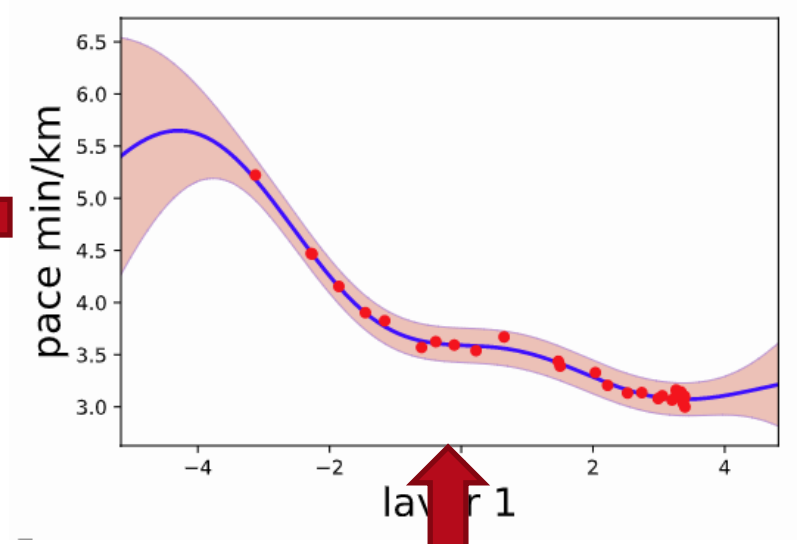


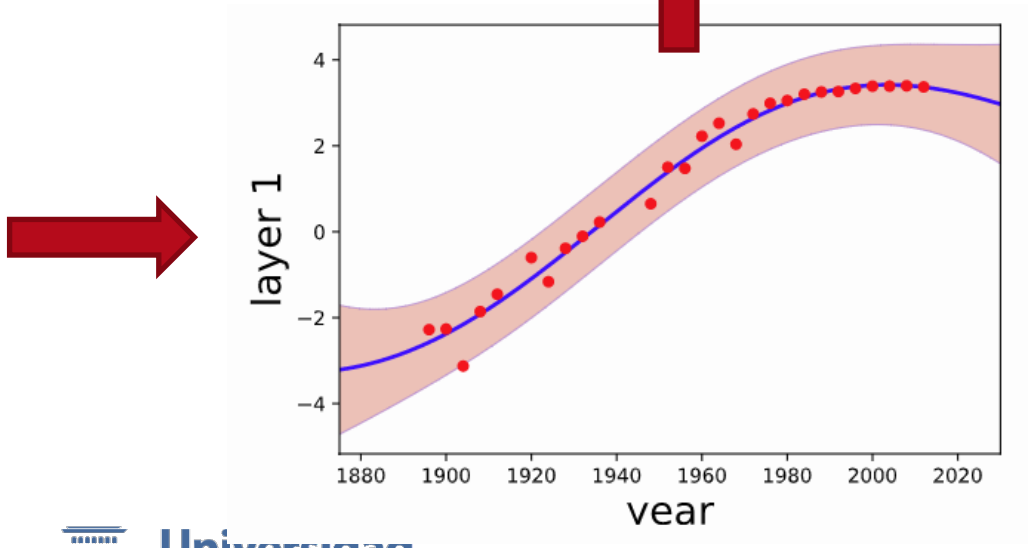


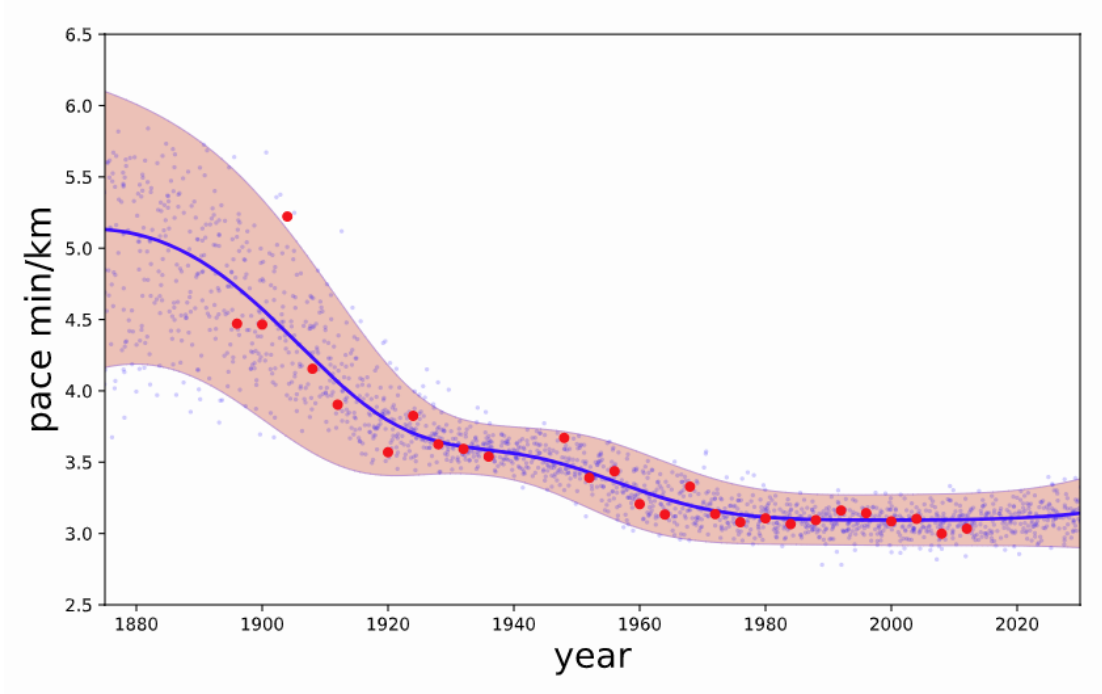
- 1904 Olympic marathon
  - Unusually high temperatures and humidity, coupled with poor organization and planning, meant that the marathon in 1904 had the slowest winning time and most bizarre outcome in Olympic history. https://www.statista.com/statistics/1099351/olympics-marathon-gold-medal-times-since-1896/

Example by Neil Lawrence: http://inverseprobability.com/talks/notes/deep-gaussian-processes.html

Deep GPs for Olympic Marathon







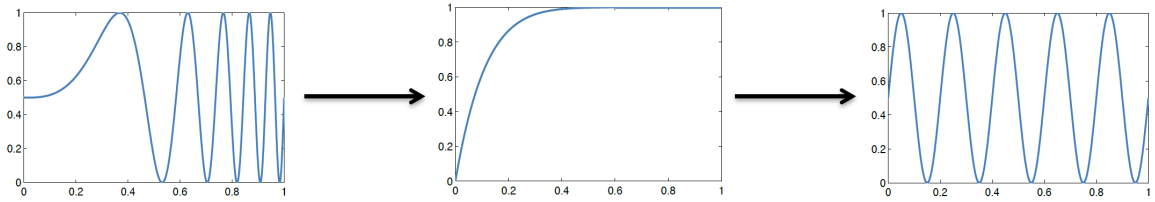
Example by Neil Lawrence: http://inverseprobability.com/talks/notes/deep-gaussian-processes.html

Best of both worlds

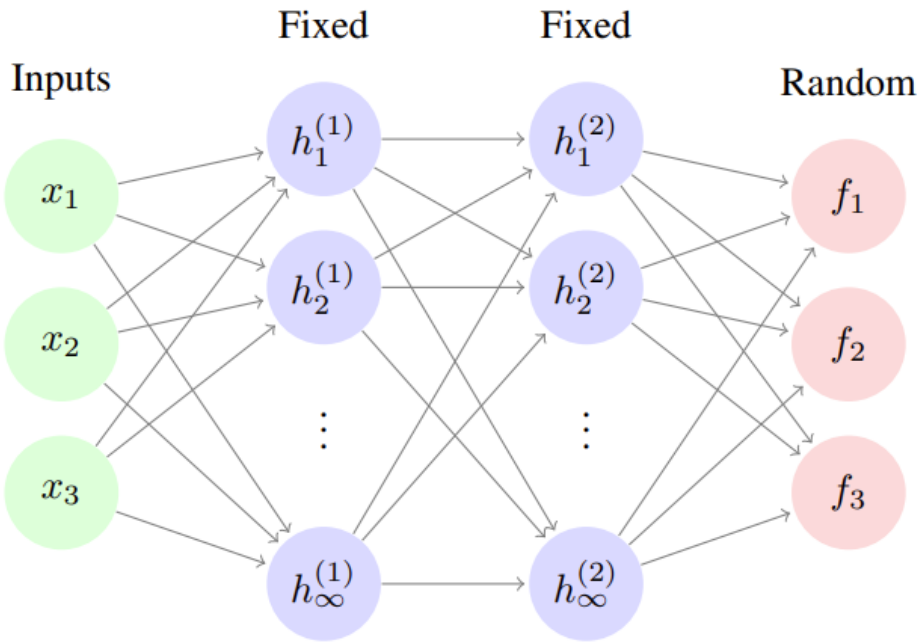
- Neural Networks
  - Powerful representation learning
  - Straightforward learning
  - High confidence in out-of-distribution inputs
  - Difficult uncertainty representation (inferring a posterior over millions of parameters)
- Gaussian Processes
  - Reliable uncertainty estimates, interpretable.
  - No overfitting, just a handful of parameters learnt marginal likelihood
  - No explicit representation learning.
  - Poor scalability

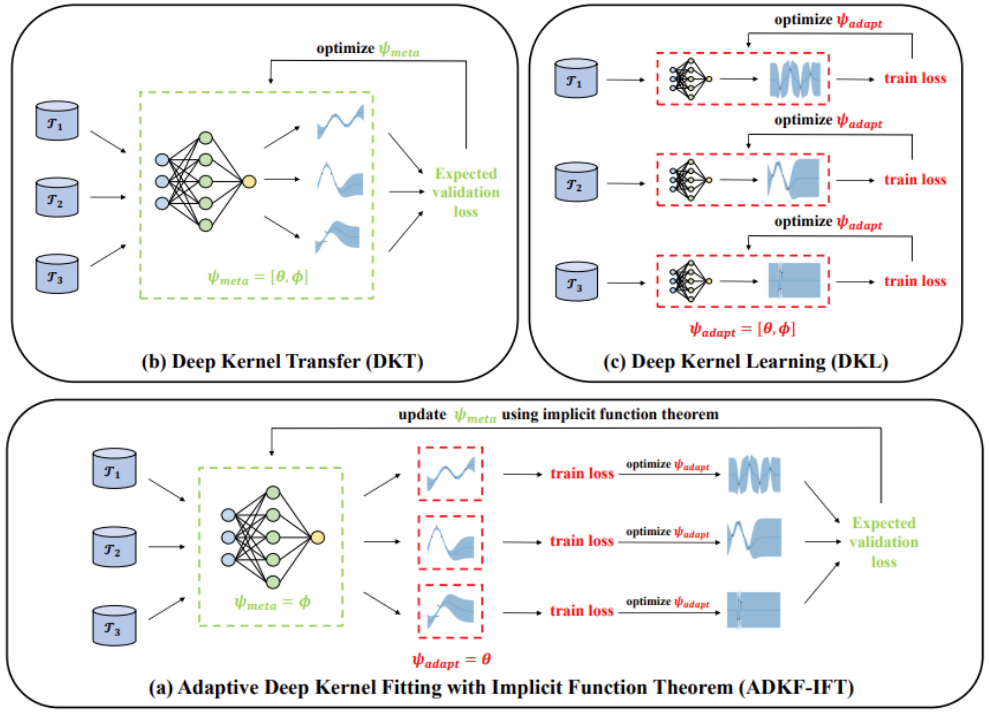
GPs with deep kernels

- Mapped inputs: $k(\psi(x), \psi(x'))$



Net corresponding to a GP with a deep kernel





Aleatoric uncertainty in classification DNNs

- Let $\boldsymbol{f^W}: \boldsymbol{x} \rightarrow \boldsymbol{y}$ be a deep network and $\mathcal{D} = \{(\boldsymbol{x}^{(1)}, y^{(1)}), \cdots, (\boldsymbol{x}^{(i)}, y^{(i)}), \cdots, (\boldsymbol{x}^{(N)}, y^{(N)})\}$ an annotated dataset
- Captured by predicting a categorical distribution $p(\boldsymbol{y}|\boldsymbol{w})$
- Likelihood: $p(\boldsymbol{y}|\boldsymbol{X}, \boldsymbol{w}) = \prod_{i=1}^{N} p(y^{(i)}|\boldsymbol{x}^{(i)}, \boldsymbol{w})$

- $\boldsymbol{w}_{MLE}$ is obtained by minimizing the negative log-likelihood, equivalent to minimizing the cross-entropy loss

$$\boldsymbol{w}_{MLE} = \arg min_{\boldsymbol{w}} - \sum_{i=1}^{N} \log(p(y^{(i)}|\boldsymbol{x}^{(i)}, \boldsymbol{w}))$$

- At test time, the model predicts the aleatoric uncertainty distribution $p(y|\boldsymbol{x}, \boldsymbol{w}_{MLE})$
  - Heteroscedastic!

F. K. Gustafsson, M. Danelljan, T. B. Schon, Evaluating scalable bayesian deep learning methods for robust computer vision, CVPRW. 2020.

Aleatoric uncertainty in regression DNNs

- Let $\boldsymbol{f^w}: \boldsymbol{x} \rightarrow \boldsymbol{y}$ be a deep network and $\mathcal{D} = \{(\boldsymbol{x}^{(1)}, y^{(1)}), \cdots, (\boldsymbol{x}^{(i)}, y^{(i)}), \cdots, (\boldsymbol{x}^{(N)}, y^{(N)})\}$ an annotated dataset
- Gaussian likelihood generalizes L2: $p(y|\boldsymbol{x}, \boldsymbol{w}) \sim \mathcal{N}(y|\mu_{\boldsymbol{w}}(\boldsymbol{x}), \sigma^2_{\boldsymbol{w}}(\boldsymbol{x}))$
- Heteroscedastic noise: $\mu_{\boldsymbol{W}}(\boldsymbol{x}), \sigma^2_{W}(\boldsymbol{x})$ are predicted by the DNN

$$\boldsymbol{f^W}: \boldsymbol{x} \rightarrow [\mu_{\boldsymbol{w}}, \sigma^2_{\boldsymbol{w}}]$$

- The network is trained by minimizing the negative log-likelihood

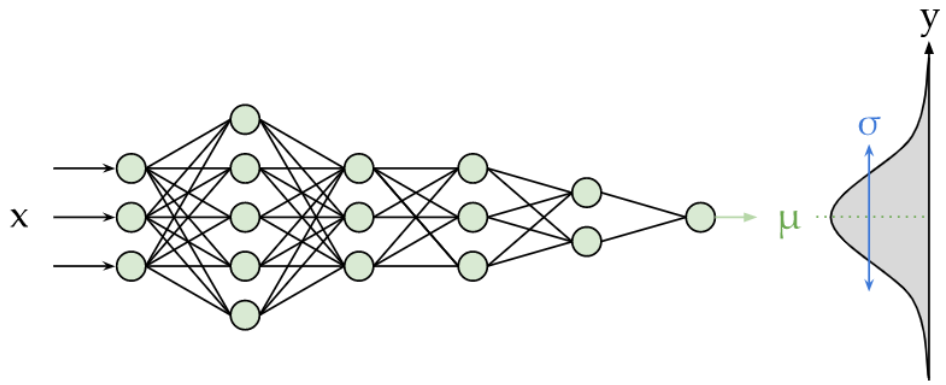
$$\boldsymbol{w}_{MLE} = \arg min_{\boldsymbol{w}} - \sum_{i=1}^{N} \log(p(y^{(i)}|\boldsymbol{x}^{(i)}, \boldsymbol{w})) = \frac{1}{N} \sum_{i=1}^{N} \frac{\|\mu_{\boldsymbol{w}} - y\|^2}{2\sigma^2_{\boldsymbol{w}}} + \frac{1}{2} \log(\sigma^2_{\boldsymbol{w}})$$

- Variants:
  - Laplace likelihood generalizes L1
  - Homoscedastic noise $\sigma^2_{W}(\boldsymbol{x}) = \sigma^2_{MAP}$

F. K. Gustafsson, M. Danelljan, T. B. Schon, Evaluating scalable bayesian deep learning methods for robust computer vision, CVPRW. 2020.

Aleatoric uncertainty in regression DNNs

Homoscedastic noise
One head



Heteroscedastic noise
Two heads

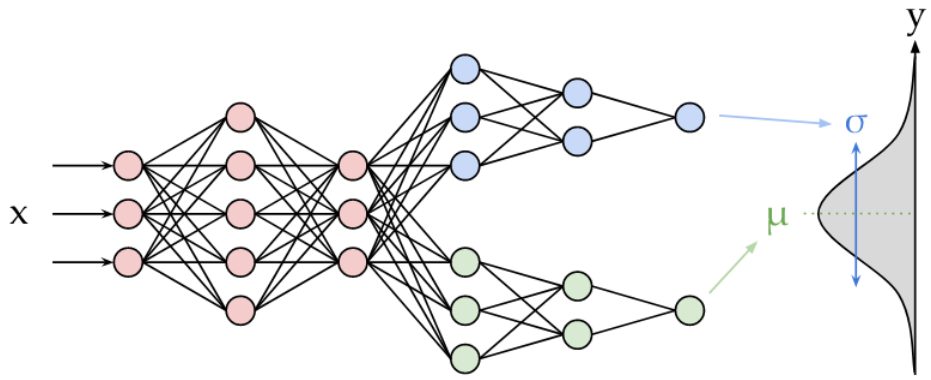


Image: Brendan Hasz

Approximate Bayesian inference

- For many models, it is not possible to compute the exact posterior

$$p(\boldsymbol{w}|\mathbf{X},\mathbf{y}) = \frac{p(\mathbf{y}|\mathbf{X},\boldsymbol{w})p(\boldsymbol{w})}{\int p(\mathbf{y}|\mathbf{X},\boldsymbol{w})p(\boldsymbol{w})d\boldsymbol{w}}$$

- and the posterior predictive

$$p(y|\boldsymbol{x},\boldsymbol{X},\boldsymbol{y}) = \int p(y|\boldsymbol{x},\boldsymbol{w})p(\boldsymbol{w}|\boldsymbol{X},\boldsymbol{y})\, d\boldsymbol{w}$$

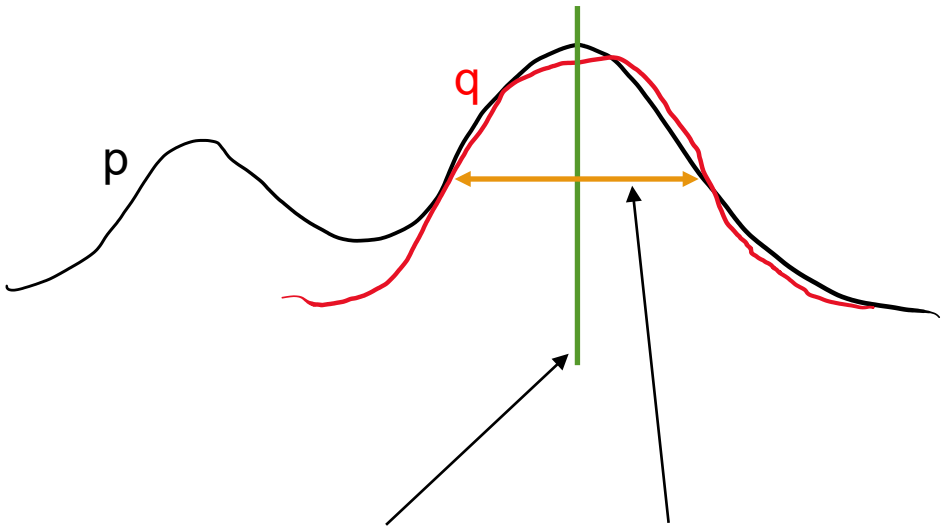
- Approximate alternatives available:
  - Function approximation
    - Examples: variational inference (check soft-RL lectures), Laplace approximation…
  - Sampling methods
    - Examples: MCMC (check the course 69152 Machine Learning!), ensembles…

Variational inference remainder

- We want to work with intractable distribution $p(\boldsymbol{w}|\mathbf{X}, \mathbf{y})$

- Let's find a *similar* tractable distribution q($\boldsymbol{w}$)
- How do we find similar distributions? **KL-divergence**

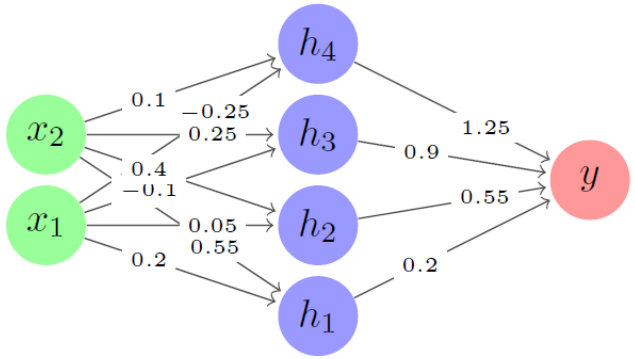
$$KL(q||p) = \int_x q(x) \log \frac{q(x)}{p(x)} dx$$



$$KL(q||p) = \mathbb{E}_q[\log q(x)] - \mathbb{E}_q[\log p(x)] = -\mathbb{E}_q[\log p(x)] - \mathcal{H}(q(x))$$

Variational inference

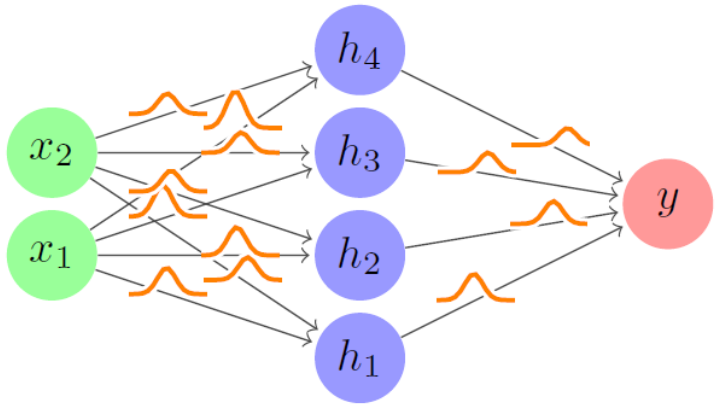
- We want to approximate this posterior



$$p(\boldsymbol{w}|\mathbf{X},\mathbf{y}) = \frac{p(\mathbf{y}|\mathbf{X},\boldsymbol{w})p(\boldsymbol{w})}{\int p(\mathbf{y}|\mathbf{X},\boldsymbol{w})p(\boldsymbol{w})d\boldsymbol{w}}$$

- Simplest prior: unit Gaussian

$$p(\boldsymbol{w}) \sim N(0, \boldsymbol{I})$$



- Simplest approximation: mean field Gaussian
  - Mean field: we assume each dimensión/parameter is independent

$$q(w_i) \sim N(\mu_i, \sigma_i)$$

Variational inference

- Remember: optimizing the KL-divergence is equivalent to optimize the ELBO

$$\mathbb{E}_q[\log p(x,y) - \log q(x)] = \log p(y) - KL(q(x)||p(x|y))$$

$$\mathbb{E}_q[\log p(x,y) - \log q(x)] = \mathbb{E}_q[\log p(y|x) + \log p(x) - \log q(x)]$$

- In our case x = **w** and y = **{X, y}**
- Bayes-by-backprop

$$ELBO = \underbrace{\mathbb{E}_{\mathbf{w}\sim q}[\log p(\mathbf{y}|\mathbf{X},\mathbf{w})]}_{\text{likelihood term}} - \underbrace{\sum_i \frac{1}{2}[\sigma_i^2 + \mu_i^2]}_{\text{prior term}} + \underbrace{\sum_i \log[\sigma_i]}_{\text{log q term}}$$

- We want to maximize ELBO → Loss = negative ELBO

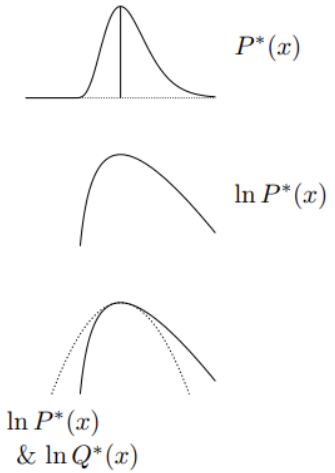
Variational inference

- Simple and accurate.
- Provides a full, simple posterior approximation (Gaussian)

- It is sensitive to initialization and priors.
- Mean-field approximation is too constraining.
  - Alternatives allow correlation between weights, but they do not scale.
- Variance of ELBO estimates can be a problem.

- Performance can be improved with Gauss-Newton method, but it requires more approximations. [Osawa et al. 2019]

Laplace approximation



- We can compute the posterior up to a constant

$$p(\boldsymbol{w}|\mathbf{X},\mathbf{y}) = \frac{p(\mathbf{y}|\mathbf{X},\boldsymbol{w})p(\boldsymbol{w})}{\int p(\mathbf{y}|\mathbf{X},\boldsymbol{w})p(\boldsymbol{w})d\boldsymbol{w}} = \frac{\mathbf{1}}{\boldsymbol{Z}} p(\mathbf{y}|\mathbf{X},\boldsymbol{w})p(\boldsymbol{w})$$

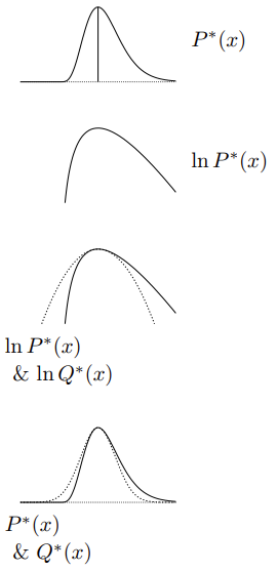
- Let's consider the maximum a posterior (MAP) loss

$$\mathcal{L}_{MAP} = -\log p(\mathbf{w}|\mathbf{X},\mathbf{y}) \qquad p(\mathbf{w}|\mathbf{X},\mathbf{y}) = \frac{1}{Z}\exp(-\mathcal{L}_{MAP})$$

- Taylor expansion of the loss around the max value $\boldsymbol{w}^*$

$$\mathcal{L}(\mathbf{w}) \approx \mathcal{L}(\mathbf{w}^*) + \nabla_w \mathcal{L}|_{\mathbf{w}^*}(\mathbf{w}-\mathbf{w}^*) + \frac{1}{2}(\mathbf{w}-\mathbf{w}^*)^T(\nabla_w^2 \mathcal{L}|_{\mathbf{w}^*})(\mathbf{w}-\mathbf{w}^*) + \dots$$

Laplace approximation (cont)



- Recover the posterior from the loss approximation

$$p(\mathbf{w}|\mathbf{X},\mathbf{y}) = \frac{1}{Z}\exp(-\mathcal{L}_{MAP}) \qquad H(\mathbf{w}^*) = \nabla_w^2 \mathcal{L}|_{\mathbf{w}^*}$$

Hessian

- Build distribution Q from the loss

$$\exp(-\mathcal{L}(\mathbf{w})) \approx \exp(-\mathcal{L}(\mathbf{w}^*)) \exp\left(\frac{1}{2}(\mathbf{w}-\mathbf{w}^*)^T H(\mathbf{w}^*)(\mathbf{w}-\mathbf{w}^*)\right)$$

$$Z \approx \exp(-\mathcal{L}(\mathbf{w}^*)) \int_{\mathbf{w}} \exp\left(\frac{1}{2}(\mathbf{w}-\mathbf{w}^*)^T H(\mathbf{w}^*)(\mathbf{w}-\mathbf{w}^*)\right) d\mathbf{w}$$

- Resulting in a Gaussian! $\mathcal{N}(\mathbf{w}|\mathbf{w}^*, H(\mathbf{w}^*)^{-1})$

$$p(\mathbf{w}|\mathbf{X},\mathbf{y}) \approx q(\mathbf{w}) = \frac{\exp\left(\frac{1}{2}(\mathbf{w}-\mathbf{w}^*)^T H(\mathbf{w}^*)(\mathbf{w}-\mathbf{w}^*)\right)}{\int \exp\left(\frac{1}{2}(\mathbf{w}-\mathbf{w}^*)^T H(\mathbf{w}^*)(\mathbf{w}-\mathbf{w}^*)\right) d\mathbf{w}}$$

Laplace approximation (cont)

- Practical advantages:
  - **cheap:** θ_{MAP} can be found by "normal" deep training, with the Hessian only computed at the end, adding minimal overhead.
  - **post-hoc:** can be added to a pre-trained network!
  - **reliable:** Hessians can be computed with automatic differentiation (more below)
  - **innocuous:** you get to keep your point estimate!

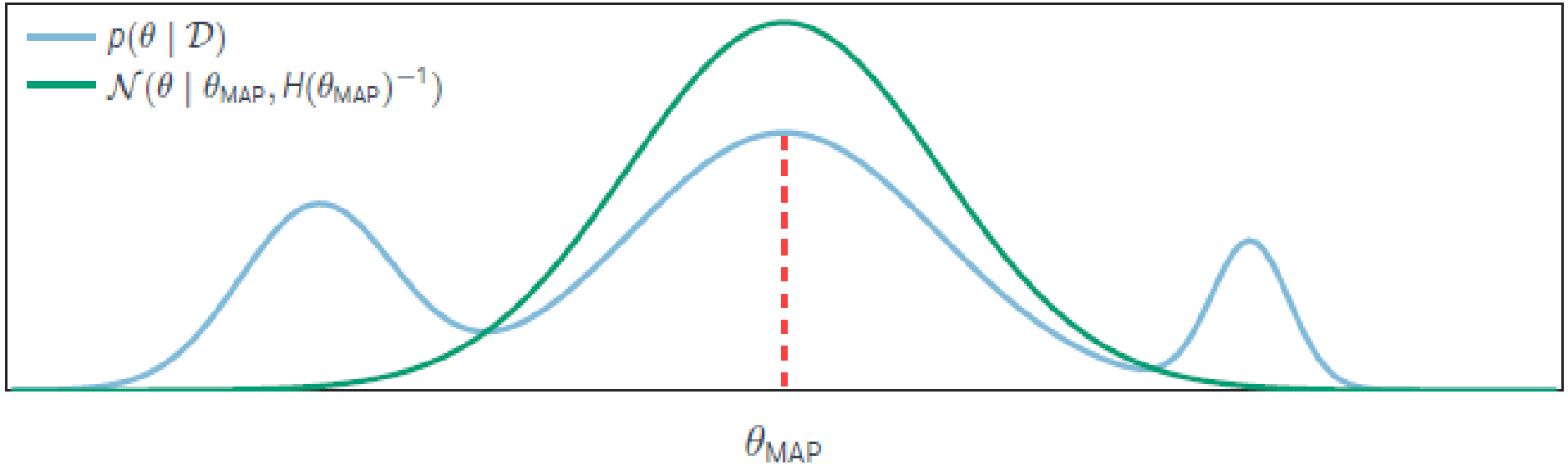
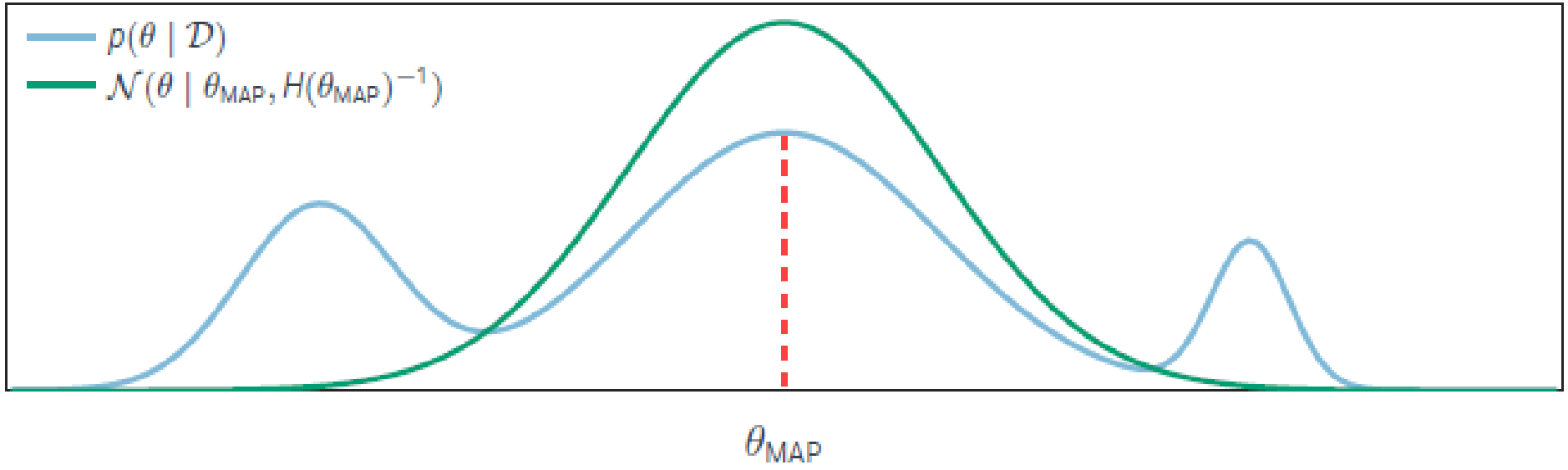


Image: Philipp Hennig

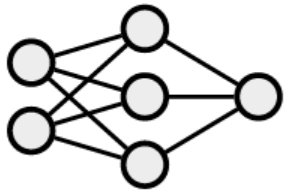
Laplace approximation (cont)

- Practical problems:
  - $\mathcal{L}$ is non-convex & SGD is used
  - $\boldsymbol{w}^*$ is not guaranteed to be a local minimum.
  - $H(\boldsymbol{w}^*)$ is not guaranteed to be positive definite.
  - $H(\mathbf{w}^*) \in \mathbb{R}^{d \times d}$ we need an inverse!
  - We can approximate the Hessian $H(\boldsymbol{w}^*)$ using Gauss-Newton and assume diagonal (mean field).

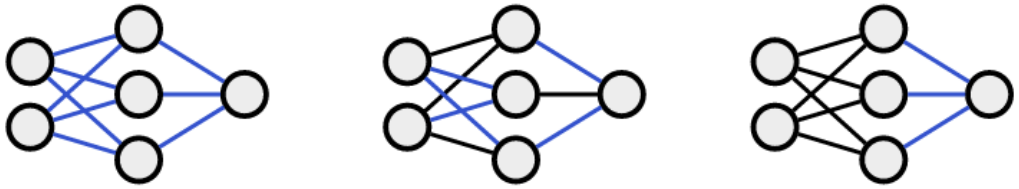


Laplace variants

Deterministic neural network f_θ

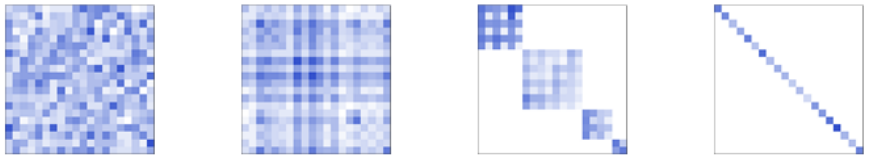


① **Weights to be treated probabilistically with Laplace**

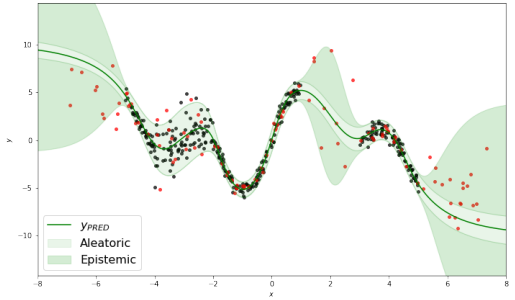


(a) All **(b)** Subnetwork **(c)** Last-Layer

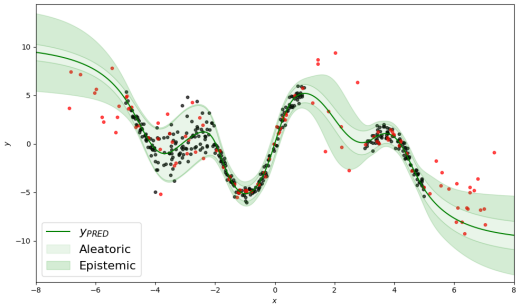
② **Approximation of the Hessian**



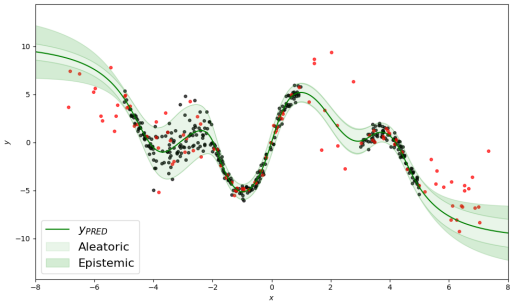
(a) Full **(b)** LRank **(c)** KFAC **(d)** Diag.



All



Subnet



Last layer

Universidad Zaragoza

Daxberger E, Kristiadi A, Immer A, Eschenhagen R, Bauer M, Hennig P. Laplace redux-effortless bayesian deep learning. Advances in Neural Information Processing Systems. 2021 Dec 6;34:20089-103.
https://github.com/AlexImmer/Laplace

Predictions with Laplace or variational

- We have a posterior $p(\boldsymbol{w}|\boldsymbol{X}, \boldsymbol{y})$, but what about the posterior predictive?

$$p(y|\boldsymbol{x}, \boldsymbol{X}, \boldsymbol{y}) = \int p(y|\boldsymbol{x}, \boldsymbol{w}) p(\boldsymbol{w}|\boldsymbol{X}, \boldsymbol{y})\, d\boldsymbol{w}$$

- Use Monte Carlo

$$\boldsymbol{w}^{(i)} \sim p(\boldsymbol{w}|\boldsymbol{X}, \boldsymbol{y})$$

$$p(y|\boldsymbol{x}, \boldsymbol{X}, \boldsymbol{y}) \approx \sum_i p\left(y|\boldsymbol{x}, \boldsymbol{w}^{(i)}\right)$$

  - Simple
  - General
  - Expensive
  - We lose posterior support

Linearize Laplace or variational

- We have a posterior $p(\boldsymbol{w}|\boldsymbol{X},\boldsymbol{y})$, but what about the posterior predictive?

$$p(y|\boldsymbol{x},\boldsymbol{X},\boldsymbol{y}) = \int p(y|\boldsymbol{x},\boldsymbol{w})p(\boldsymbol{w}|\boldsymbol{X},\boldsymbol{y})\,d\boldsymbol{w}$$

- Use a linear approximation! $y = f(\boldsymbol{x}) = \phi^T(\boldsymbol{x})\boldsymbol{w} + \epsilon$

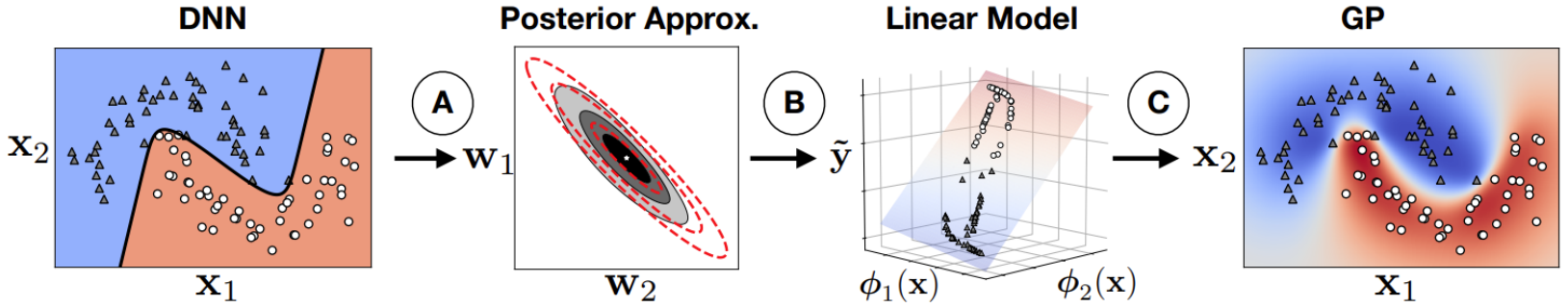
- Use again Taylor

$$f_{\text{lin}}(\mathbf{x}) = f_{\mathbf{w}^*}(\mathbf{x}) + J(\mathbf{x})(\mathbf{w} - \mathbf{w}^*)$$

  - with the Jacobian $J(\mathbf{x}) = \nabla_w(\mathbf{x})|_{\mathbf{w}^*}$

- Then the posterior is: $y_{\text{lin}} \sim \mathcal{N}(f_{\mathbf{w}^*}(\mathbf{x}), J(\mathbf{x})H(\mathbf{w}^*)^{-1}J(\mathbf{x})^T)$

Approximate inference turns deep networs into Gaussian processes



Step A: Find a Gaussian posterior approximation given DNN weights $p(\mathbf{w}|\mathbf{y}, \mathbf{X}) \approx \mathcal{N}(\mathbf{w}|\mu, \mathbf{\Sigma})$

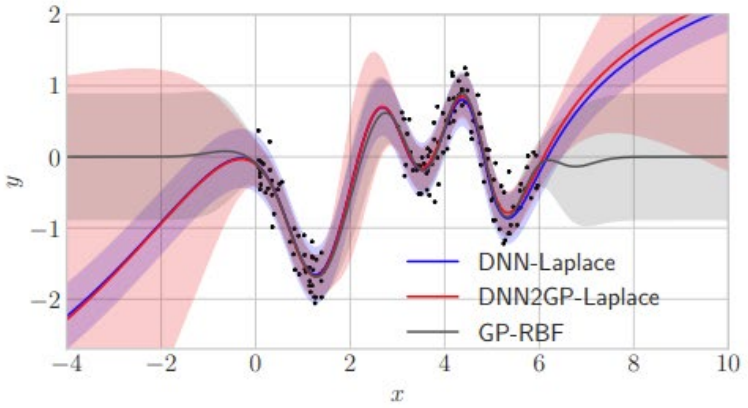
Step B: Find a linear model $\tilde{y} = \boldsymbol{\phi}(\mathbf{x})^\top \mathbf{w} + \epsilon$ with outputs $\tilde{y}$, features $\boldsymbol{\phi}(\mathbf{x})$, and noise ϵ, such that its posterior is equal to the Gaussian approximation, i.e., $p(\mathbf{w}|\tilde{\mathbf{y}}, \mathbf{X}) = \mathcal{N}(\mathbf{w}|\mu, \mathbf{\Sigma})$

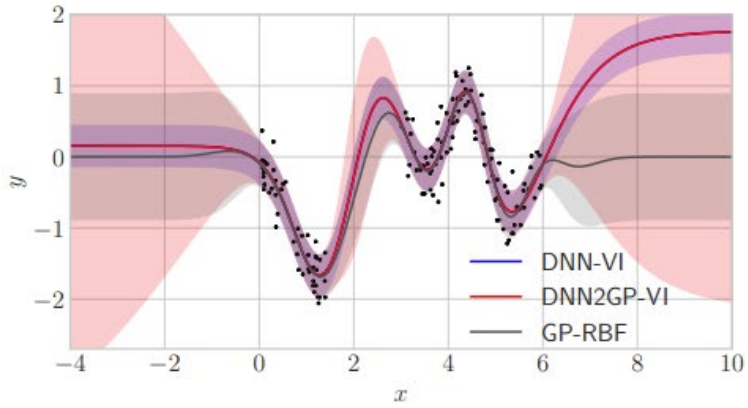
Step C: Find a GP whose predictions are the same as those of the linear model.

$$\tilde{\mathbf{y}} = \mathbf{f}(\mathbf{x}) + \boldsymbol{\epsilon}$$

$$\mathbf{f}(\mathbf{x}) \sim \mathcal{GP}\left(0, \delta^{-1}\mathbf{J}_*(\mathbf{x})\mathbf{J}_*(\mathbf{x}')^\top\right)$$

Neural Tangent Kernel (NTK)





Khan 2019: https://proceedings.neurips.cc/paper_files/paper/2019/file/b3bbccd6c008e727785cb81b1aa08ac5-Paper.pdf

Bayesian logistic regression

- The logistic regression model represents the probability of a binary output $y_i \in \{0, 1\}$ given the input $\mathbf{x}_i$:

$$p(\mathbf{y}|\mathbf{X}, \theta) = \prod_{i=1}^{n} Ber(y_i|sigm(\theta^T \mathbf{x}_i)) = \prod_{i=1}^{n} \left[\frac{1}{1+e^{-\theta^T \mathbf{x}_i}}\right]^{y_i} \left[1 - \frac{1}{1+e^{-\theta^T \mathbf{x}_i}}\right]^{1-y_i}$$

- We also assume a Gaussian prior:

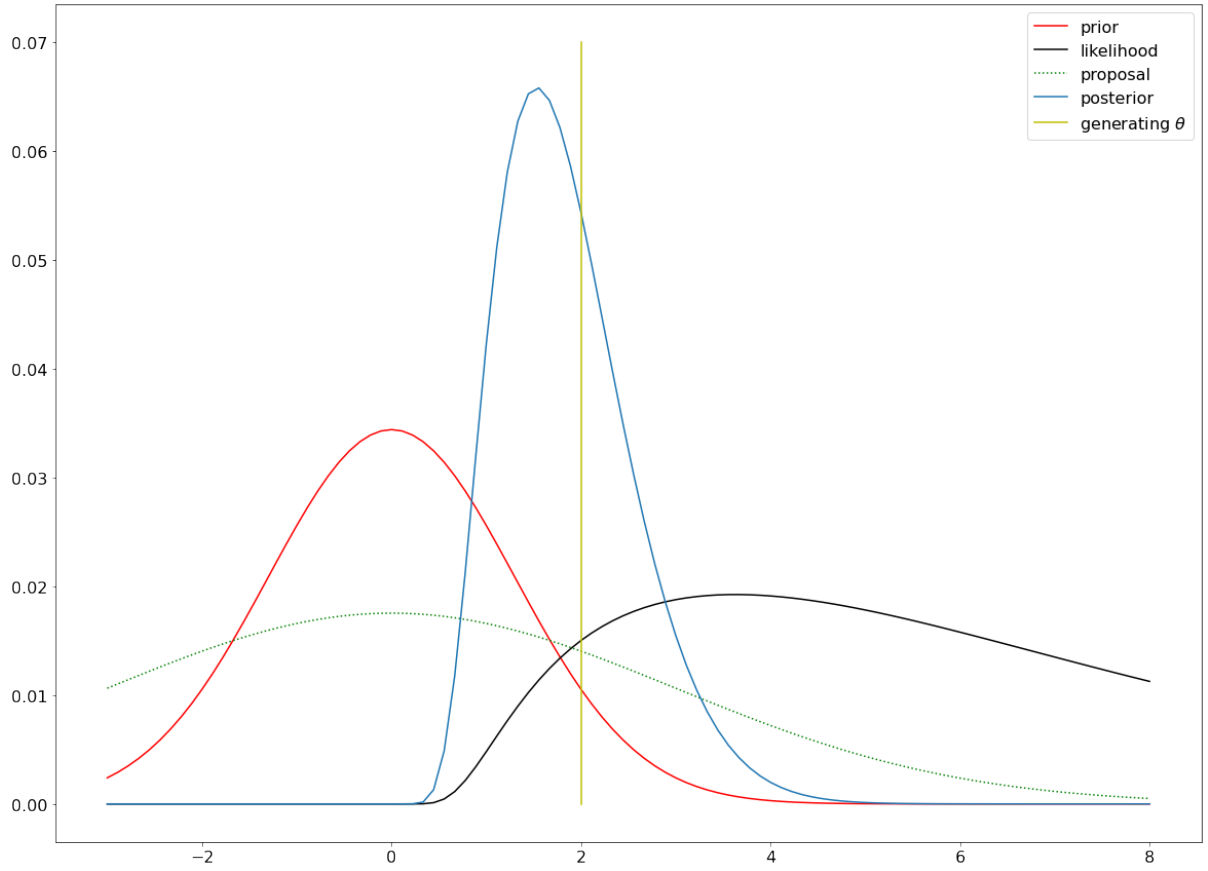
$$p(\theta) = \frac{1}{\sqrt{2\pi\sigma^2}} exp\left(-\frac{1}{2\sigma^2}(\theta - \mu)^T(\theta - \mu)\right)$$

- What is the posterior? What is the predictive posterior?

Bayesian logistic regression

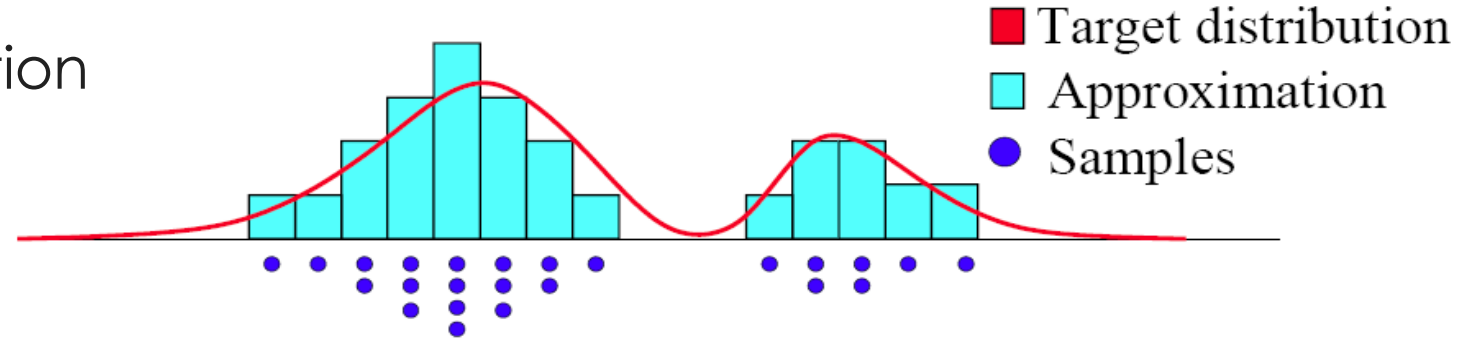
$$p(\theta|\mathbf{X},\mathbf{y}) = \frac{p(\mathbf{y}|\mathbf{X},\theta)p(\theta)}{\int p(\mathbf{y}|\mathbf{X},\theta)p(\theta)d\theta} = \frac{1}{Z}p(\mathbf{y}|\mathbf{X},\theta)p(\theta)$$

$$p(y_*|\mathbf{x}_*,\mathbf{X},\mathbf{y}) = \int p(y_*|\mathbf{x}_*,\theta)p(\theta|\mathbf{X},\mathbf{y})d\theta$$

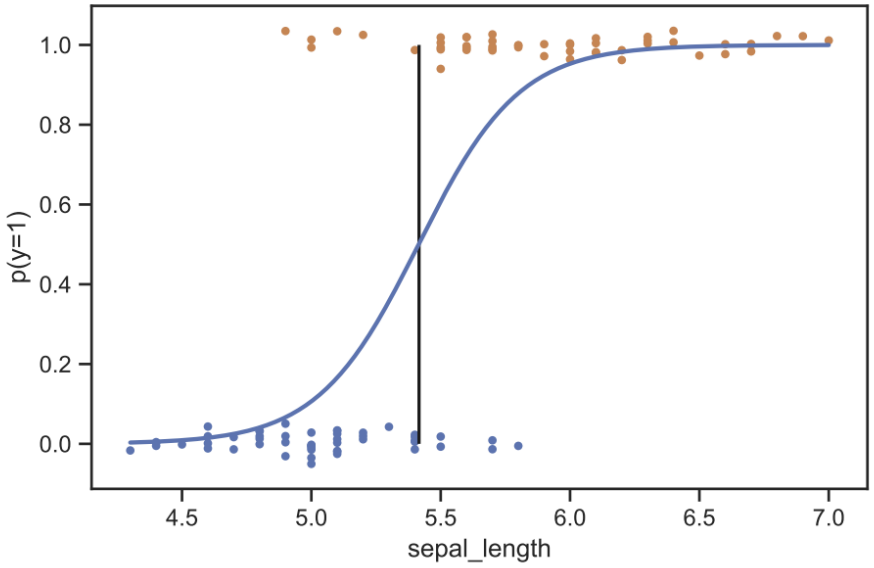


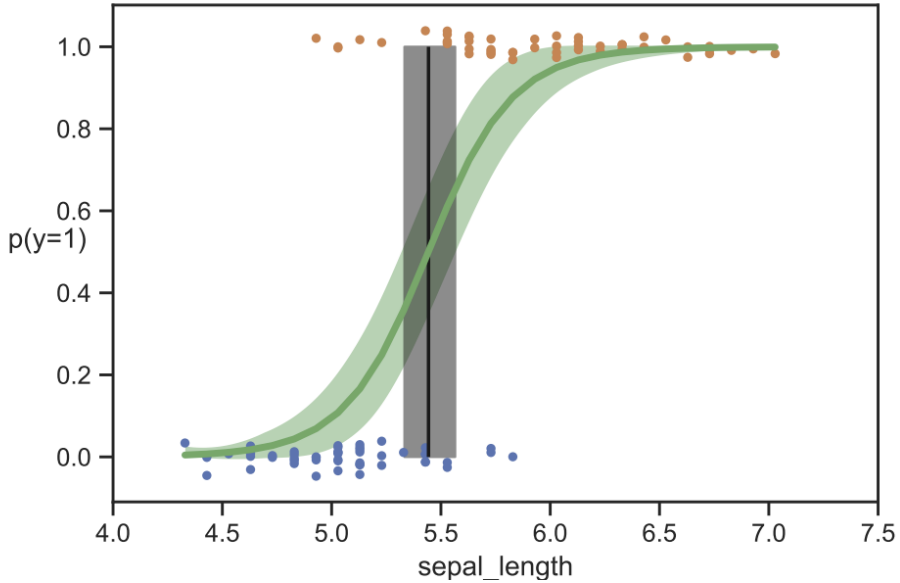
Monte Carlo Principle

- Approximate a distribution



$$I_N(f) = \frac{1}{N}\sum_{i=1}^{N} f(x^{(i)}) \quad \xrightarrow[N \to \infty]{a.s.} \quad I(f) = \int_{\mathcal{X}} f(x)p(x)dx$$



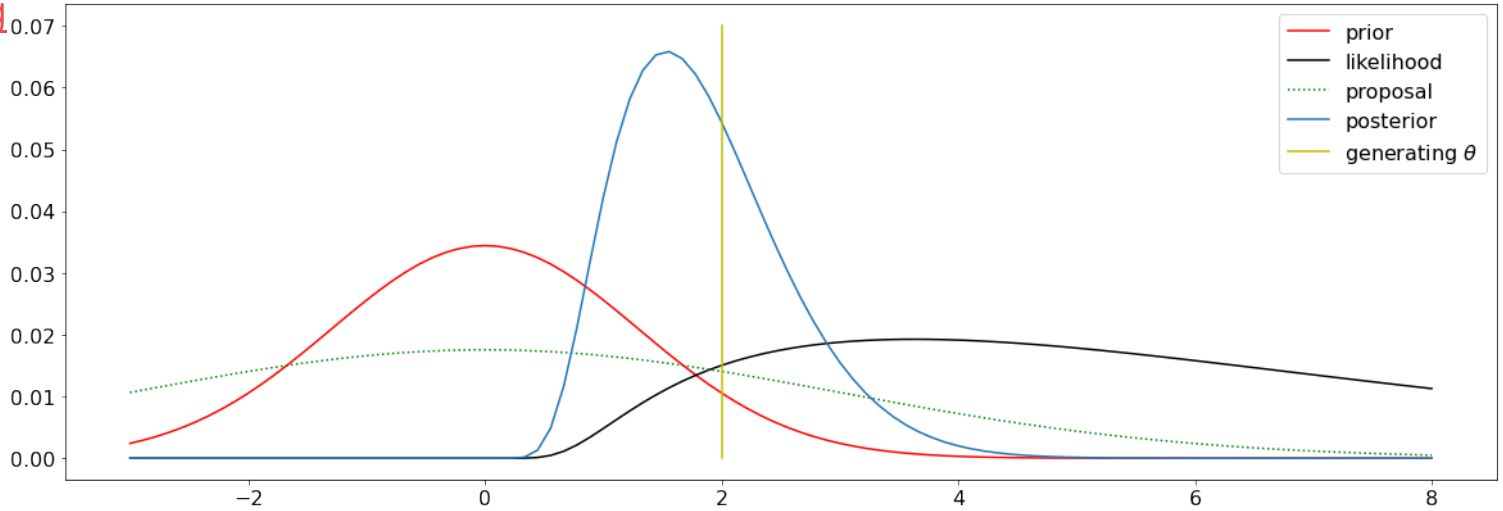


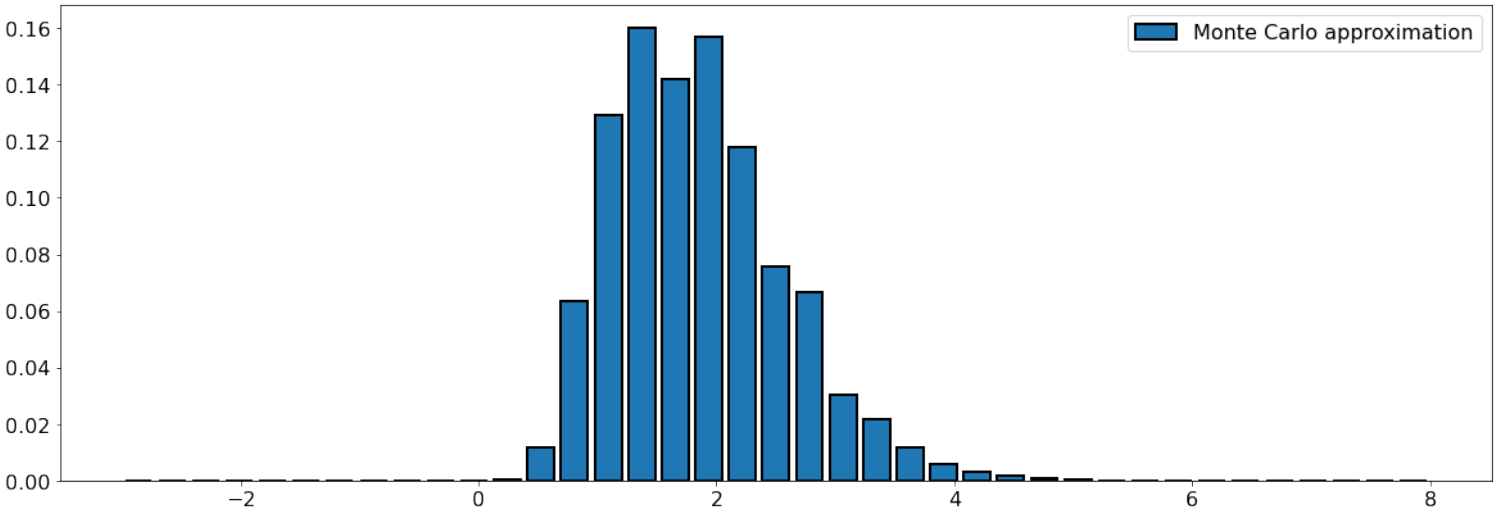
Universidad Zaragoza

Kevin P. Murphy, Probabilistic Machine Learning: An introduction, section 4.6.7.2, MIT Press, 2021, https://probml.github.io/pml-book/

Importance sampling

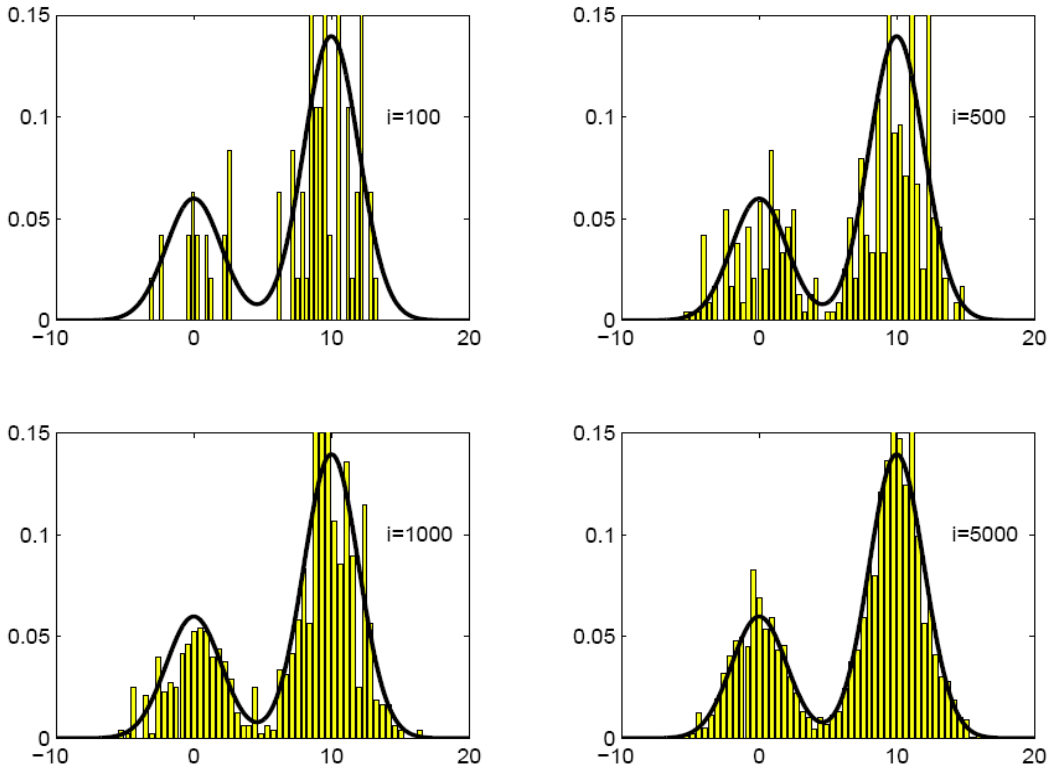
https://colab.research.google.com/drive/1AJmsrr6fWRro1GjYtfKRVpMBhXmKTzm6?usp=sharing





MH Demos

- https://chi-feng.github.io/mcmc-demo/
- https://drive.google.com/file/d/1w9fPZMuy4T5OAaETS5q6vooj7SxL4jp4/view?usp=sharing



Sample representation of neural networks

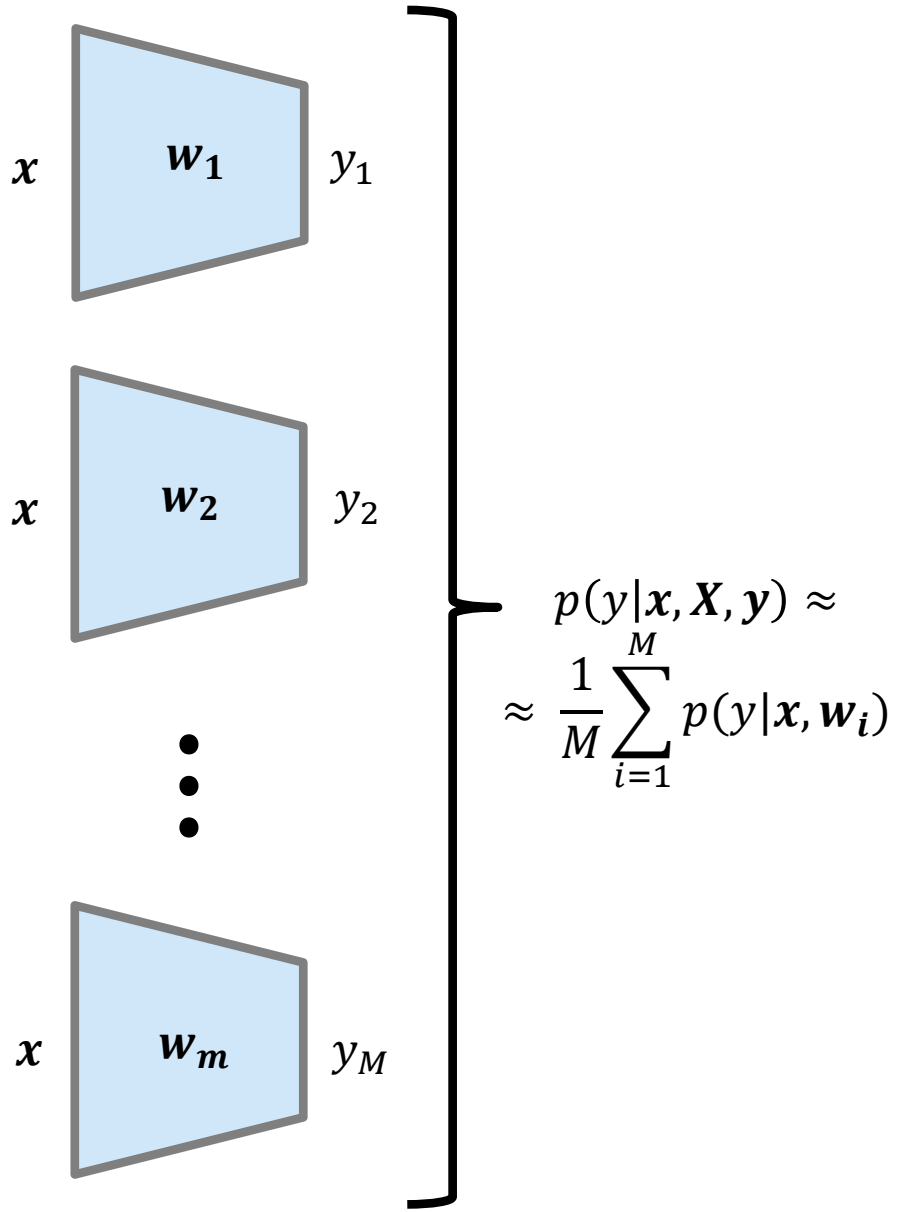
- Now we have a sample representation of the posterior $\{w_i\}_{i=1}^{M}$

$$p(\boldsymbol{w}|\mathbf{X},\mathbf{y}) \approx \frac{1}{M}\sum_{i=1}^{M}\delta_{w_i}(\boldsymbol{w})$$

- and the posterior predictive becomes

$$p(y|\boldsymbol{x},\boldsymbol{X},\boldsymbol{y}) \approx \frac{1}{M}\sum_{i=1}^{M}p(y|\boldsymbol{x},\boldsymbol{w}_i)$$

- We assume that we have a different network per sample. We can compute the prediction as the average of each network prediction.
- We can only use a small number of samples! $\mathcal{O}(M)$



Uncertainty representation with samples

- Regression: Gaussian prediction $[y_i, \sigma_{y,i}] = f(\mathbf{x}, \mathbf{w}_i)$

$$\sigma^2 = \underbrace{\frac{1}{M}\sum_{i=1}^{M}(y_i - \bar{y})^2}_{\text{epistemic}} + \underbrace{\frac{1}{M}\sum_{i=1}^{M}\sigma_{y,i}^2}_{\text{aleatoric}}$$

$$\bar{y} = \frac{1}{M}\sum_{i=1}^{M} y_i$$

- Classification: Categorical prediction $\mathbf{p}_i = f(\mathbf{x}, \mathbf{w}_i)$

$$\sigma^2 = \underbrace{\frac{1}{M}\sum_{i=1}^{M}(\mathbf{p}_i - \bar{\mathbf{p}})(\mathbf{p}_i - \bar{\mathbf{p}})^T}_{\text{epistemic}} + \underbrace{\frac{1}{M}\sum_{i=1}^{M}\text{diag}(\mathbf{p}_i) - \mathbf{p}_i\mathbf{p}_i^T}_{\text{aleatoric}}$$

$$\bar{\mathbf{p}} = \frac{1}{M}\sum_{i=1}^{M}\mathbf{p}_i$$

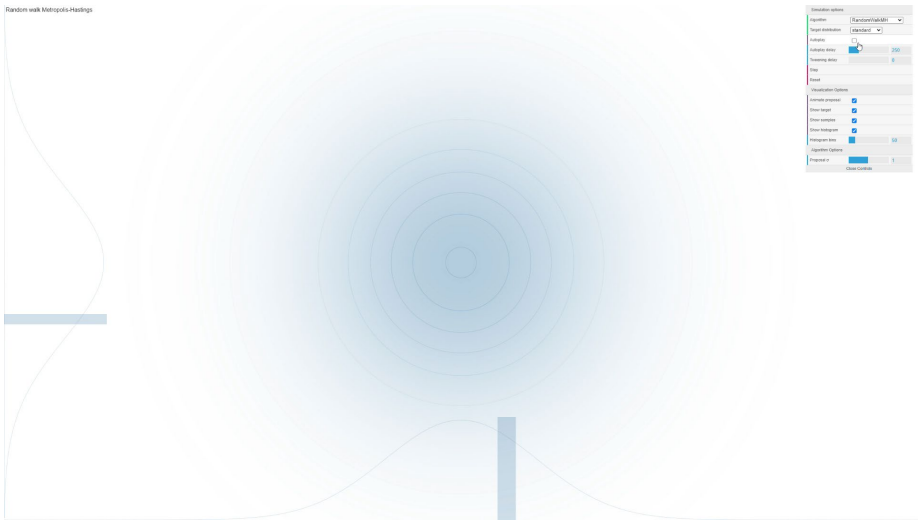
Hamiltonian Monte Carlo

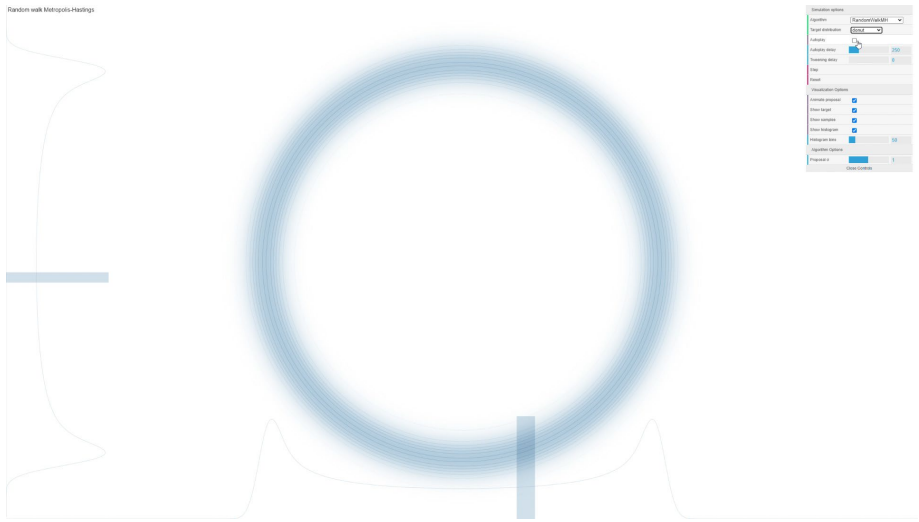
Radford M. Neal

- Let your particles follow dynamics (gradient).
- Dynamics on $-\log p(x)$.
- Dynamics == gradient
- First Bayesian neural network.
- Much more efficient than MCMC.

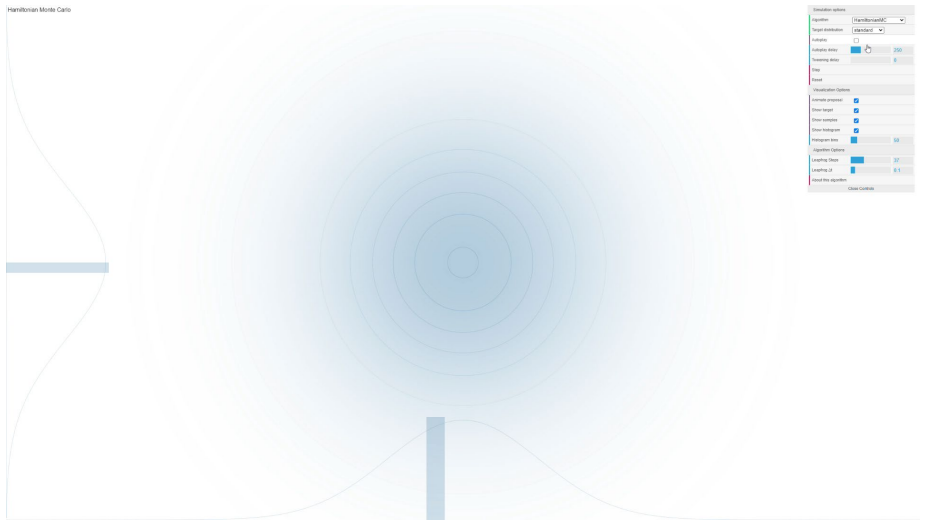
- Many variants (eg: Langevin Dynamics)

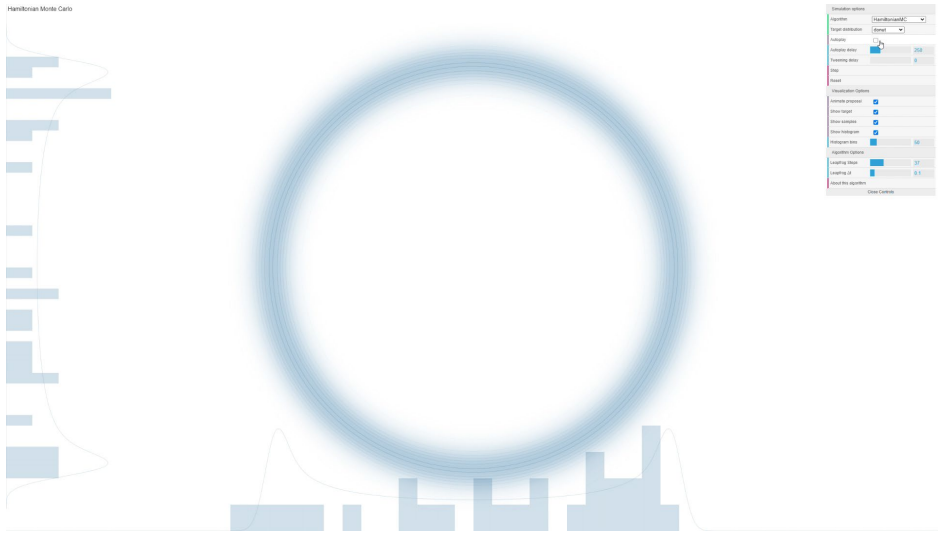
- MCMC





- Hamiltonian MC





Universidad Zaragoza

Stochastic Gradient Langevin Dynamics

Gradient ascent

$$\Delta \mathbf{w}_t = \frac{\epsilon}{2}\left(\nabla \log p(\mathbf{w}_t) + \sum_{i=1}^{N} \nabla \log p(y_i|\mathbf{w}_t, \mathbf{x}_i)\right)$$

MCMC-Langevin Dynamics

$$\Delta \mathbf{w}_t = \frac{\epsilon}{2}\left(\nabla \log p(\mathbf{w}_t) + \sum_{i=1}^{N} \nabla \log p(y_i|\mathbf{w}_t, \mathbf{x}_i)\right) + \eta_t$$

Metropolis-Hasting accept step $\quad \eta_t \sim \mathcal{N}(0, \epsilon)$

Stochastic gradient ascent

$$\Delta \mathbf{w}_t = \frac{\epsilon_t}{2}\left(\nabla \log p(\mathbf{w}_t) + \frac{N}{n}\sum_{i=1}^{n} \nabla \log p(y_{ti}|\mathbf{w}_t, \mathbf{x}_{ti})\right)$$

$$\sum_{t=1}^{\infty} \epsilon_t = \infty \qquad \sum_{t=1}^{\infty} \epsilon_t^2 < \infty$$

Stochastic Gradient Langevin Dynamics

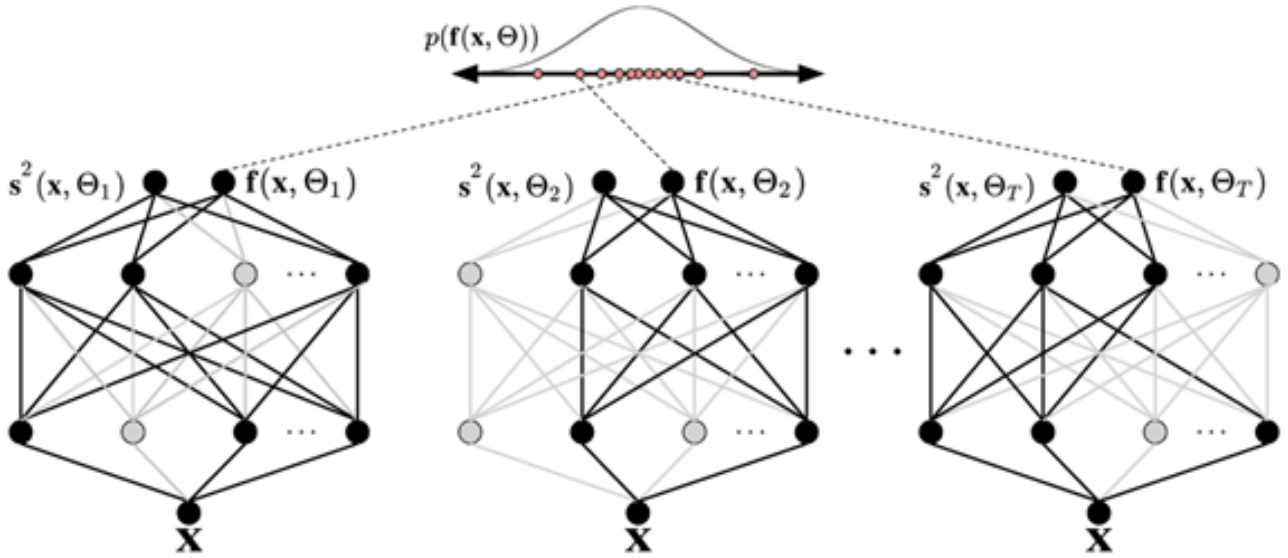
$$\Delta \mathbf{w}_t = \frac{\epsilon_t}{2}\left(\nabla \log p(\mathbf{w}_t) + \frac{N}{n}\sum_{i=1}^{n} \nabla \log p(y_{ti}|\mathbf{w}_t, \mathbf{x}_{ti})\right) + \eta_t$$

$$\sum_{t=1}^{\infty} \epsilon_t = \infty \qquad \sum_{t=1}^{\infty} \epsilon_t^2 < \infty \qquad \eta_t \sim \mathcal{N}(0, \epsilon_t)$$

~~Metropolis-Hasting accept step~~

Monte Carlo Dropout

- MC dropout is a simple and scalable approach:
  - Instead of sampling the whole weight space, sample if a weight is active or not.
  - We can still use the optimal/trained values for the weights → Good performance.
  - M forward passes with dropout at test time $\boldsymbol{w}_d$
  - We can use a pretrained network. No training overhead.



Y. Gal and Z. Ghahramani, Dropout as a Bayesian approximation: Representing model uncertainty in deep learning, ICML, 2016

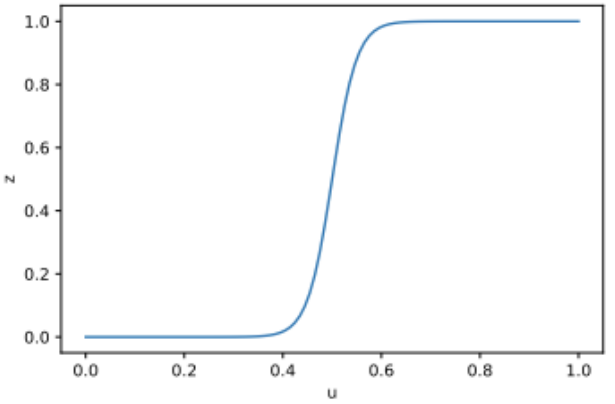
Behind the curtain: MC Dropout

- We are actually doing variational inference on a deep GP
  - The approximate distribution q(w) is a distribution over matrices whose columns are randomly set to zero

$$q(\mathbf{w}) \Rightarrow \mathbf{W}_i = \mathbf{M}_i \cdot \text{diag}\left(\left[\mathbf{z}_{i,j}\right]_{j=1}^{K_i}\right)$$

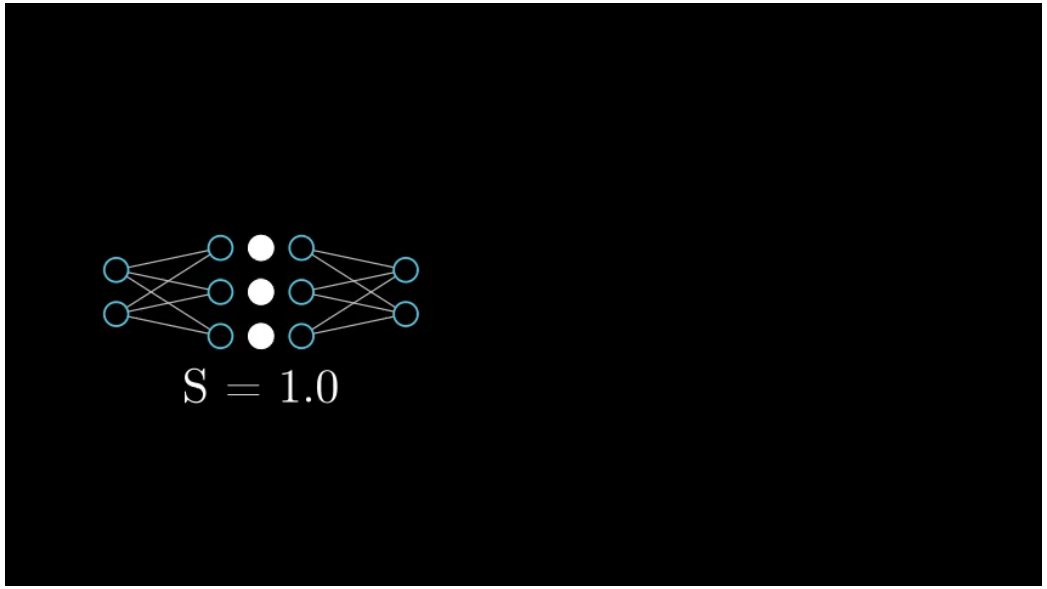
$$\mathbf{z}_{i,j} \sim \text{Bernoulli}(p_i)$$

  - The binary variable $z_{i,j} = 0$ corresponds to unit *j* in layer *i-1* being dropped out as an input to layer *i*.



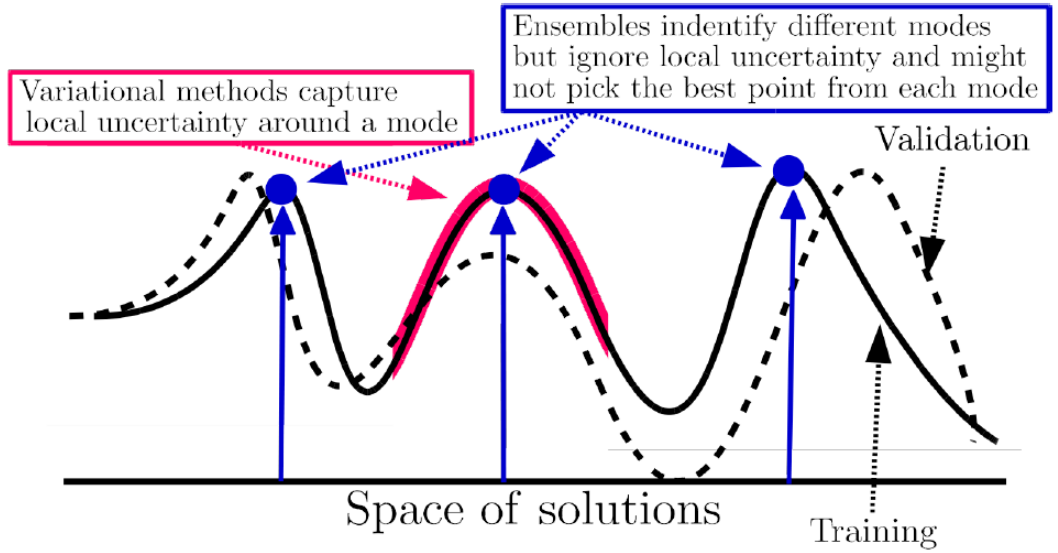
Concrete distribution

- Variants:
  - Better sampling methods
    - Replace Bernoulli with Concrete distribution.
    - Concrete = CONtinuous to disCRETE.
  - Masksensembles



Deep ensembles

- Sample approximation:
  - Oriented to small sample sets
  - Good performance
  - Can be applied to any network or algorithm (CNN, RNN, supervised, RL…)

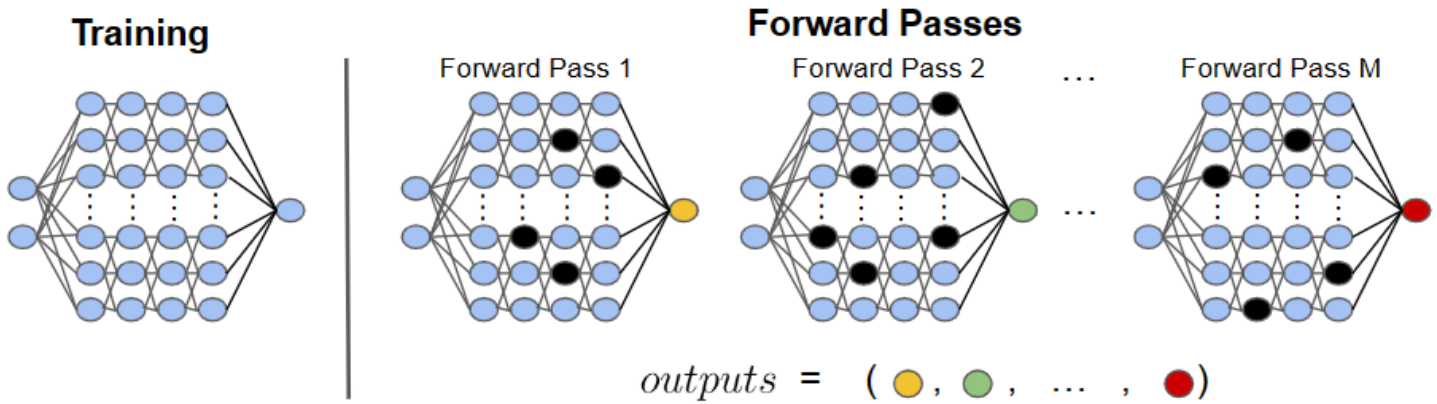


- M point estimates $\boldsymbol{w}_e$ are learned minimizing a loss (e.g.: ML, MAP), starting from different seeds and shuffling the data.
  - Expensive training and memory (M different networks)

- Trained networks capture diversity in local solutions.
- In principle, not true posterior
  - Bayesian model averaging
  - Better than local approximations

B. Lakshminarayanan, A. Pritzel, and C. Blundell, Simple and scalable predictive uncertainty estimation using deep ensembles, NeurIPS, 2017

MC Dropout vs Deep ensembles



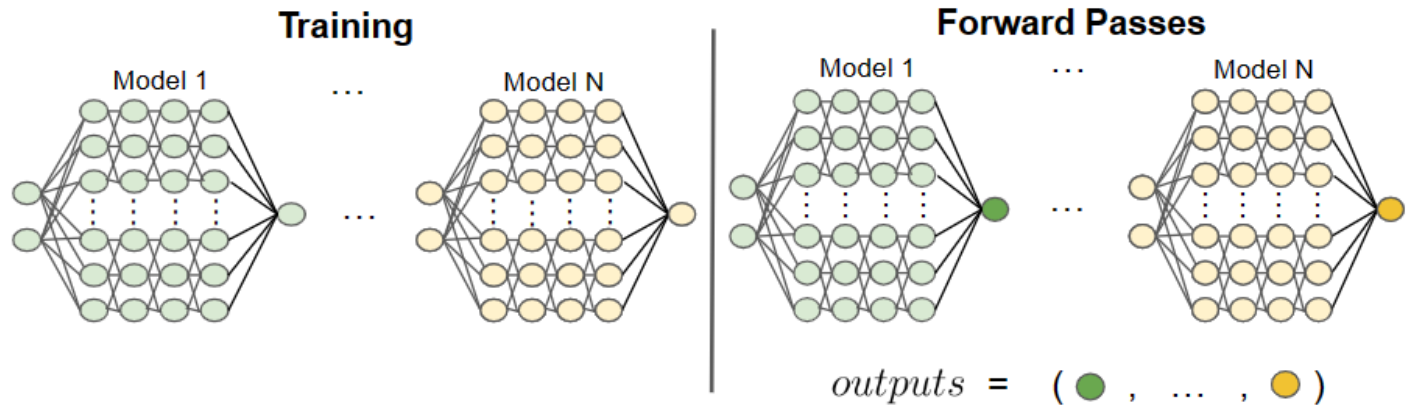
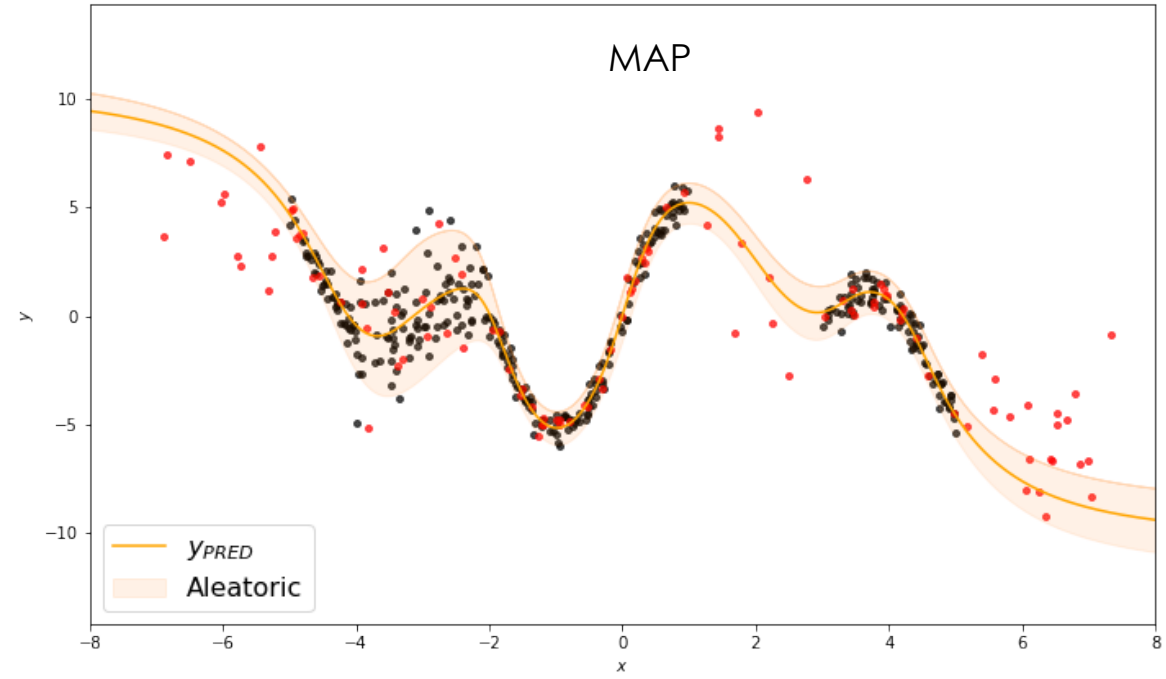
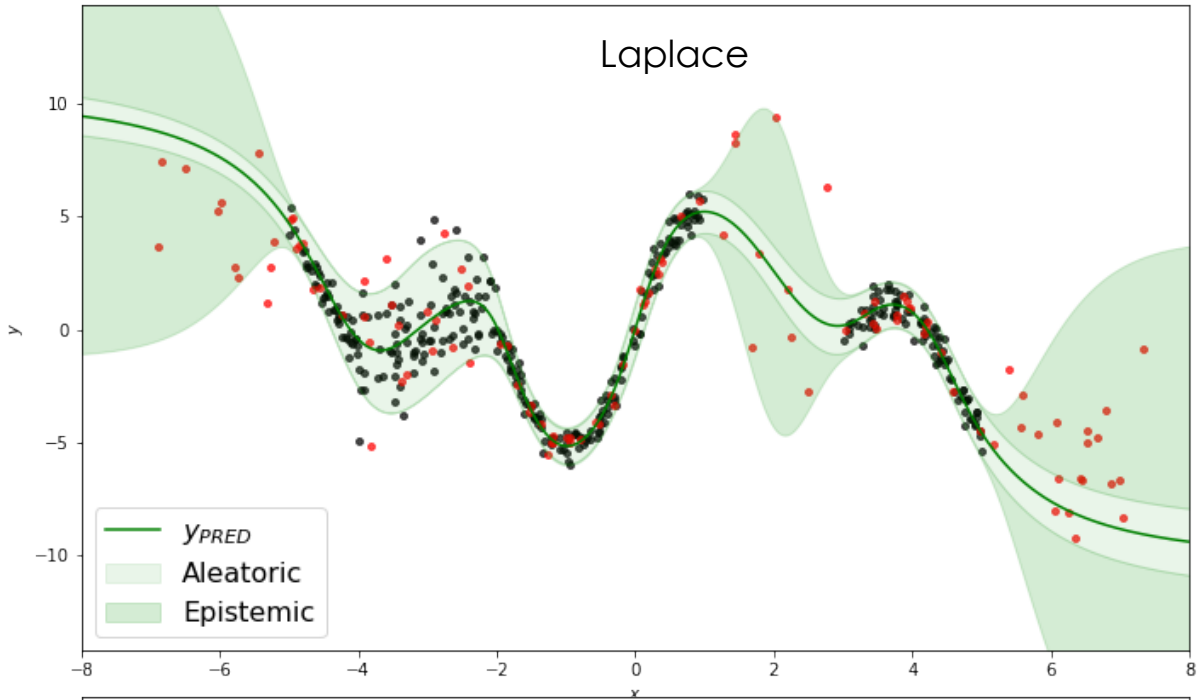
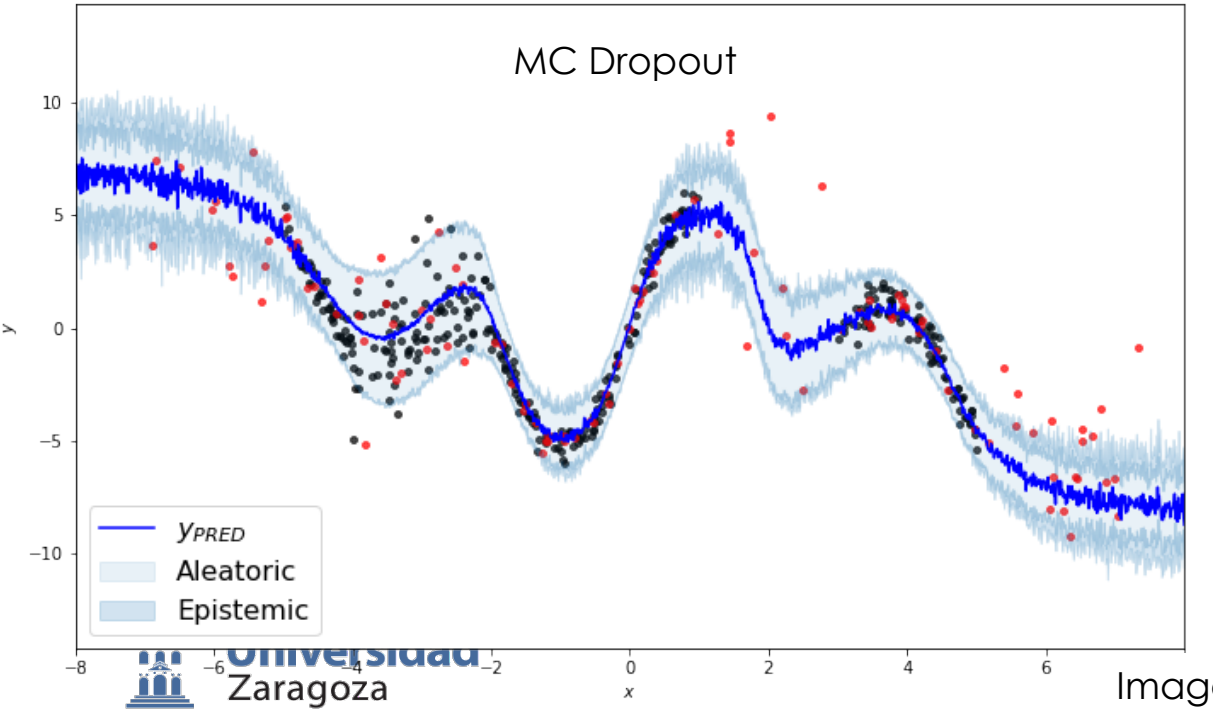
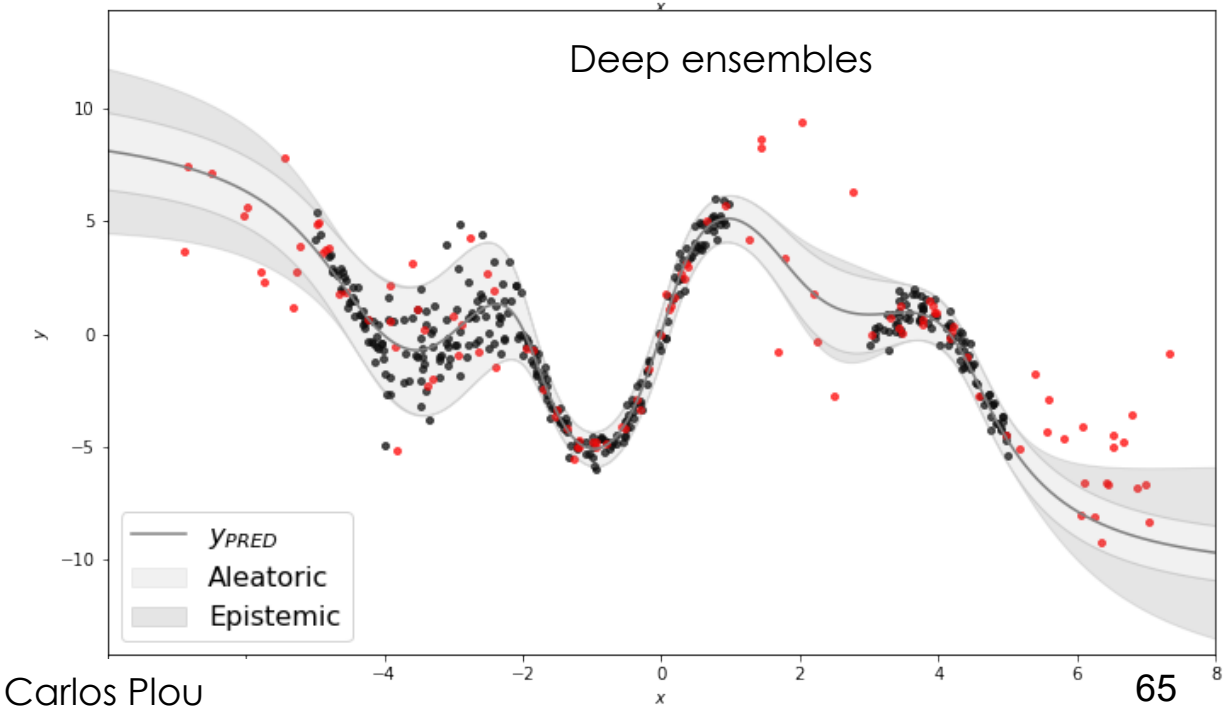


Image: Carlos Plou









Universidad Zaragoza

Images: Carlos Plou

Prior for MC Dropout or Deep Ensembles

- How are we defining the prior?

$$\mathcal{L}_{MAP} = -\left[\sum_{i=1}^{N} \log(p(y^{(i)}|\boldsymbol{x}^{(i)}, \boldsymbol{w})) + \log p(\boldsymbol{w})\right]$$

- Remember L2 regularization?

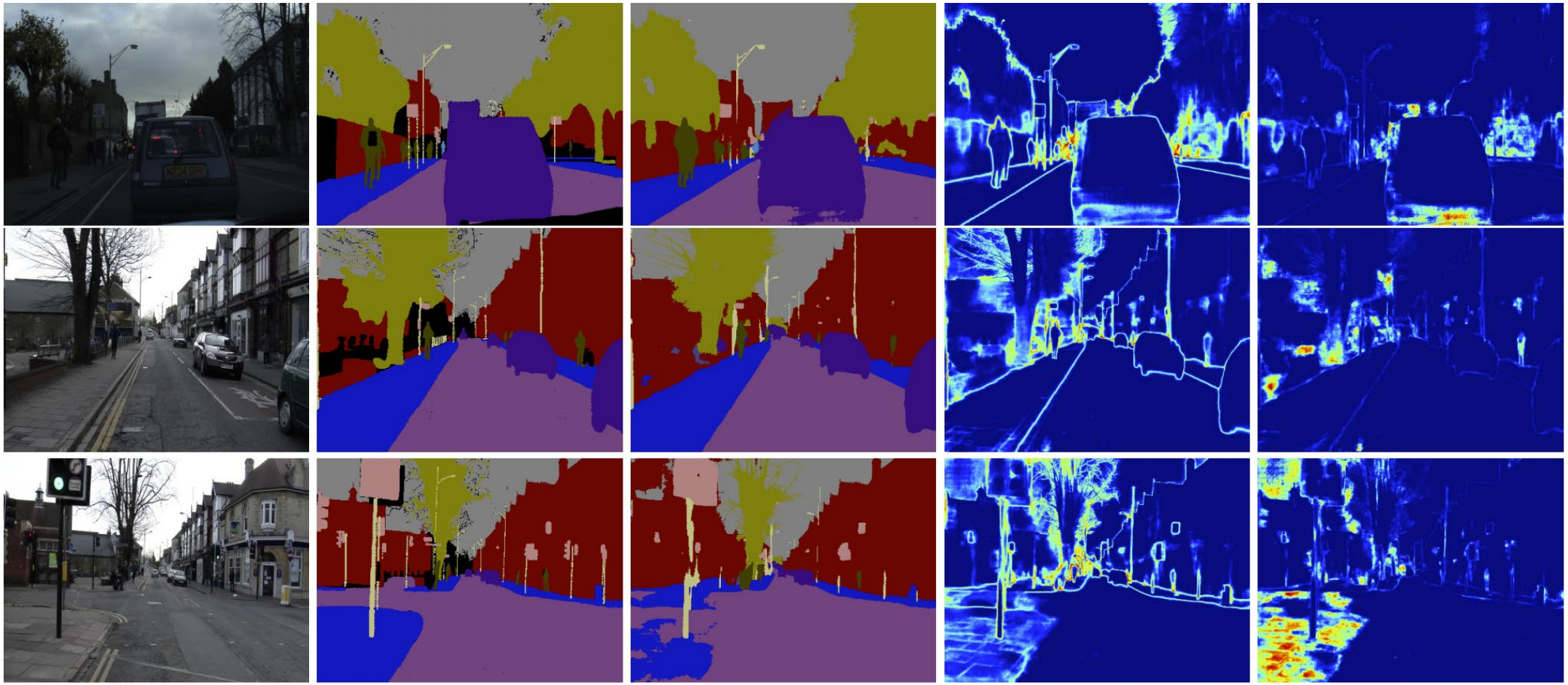
$$\mathcal{L}_{REG} = \mathcal{L}_{ML} + \frac{\lambda}{2}\|\boldsymbol{w}\|^2$$

- Take a Gaussian likelihood

$$\mathcal{L}_{ML} = \frac{1}{N}\sum_{i=1}^{N} \frac{\|\mu_{\boldsymbol{w}} - y\|^2}{2\sigma_{\boldsymbol{w}}^2} + \frac{1}{2}\log(\sigma_{\boldsymbol{w}}^2)$$

- Then, $\mathcal{L}_{REG} = \mathcal{L}_{MAP}$ where the prior is a Gaussian $p(\boldsymbol{w}) \sim N(0, \lambda^{-1}\boldsymbol{I})$
- We can use more advanced priors for specific problems
  - See K.P. Murphy Book2 Chapter 17 https://probml.github.io/pml-book/book2.html

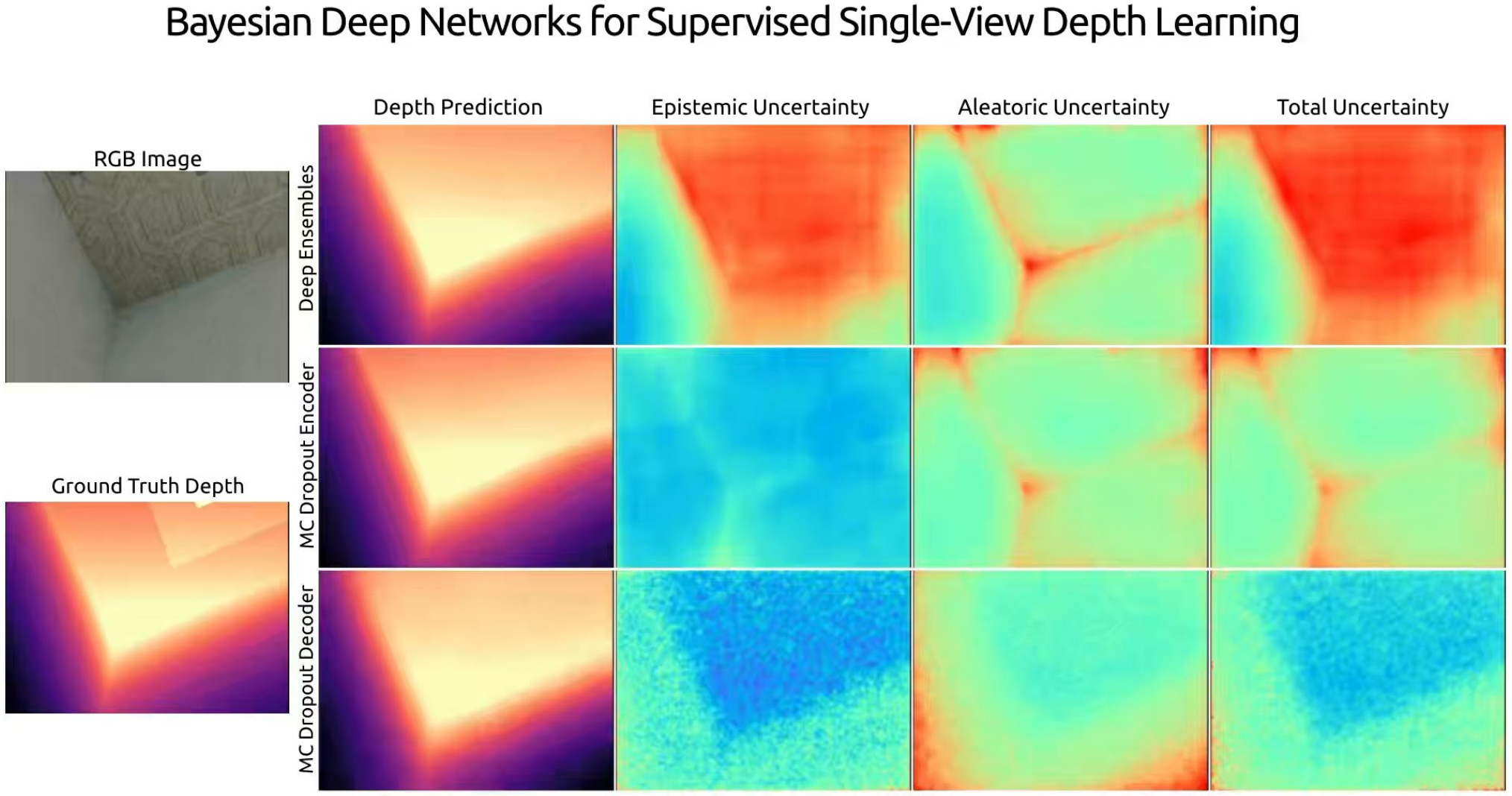
Segmentation examples



(a) Input Image (b) Ground Truth (c) Semantic Segmentation (d) Aleatoric Uncertainty (e) Epistemic Uncertainty

Alex Kendall and Yarin Gal. What Uncertainties Do We Need in Bayesian Deep Learning for Computer Vision?, NeurIPS, 2017

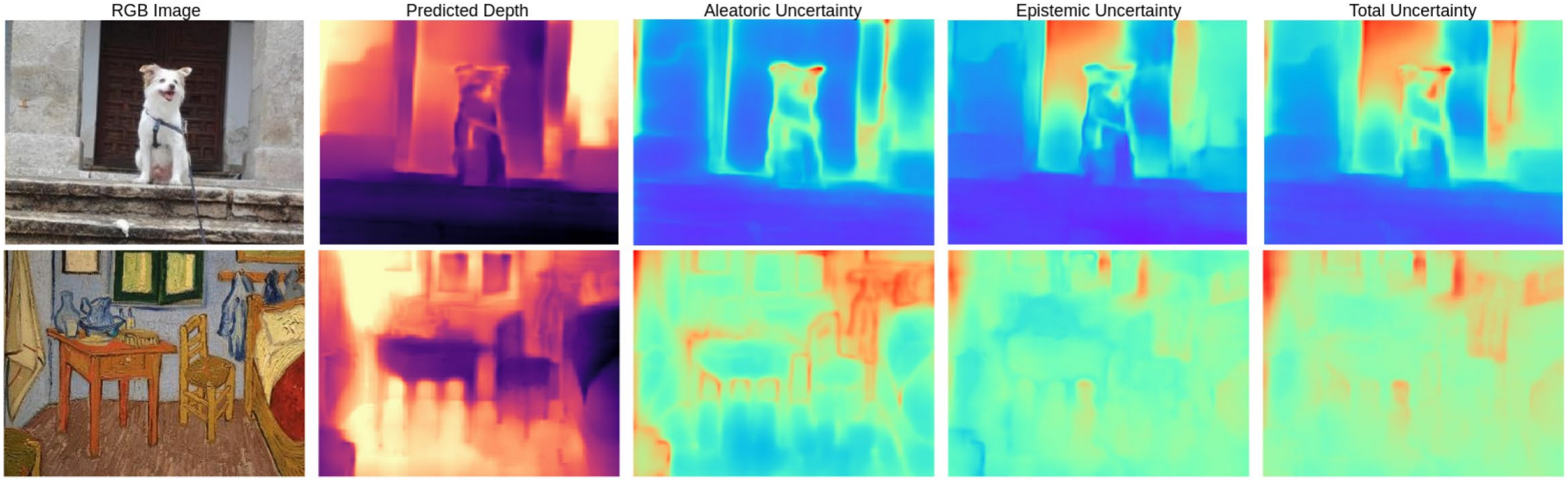
Depth prediction examples



Javier Rodríguez-Puigvert, Rubén Martínez-Cantín, Javier Civera, Bayesian Deep Networks for Supervised Single-View Depth Learning, IEEE-RAL, 2021

Depth prediction examples

- Out-of-distribution images – the network was trained in images showing empty rooms indoors



Javier Rodríguez-Puigvert, Rubén Martínez-Cantín, Javier Civera, Bayesian Deep Networks for Supervised Single-View Depth Learning, IEEE-RAL, 2021

Segmentation and depth prediction examples

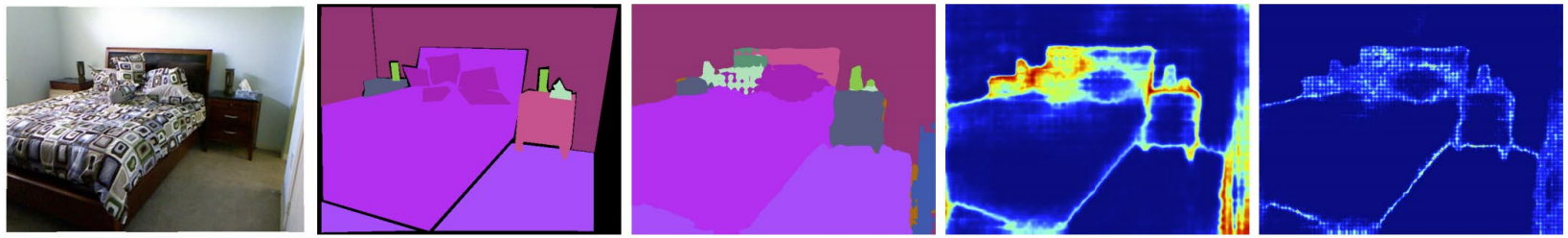


Figure 4: NYUv2 40-Class segmentation. From top-left: input image, ground truth, segmentation, aleatoric and epistemic uncertainty.

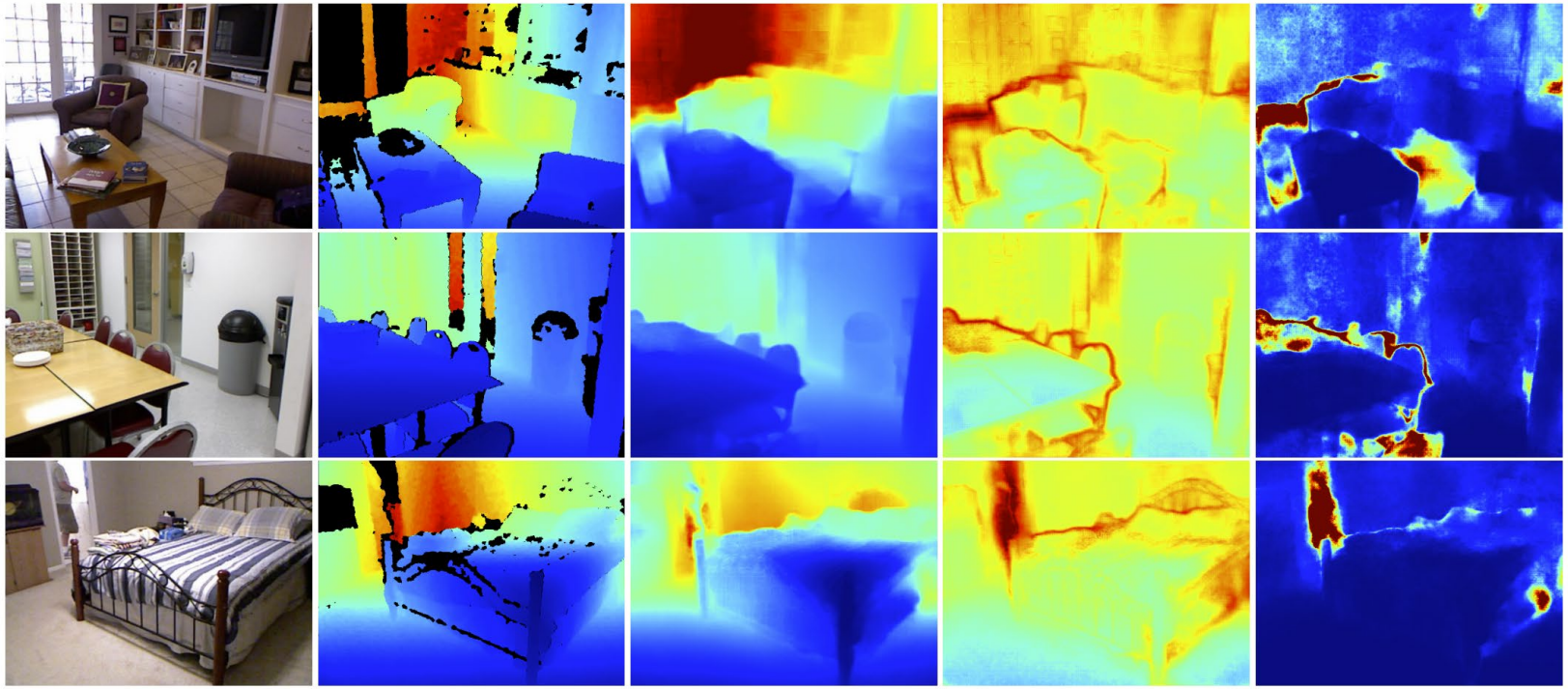
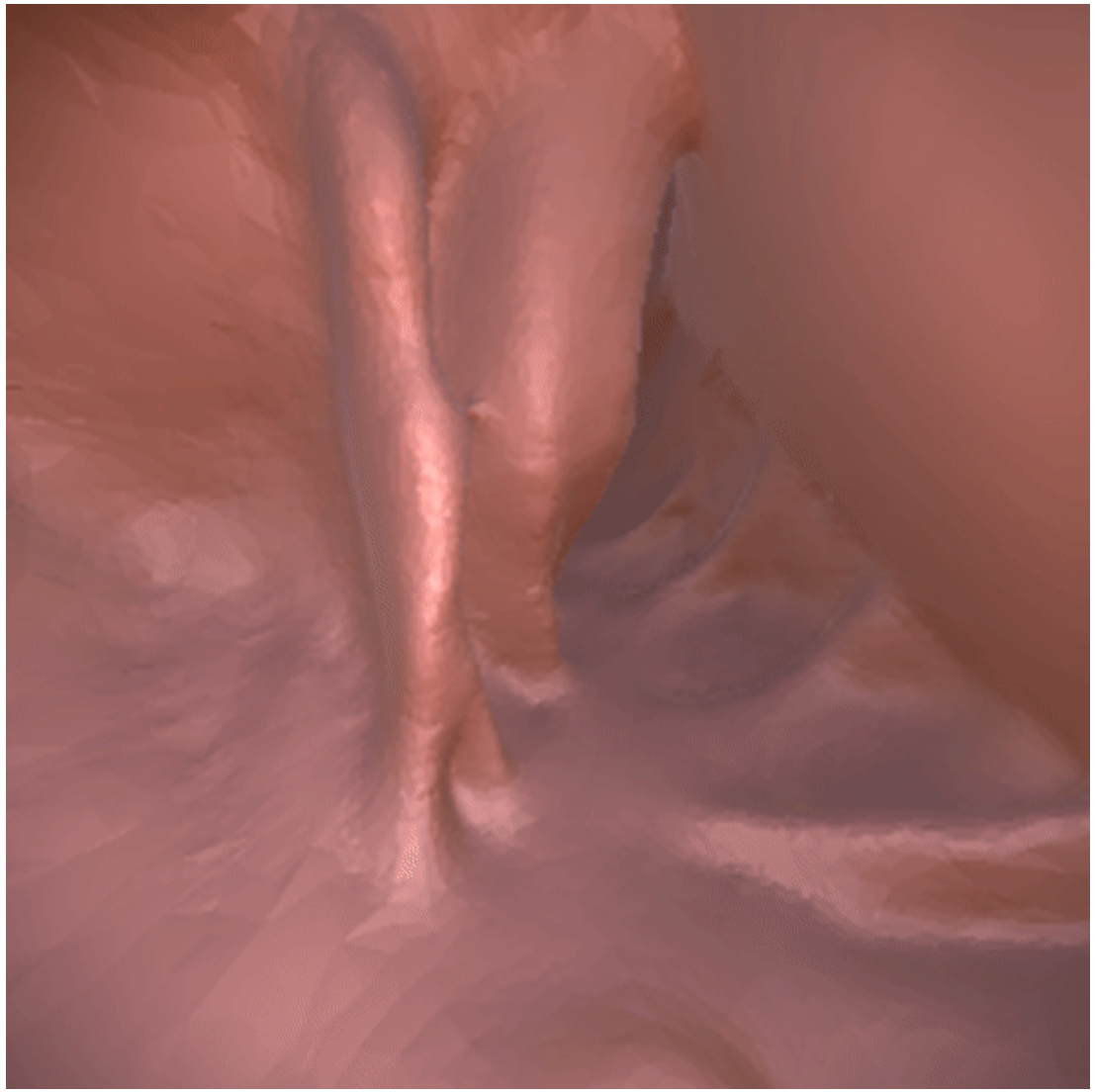


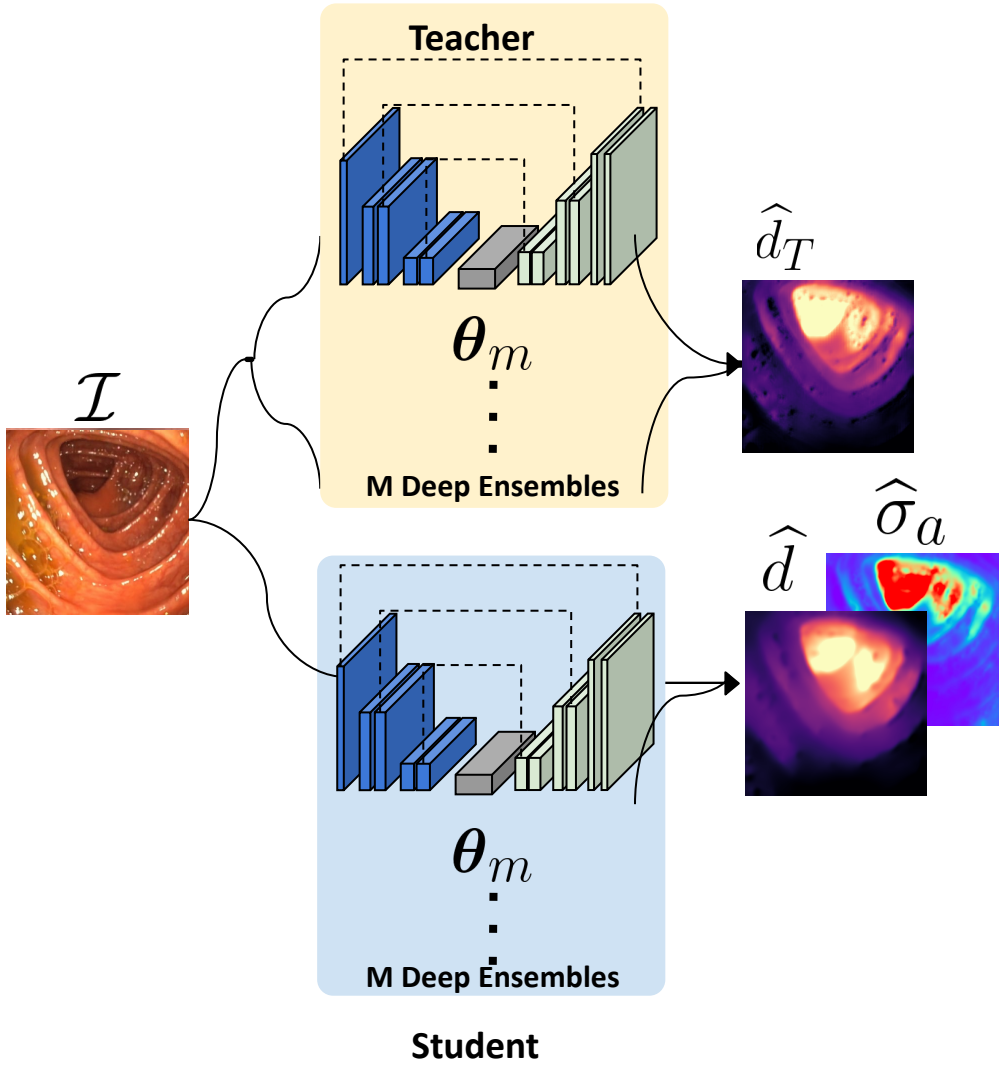
Figure 5: NYUv2 Depth results. From left: input image, ground truth, depth regression, aleatoric uncertainty, and epistemic uncertainty.

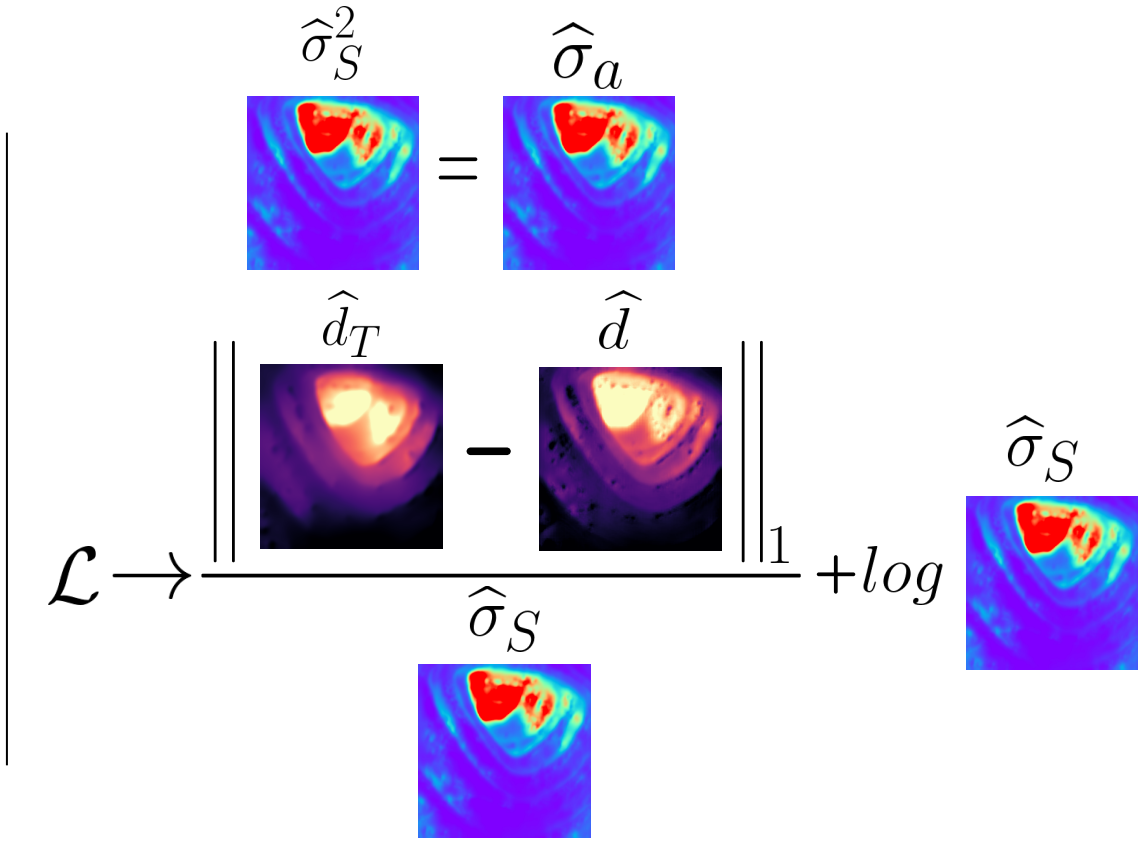
Alex Kendall and Yarin Gal. What Uncertainties Do We Need in Bayesian Deep Learning for Computer Vision?, NeurIPS, 2017

Depth “labels” in colonoscopies

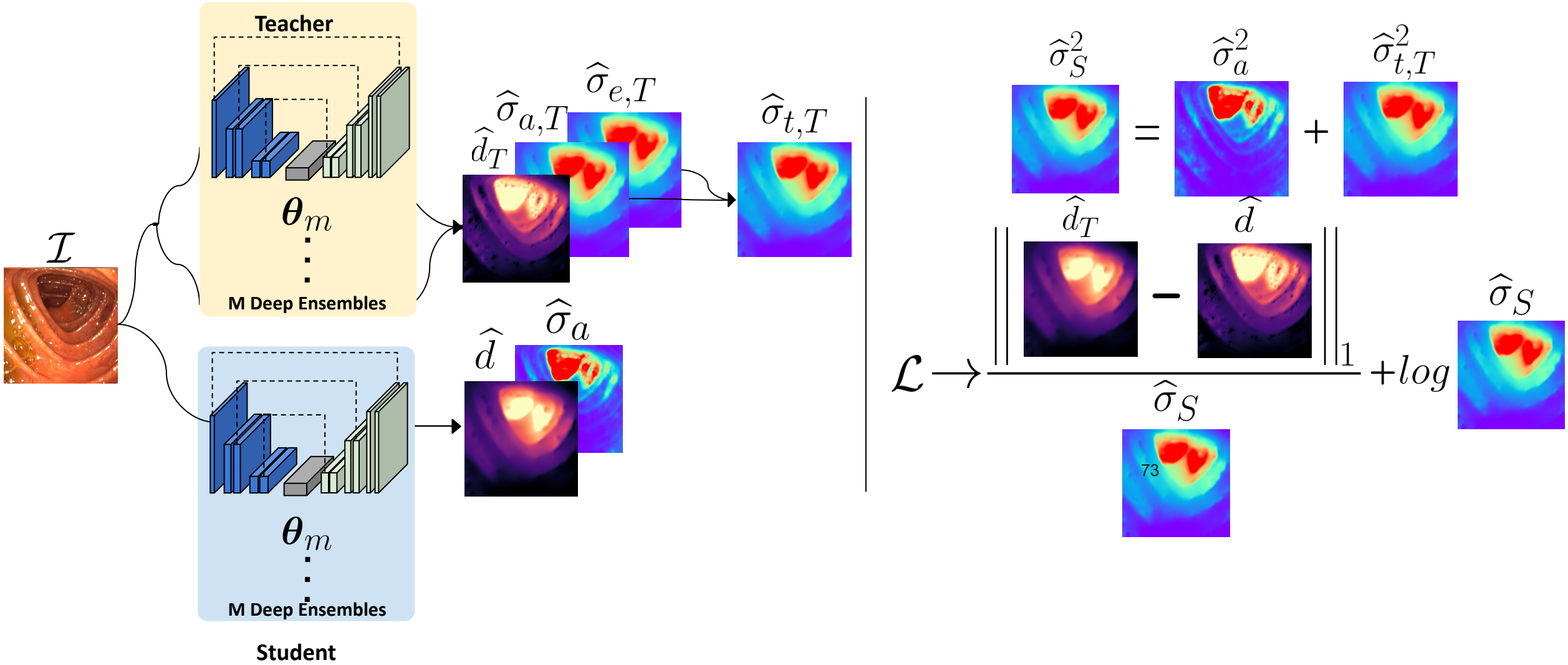


Teacher-Student

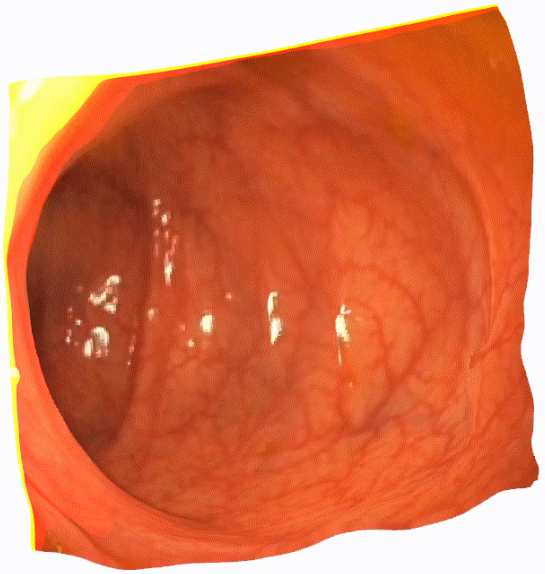




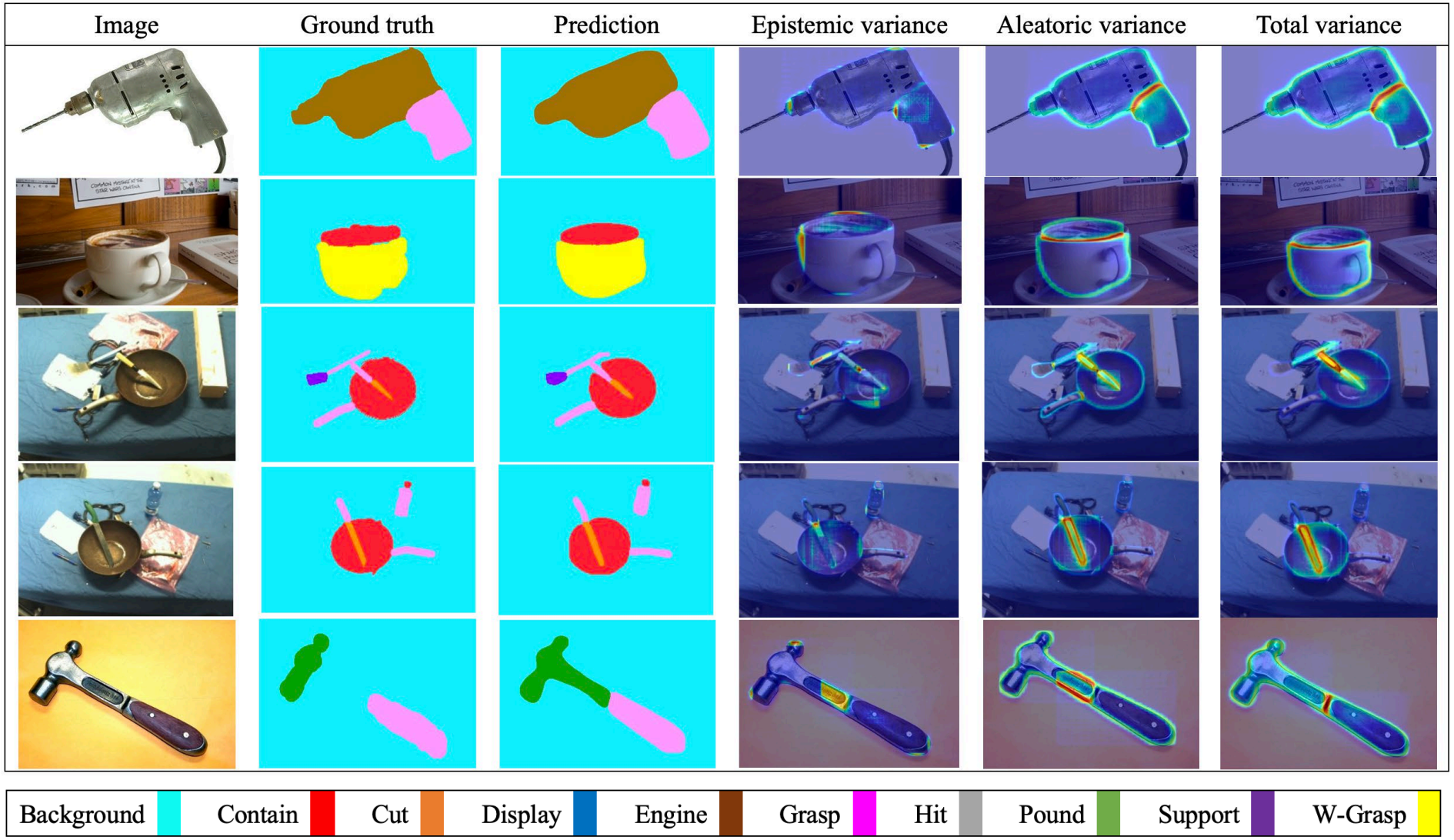
Teacher-Student with Uncertain Teacher



Depth predictions from uncertain teacher-student



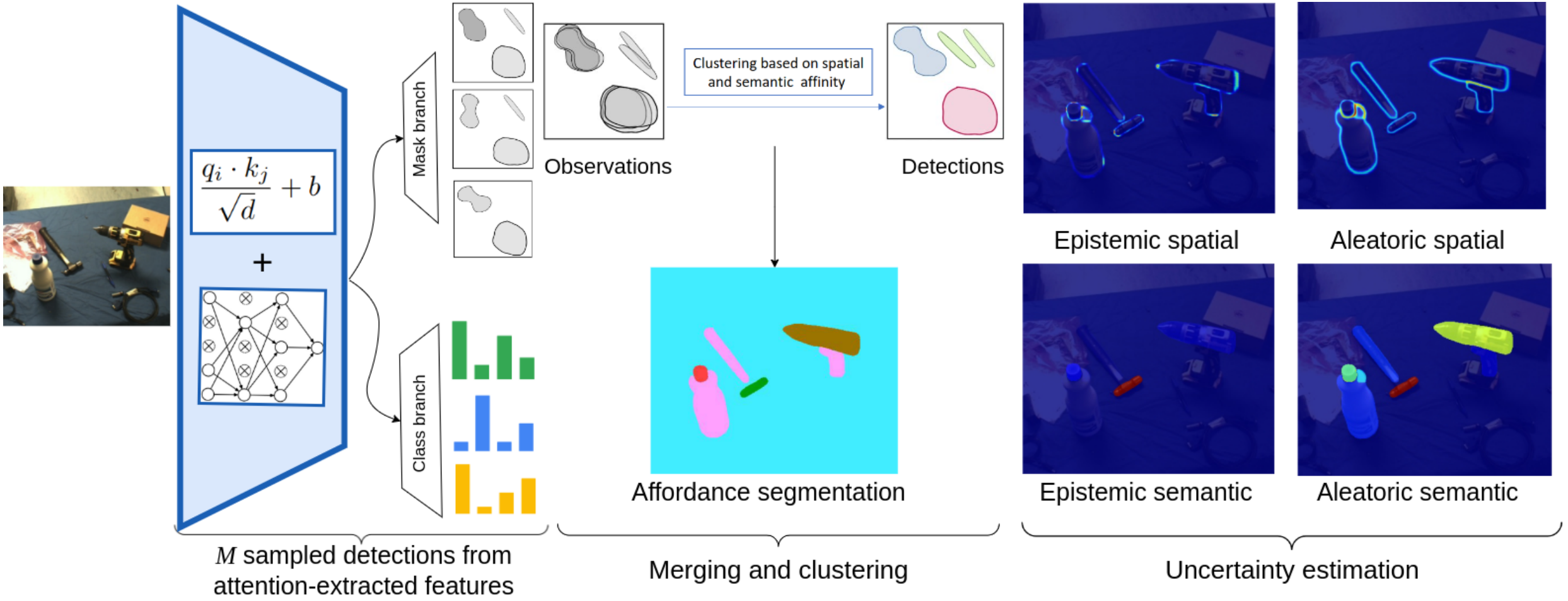
Affordance segmentation



Universidad Zaragoza

Mur-Labadia, Lorenzo, Martinez-Cantin, Ruben and Guerrero, Jose J. (2023) *Bayesian deep learning for affordance segmentation in images.* In Proc. of the IEEE International Conference on Robotics and Automation.

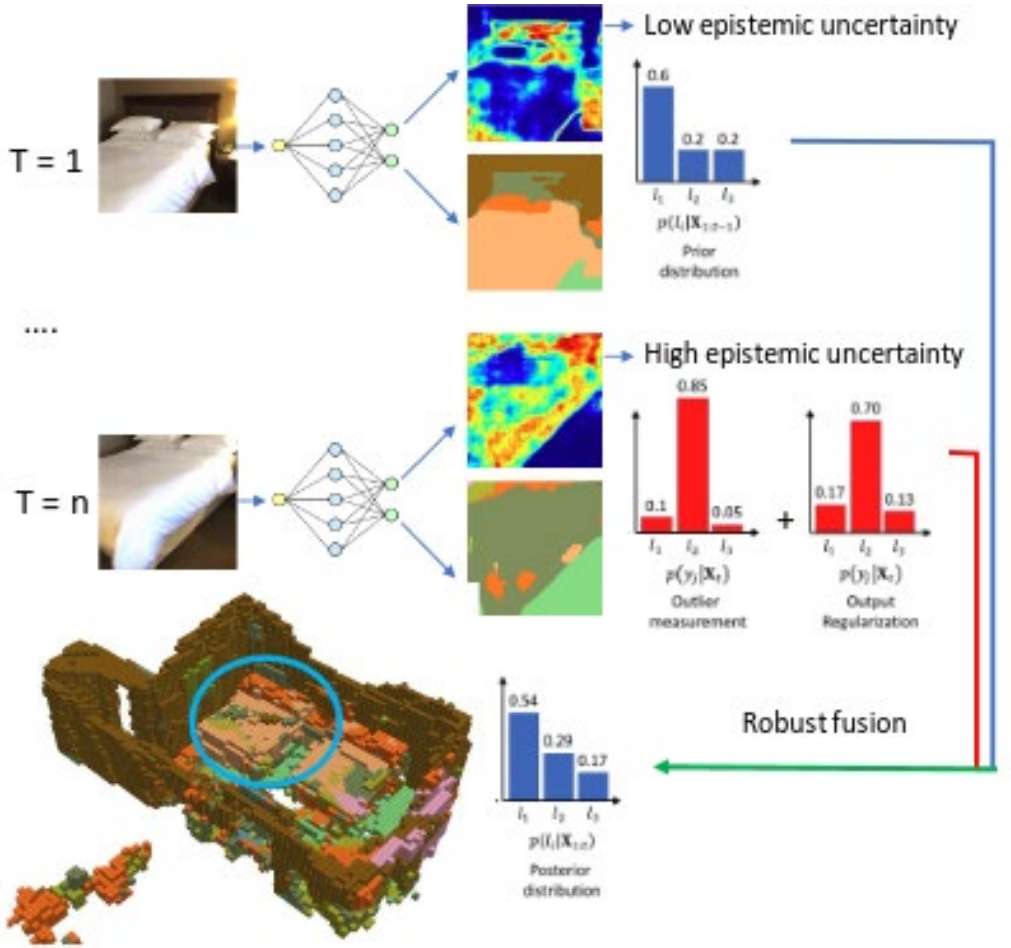
Affordance segmentation



Mur-Labadia, Lorenzo, Martinez-Cantin, Ruben and Guerrero, Jose J. (2023) *Bayesian deep learning for affordance segmentation in images.* In Proc. of the IEEE International Conference on Robotics and Automation.

Data fusión with Bayesian neural networks

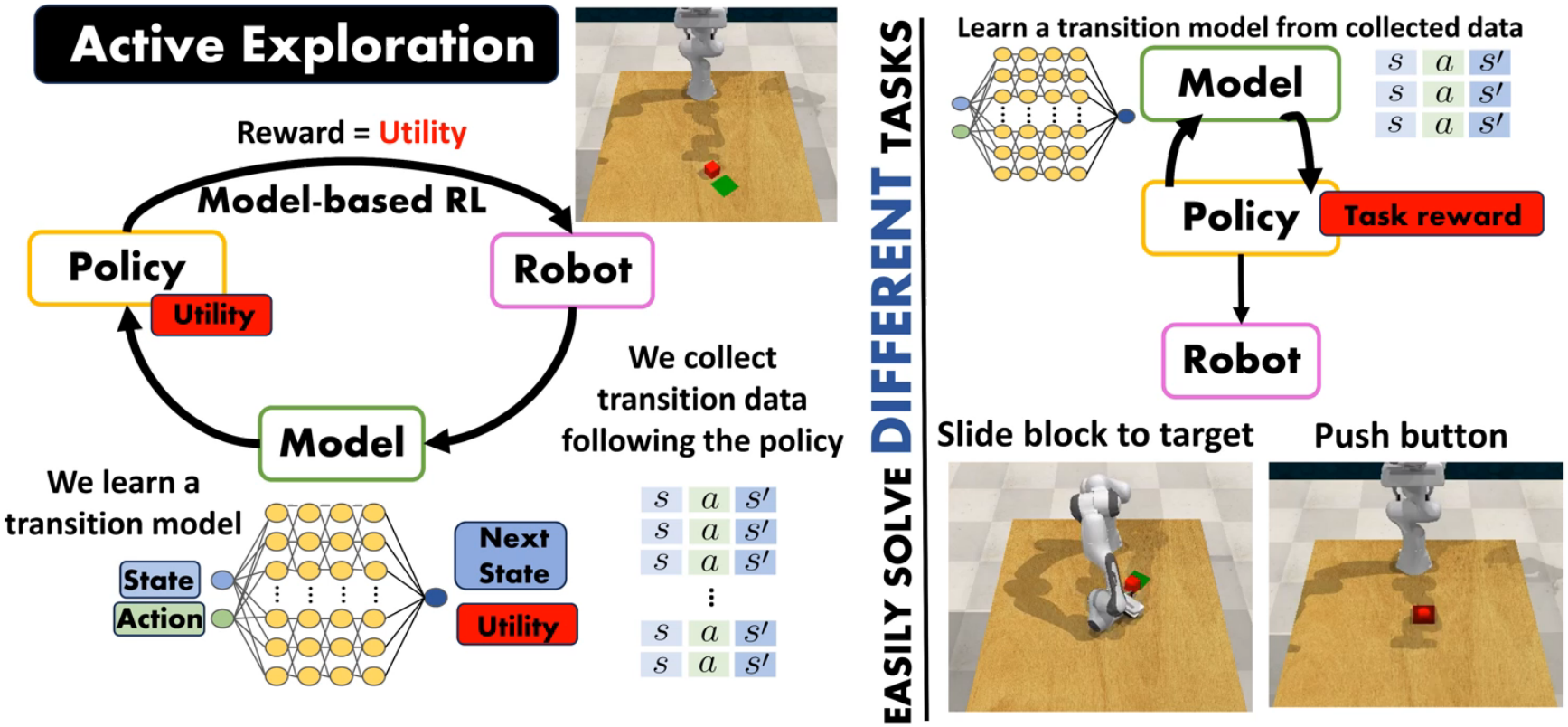
- We can use uncertainty to combine different sources or views.
- Reduce aleatoric uncertainty (noise).
- Epistemic uncertainty measures reliability (e.g.: outliers).



Active Exploration in Bayesian model-based Reinforcement Learning for Robot Manipulation

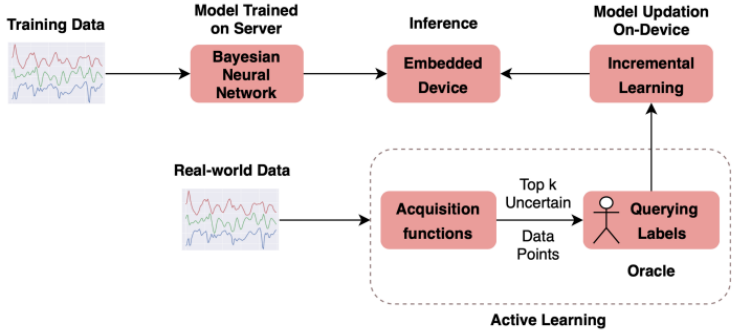
Carlos Plou, Ana C. Murillo, Rubén Martínez-Cantín

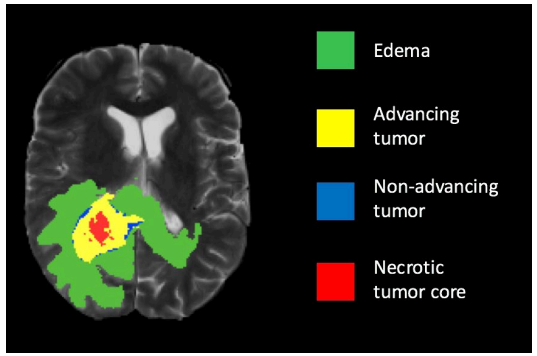
Instituto de Investigación en Ingeniería de Aragón (i3A) and DIIS, University of Zaragoza

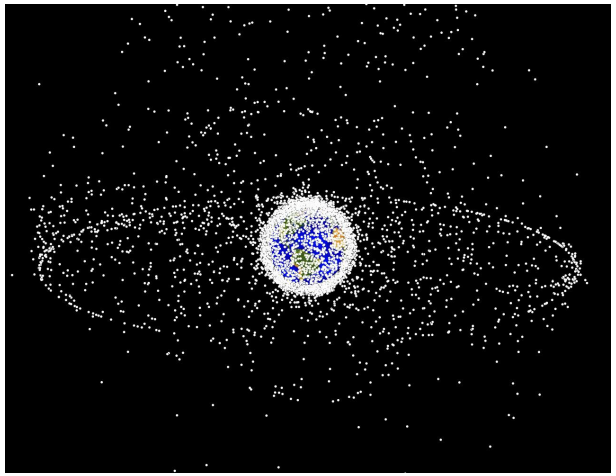


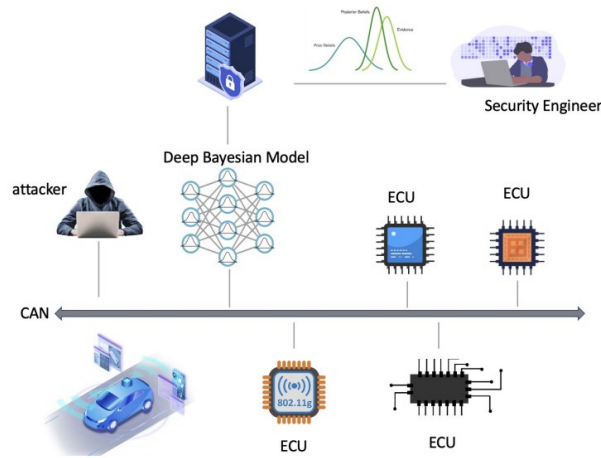
Recent applications

- Fairness in medical imaging
- Spacecraft colission detection
- Precision agriculture
- Car hacking detection
- Active learning for wearable sensors
- …









How to evaluate the uncertainty prediction?

- We do NOT have ground truth for uncertainties!
- We check consistency against the errors:
  - **Ordering**: Larger errors should appear for larger predictive uncertainties
    - Only evaluates relative consistency, not absolute one
  - **Calibration**: The errors should be within the uncertainty boundaries
    - Evaluates the global consistency of the uncertainties
- Types of prediction
  - **Continuous**: For example, regression.
    - Typically, the prediction is a *Gaussian* (μ, σ^2)
  - **Discrete:** For example, classification.
    - The most common prediction is a *categorical distribution*, which has a probability vector, with a value for each class. We can also take the maximum as the predicted class (y) and its probability as the predicted confidence (p).

F. K. Gustafsson, M. Danelljan, T. B. Schon, Evaluating scalable Bayesian deep learning methods for robust computer vision, CVPRW. 2020.
https://openaccess.thecvf.com/content_CVPRW_2020/papers/w20/Gustafsson_Evaluating_Scalable_Bayesian_Deep_Learning_Methods_for_Robust_Computer_Vision_CVPRW_2020_paper.pdf

ECE (calibration, discrete)

- ECE (↓): **E**xpected **C**alibration **E**rror
- Standard to evaluate uncertainty calibration
  - Partition predictions $(\hat{y}_i, \hat{p}_i)$ into M equally spaced bins
  - For each bin, average the difference between accuracy and confidence

$$acc(B_m) = \frac{1}{|B_m|}\sum_{i \in B_m} \mathbf{1}(\hat{y}_i = y_i) \qquad conf(B_m)\ \frac{1}{|B_m|}\sum_{i \in B_m} \hat{p}_i$$

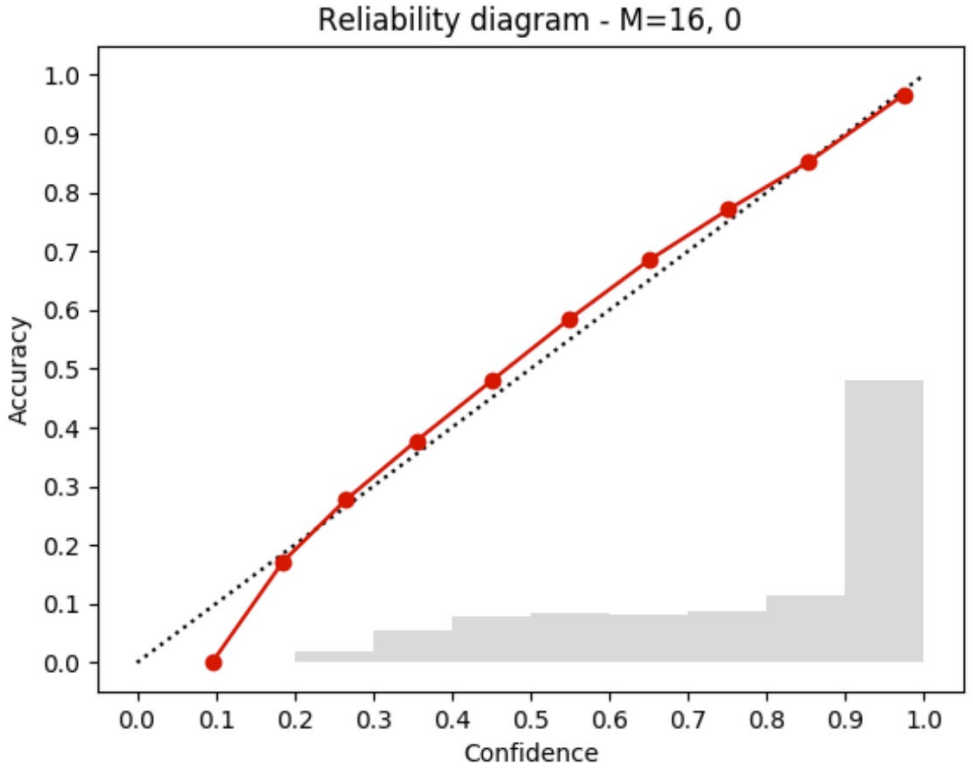
$$ECE = \sum_{m=1}^{M} \frac{|B_m|}{n} |acc(B_m) - conf(B_m)|$$

  - B_m: set of indices that fall into a confidence interval
  - n: number of samples

C. Guo, G. Pleiss, Y. Sun, K. Q. Weinberger, On calibration of modern neural networks, ICML 2017

Reliability diagrams

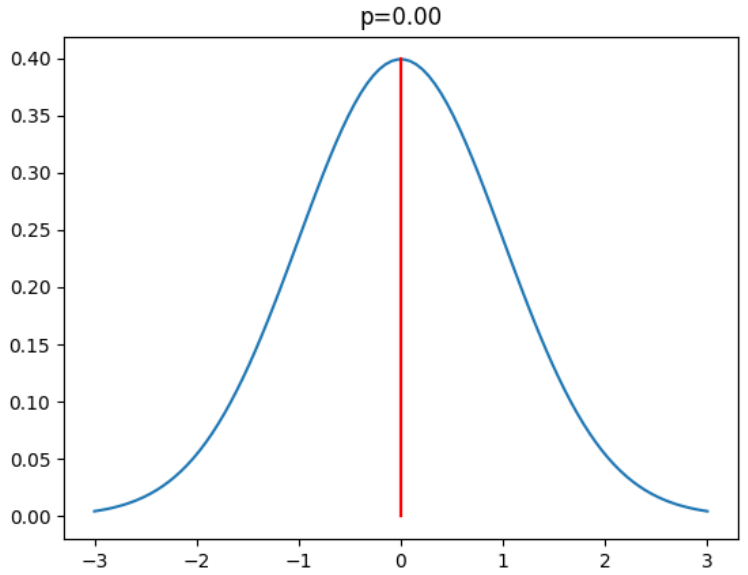
- Plots predicted confidence against accuracy
- Curves deviating from 45° are a sign of miss-calibration

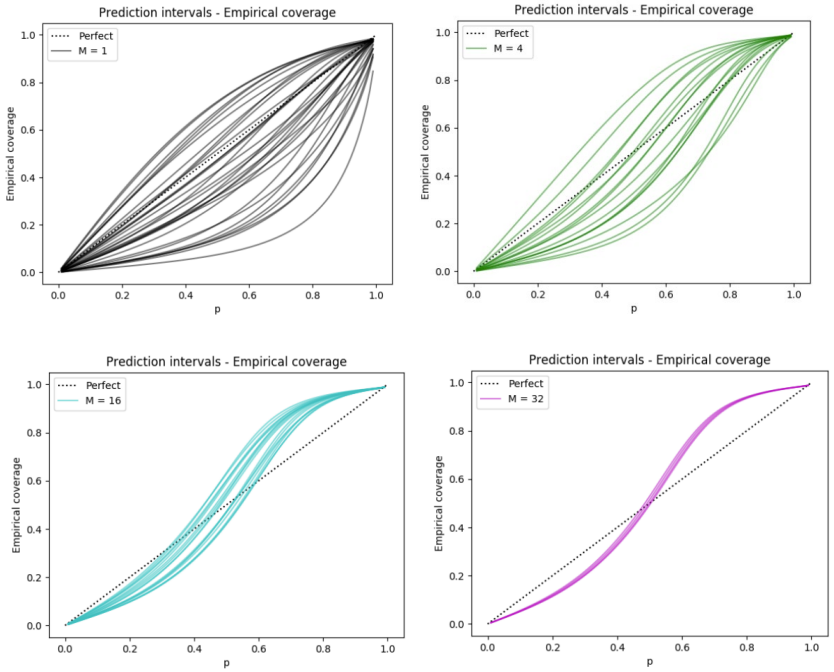


F. K. Gustafsson, M. Danelljan, T. B. Schon, Evaluating scalable Bayesian deep learning methods for robust computer vision, CVPRW. 2020.

AUCE (calibration, continuous)

- AUCE (↓): **A**rea **U**nder the **C**alibration **E**rror curve
- Generalization of ECE to regression setting
  - Assumes output is a Gaussian (μ, σ^2)
- We build the interval $\mu \pm \Phi^{-1}\left(\frac{p+1}{2}\right)\sigma$ for each probability $p \in (0,1)$. Then, we compute the proportion of predictions $\tilde{p}$ for which the true value falls within the interval.
- We compute the area under the curve $|\tilde{p} - p|$ for multiple p
  - In a well calibrated model $|\tilde{p} - p| = 0$.





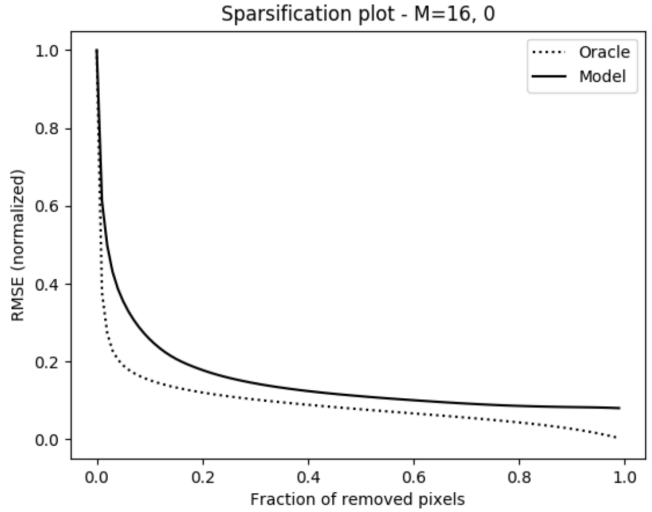
Brier score, $BS(\downarrow)\in[0,1]$

- Strictly proper score function to measure the classification accuracy of probabilistic predictions
  - Strictly proper means that only the true probabilities give the expected maximum score
- Brier score for binary classification: $BS = \frac{1}{N}\sum_{t=1}^{N}(\boldsymbol{f^w}(\boldsymbol{x}_t) - y_t)^2$
- Extension to multiclass: $BS = \frac{1}{N}\sum_{t=1}^{N}\sum_{i=1}^{R}(\boldsymbol{f^w}(\boldsymbol{x}_{ti}) - y_{ti})^2$
  - N: number of samples
  - R: number of classes
- Example: rain/no rain
  - P(rain)=1, rain=true, BS=0
  - P(rain)=0,7, rain= true, BS=(0,7-1)^2=0,09

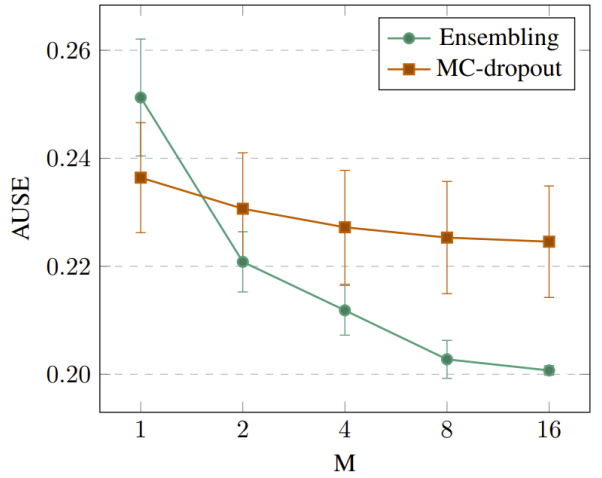
G. W. Brier. Verification of forecasts expressed in terms of probability. Monthly weather review, 1950.

AUSE (ordering)

- AUSE (↓): **A**rea **U**nder the **S**parsification **E**rror curve
- Sparsification error curve:
  - Y-axis: error metric (RMSE, Brier…)
  - X-axis: % removed data
  - Oracle: removes data from highest to lowest errors
  - Model: removes data from highest to lowest uncertainties



What happens if estimated uncertainties are scaled 2x with respect to the actual errors?



AUSE for different number of models, MC-Dropout and Ensembling

F. K. Gustafsson, M. Danelljan, T. B. Schon, Evaluating scalable Bayesian deep learning methods for robust computer vision, CVPRW. 2020.

References

- Main reference:
  - Sergey Levine, Deep Reinforcement Learning, CS 285 at UC Berkeley
    - http://rail.eecs.berkeley.edu/deeprlcourse/

- Other courses:
  - David Silver, Reinforcement Learning, UCL
    - https://www.davidsilver.uk/teaching/
  - Nando de Freitas, Machine learning, Oxford
    - https://www.cs.ox.ac.uk/people/nando.defreitas/machinelearning/

- Books:
  - Sutton & Barto, Reinforcement Learning: An Introduction
    - http://incompleteideas.net/book/the-book-2nd.html
  - Csaba Szepesvári, Algorithms of Reinforcement Learning
    - https://sites.ualberta.ca/~szepesva/rlbook.html

Universidad Zaragoza
1542

Applications of Deep Learning (69158)

Máster in Robotics, Graphics and Computer Vision